

ets_marl - Default RQ Analysis - MAIN

May 10, 2026

1 ETS MARL — Default-Config Research-Question Analysis

This notebook focuses on what the **default configuration** alone can tell us about the thesis research questions. It deliberately does **not** attempt cross-config (variant) comparisons — those will be addressed in a separate experiments notebook.

Research questions (full text in docs/research_questions.md and the top-level README.md):

- **Main RQ** — How can MARL be used to simulate the EU ETS, and what does the simulation reveal about its regulatory mechanisms and the strategic behaviour of power-sector participants?
- **Sub-RQ 1** — What environment / algorithm design choices are required to build a *stable, behaviourally credible* simulation?
- **Sub-RQ 2** — How sensitive are market outcomes (price stability, compliance, decarbonisation pace) to the regulatory mechanisms of the EU ETS? (*Default-only descriptive baseline; cross-variant claims deferred.*)
- **Sub-RQ 3** — How do financial vs environmental objectives shape company strategies, and how do those aggregate into market-level outcomes?

Structure

1. Setup — locate project root.
2. Imports.
3. Multi-sweep loader (allows pulling in **several** default-config sweeps at once — extends the single-spec pattern from ets_marl - Sweep Analysis.ipynb).
4. Discovery & loading.
5. Sanity overview (counts, episode coverage, default-only filter).
6. **§RQ1** — convergence, cross-seed reproducibility, behavioural credibility, pathology.
7. **§RQ2** — within-default regulatory mechanism behaviour (cap → price, MSR / TNAC, penalty incidence, decarbonisation pace).
8. **§RQ3** — strategic behaviour by archetype and reward weighting, aggregated to system-level outcomes.

The notebook is intentionally compact: each section ships the headline plot or table that supports the corresponding RQ claim, and stops there.

```
[106]: # Install notebook dependencies if needed.
# Set INSTALL_NOTEBOOK_DEPS = True (or run the cell with required packages
↳missing)
# to pip-install everything in notebooks/requirements-notebooks.txt using the
```

```

# CURRENT kernel's interpreter. This is more reliable than `!pip install -r
↳ {path}`
# (IPython magics don't expand `{var}` at runtime) and works on Azure / Colab /
↳ local.
import subprocess, sys, importlib, re
from pathlib import Path

INSTALL_NOTEBOOK_DEPS = False # flip to True to force-reinstall

# Map PyPI distribution name -> import module name when they differ.
_DIST_TO_IMPORT = {
    "pyyaml": "yaml",
    "scikit-learn": "sklearn",
    "imbalanced-learn": "imblearn",
    "opencv-python": "cv2",
    "pillow": "PIL",
    "beautifulsoup4": "bs4",
    "ipython": "IPython",
}

def _find_requirements():
    req_name = "requirements-notebooks.txt"
    here = Path.cwd()
    candidates = [here / req_name, here / "notebooks" / req_name]
    candidates.extend(p / "notebooks" / req_name for p in here.parents)
    return next((p for p in candidates if p.exists()), None)

def _parse_requirements(req_path):
    """Extract distribution names from a requirements file (skip comments,
    ↳ options, env markers)."""
    dists = []
    for raw in req_path.read_text(encoding="utf-8").splitlines():
        line = raw.strip()
        if not line or line.startswith("#") or line.startswith("-"):
            continue
        # Strip env markers and extras: "pkg[extra]>=1.0; python_version>='3.9'"
        line = line.split(";", 1)[0].strip()
        m = re.match(r"^([A-Za-z0-9_\.\\-]+)", line)
        if m:
            dists.append(m.group(1).lower())
    return dists

def _missing_required(dists):
    missing = []
    for dist in dists:
        mod = _DIST_TO_IMPORT.get(dist, dist.replace("-", "_"))
        try:

```

```

        importlib.import_module(mod)
    except ImportError:
        missing.append(dist)
    return missing

req_path = _find_requirements()
if req_path is None:
    raise FileNotFoundError("Could not find notebooks/requirements-notebooks.
↳txt")

dists = _parse_requirements(req_path)
missing = _missing_required(dists)

if INSTALL_NOTEBOOK_DEPS or missing:
    if missing:
        print(f"Missing packages: {missing}")
        print(f"Installing notebook dependencies from: {req_path}")
        subprocess.check_call([sys.executable, "-m", "pip", "install", "-r",
↳str(req_path)])
    else:
        print(f"All {len(dists)} notebook dependencies present; skipping install.")
        print("(Set INSTALL_NOTEBOOK_DEPS = True to force a reinstall.)")

```

All 14 notebook dependencies present; skipping install.
(Set INSTALL_NOTEBOOK_DEPS = True to force a reinstall.)

1.1 1. Setup — Locate Project Root

```

[107]: import os, sys
from pathlib import Path

def _is_project_root(path: str) -> bool:
    return os.path.isdir(os.path.join(path, "src")) and os.path.isdir(os.path.
↳join(path, "configs"))

_candidates = ["..", ".", os.path.join("..", ".."),
               os.path.join("Thesis-Energy-Auction", "ets_marl_happo_current"),
↳"Thesis-Energy-Auction"]
_found = _is_project_root(".")
if not _found:
    for _c in _candidates:
        if _is_project_root(_c):
            os.chdir(_c); _found = True; break
if not _found:
    raise FileNotFoundError(f"Cannot find project root (needs src/ + configs/).
↳\nCWD: {os.getcwd()} | Contents: {os.listdir('.')}")

```

```
PROJECT_ROOT = Path.cwd().resolve()
sys.path.insert(0, str(PROJECT_ROOT))
print(f"Working directory: {PROJECT_ROOT}")
```

Working directory: C:\Users\danie\Documents\Python_Scripts\Master Thesis\Thesis-Energy-Auction

1.2 2. Imports

```
[108]: import re
import yaml
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
from src.utils.run_data import load_run_csv, glob_run_logs # parquet-cached
↳ log loader
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style('whitegrid')
pd.set_option('display.max_columns', 200)
pd.set_option('display.width', 200)

from src.utils.sweep import load_sweep_spec, expand_dotted_overrides, deep_merge
```

1.3 3. Sweep Specs to Load

List one or more sweep specs (paths relative to the project root). All of them are loaded and merged into a single dict keyed by (`sweep_name`, `variant_name`, `seed`). The default below loads only `default_seeds.yaml`, but additional default-config sweeps can be appended freely — every run that survives the §5 filter is treated as another seed of the *same* default-config population.

If a sweep contains variants whose `overrides` deviate from the base config, those variants are flagged in §5 and excluded from the analysis by default (this notebook is for the default config only). To include them anyway, set `STRICT_DEFAULT_ONLY = False` in §5.

```
[109]: # Add additional default-config sweeps here as the project produces them.
SWEEP_SPECS = [
    PROJECT_ROOT / 'configs' / 'sweeps' / 'default_seeds.yaml',
    # PROJECT_ROOT / 'configs' / 'sweeps' / 'default_seeds_old.yaml',
    #PROJECT_ROOT / 'configs' / 'sweeps' / 'default_seeds_expanded.yaml'
]

# Optional: override the output_dir from the spec (useful if results were
# moved). One entry per spec, or None to use the spec's own output_dir.
```

```

OUTPUT_DIR_OVERRIDES = [
    PROJECT_ROOT / 'results' / 'sweeps' / 'default',
    # PROJECT_ROOT / 'results' / 'sweeps',
    # PROJECT_ROOT / 'results' / 'sweeps' / 'default'
]
assert len(OUTPUT_DIR_OVERRIDES) == len(SWEEP_SPECS), "one override (or None)␣
↳per spec"

for sp in SWEEP_SPECS:
    if not Path(sp).exists():
        warnings.warn(f"Spec not found: {sp}")

print(f"Loading {len(SWEEP_SPECS)} sweep spec(s):")
for sp, od in zip(SWEEP_SPECS, OUTPUT_DIR_OVERRIDES):
    print(f"  - {sp} (override={od})")

```

Loading 1 sweep spec(s):

```

- C:\Users\danie\Documents\Python_Scripts\Master Thesis\Thesis-Energy-
Auction\configs\sweeps\default_seeds.yaml
(override=C:\Users\danie\Documents\Python_Scripts\Master Thesis\Thesis-Energy-
Auction\results\sweeps\default)

```

1.4 4. Multi-Sweep Discovery & Loading

```

[110]: from src.utils.run_data import log_exists, default_ep_columns,␣
↳default_yr_columns
from tqdm.auto import tqdm

def _resolve_run_files(variant_name, seed, variant_dir, output_dir):
    """Return (training_log, year_log) paths or (None, None). Tagged then␣
↳untagged."""
    search_dirs = []

    # Existing locations
    for d in (variant_dir, output_dir):
        if d is not None:
            d = Path(d)
            if d not in search_dirs:
                search_dirs.append(d)

    # Extra fallback: results/sweep/<tag>[/<variant>]
    # (tag inferred from output_dir name, e.g. default_seeds)
    if output_dir is not None:
        tag = Path(output_dir).name
        alt_base = PROJECT_ROOT / "results" / "sweep" / tag
        for d in (alt_base / variant_name, alt_base):

```

```

        if d not in search_dirs:
            search_dirs.append(d)

    training_names = [f"training_log_{variant_name}_s{seed}.csv",
↪f"training_log_s{seed}.csv"]
    year_names = [f"year_log_{variant_name}_s{seed}.csv", f"year_log_s{seed}.
↪csv"]

    tlog = ylog = None
    for base in search_dirs:
        if tlog is None:
            for n in training_names:
                cand = base / n
                if log_exists(cand):
                    tlog = cand
                    break
        if ylog is None:
            for n in year_names:
                cand = base / n
                if log_exists(cand):
                    ylog = cand
                    break
        if tlog is not None and ylog is not None:
            break

    return tlog, ylog

# Project both logs onto the analysis-superset columns at load time. The
# year_log carries 270+ diagnostic columns; pulling only the ~80 actually
# used by this notebook cuts in-RAM size by ~5-10x. Unknown columns are
# silently dropped by load_run_csv.
EP_COLUMNS = default_ep_columns()
YR_COLUMNS = default_yr_columns()

def _load_csv_or_none(path, columns=None):
    if path is None: return None
    return load_run_csv(path, columns=columns)

def _empty_ep(yr=None):
    if yr is not None and 'episode' in yr.columns:
        return pd.DataFrame({'episode': np.sort(yr['episode']).dropna().
↪unique())})
    return pd.DataFrame({'episode': pd.Series(dtype='int64')})

def _empty_yr(ep=None):
    if ep is not None and 'episode' in ep.columns and len(ep) > 0:
        return pd.DataFrame({'episode': ep['episode'].astype('int64').values,

```

```

        'year': np.zeros(len(ep), dtype='int64'))}
    return pd.DataFrame({'episode': pd.Series(dtype='int64'), 'year': pd.
↳Series(dtype='int64')})

specs = []
for spec_path, override in zip(SWEEP_SPECS, OUTPUT_DIR_OVERRIDES):
    if not Path(spec_path).exists():
        continue
    spec = load_sweep_spec(str(spec_path))
    sweep_name = Path(spec_path).stem
    with open(spec['base_config'], encoding='utf-8') as f:
        base_cfg = yaml.safe_load(f)
    output_dir = Path(override) if override is not None else (PROJECT_ROOT /
↳spec['output_dir']).resolve()
    specs.append({'name': sweep_name, 'spec': spec, 'base_cfg': base_cfg,
↳'output_dir': output_dir})

# Pre-enumerate every (sweep, variant, seed) we'll attempt so tqdm has a real
↳total.
load_jobs = []
for s in specs:
    for v in s['spec']['variants']:
        seeds = v['seeds'] if v['seeds'] is not None else s['spec']['seeds']
        variant_dir = s['output_dir'] / v['name']
        merged_cfg = deep_merge(s['base_cfg'], expand_dotted_overrides(v.
↳get('overrides') or {}))
        flat_overrides = expand_dotted_overrides(v.get('overrides') or {})
        for seed in seeds:
            load_jobs.append((s, v, seed, variant_dir, merged_cfg,
↳flat_overrides))

# Build (sweep, variant, seed) → run dict.
runs = {}
discovery = []
for s, v, seed, variant_dir, merged_cfg, flat_overrides in tqdm(
    load_jobs, desc='Loading sweep CSVs', unit='run', leave=True
):
    tlog, ylog = _resolve_run_files(v['name'], seed, variant_dir,
↳s['output_dir'])
    row = {
        'sweep': s['name'], 'variant': v['name'], 'seed': seed,
        'has_overrides': bool(flat_overrides),
        'training_log': str(tlog) if tlog else None,
        'year_log': str(ylog) if ylog else None,
        'has_training_log': tlog is not None,
        'has_year_log': ylog is not None,

```

```

        'found': (tlog is not None) or (ylog is not None),
    }
    discovery.append(row)
    if not row['found']:
        continue
    ep_df = _load_csv_or_none(tlog, columns=EP_COLUMNS)
    yr_df = _load_csv_or_none(ylog, columns=YR_COLUMNS)
    if ep_df is None and yr_df is None:
        continue
    if ep_df is None: ep_df = _empty_ep(yr_df)
    if yr_df is None: yr_df = _empty_yr(ep_df)
    runs[(s['name'], v['name'], seed)] = {
        'ep_df': ep_df, 'yr_df': yr_df, 'config': merged_cfg,
        'has_overrides': bool(flat_overrides),
        'overrides': flat_overrides,
        'has_training_log': bool(tlog), 'has_year_log': bool(ylog),
    }

discovery_df = pd.DataFrame(discovery)
n_found = int(discovery_df['found'].sum()) if not discovery_df.empty else 0
print(f"Discovered {len(discovery_df)} (sweep, variant, seed) triples; loaded_{n_found}")
if not runs:
    raise RuntimeError("No runs loaded. Check SWEEP_SPECS / OUTPUT_DIR_OVERRIDES.")
discovery_df

# Free chunked CSV/parquet readers held by pandas/pyarrow before continuing.
import gc; gc.collect()

```

Loading sweep CSVs: 0%| | 0/4 [00:00<?, ?run/s]

Discovered 4 (sweep, variant, seed) triples; loaded 4.

[110]: 280

1.5 5. Default-Only Filter & Sanity Overview

[111]: STRICT_DEFAULT_ONLY = True # if True, drop runs whose variant has any override.

```

if STRICT_DEFAULT_ONLY:
    n_before = len(runs)
    runs = {k: v for k, v in runs.items() if not v['has_overrides']}
    n_after = len(runs)
    if n_after < n_before:
        print(f"Filtered out {n_before - n_after} run(s) whose variant_
        overrides the base config.")

```



```

    if n_after == 0:
        raise RuntimeError("STRICT_DEFAULT_ONLY filtered out every run. Set_
↳STRICT_DEFAULT_ONLY = False to inspect non-default variants.")

# Treat every surviving run as a default-config seed. Build a flat list.
ALL_RUN_KEYS = sorted(runs.keys())
print(f"\nUsable default-config runs: {len(ALL_RUN_KEYS)}")
for (sw, var, seed), run in sorted(runs.items()):
    n_eps = len(run['ep_df'])
    n_yrs = len(run['yr_df'])
    print(f"  {sw}/{var}/seed={seed:>4}  ({n_eps:>6} episodes, {n_yrs:>7}
↳year-rows)")

# Pick a reference config. All defaults agree on schema; just take the first.
REFERENCE_CONFIG = runs[ALL_RUN_KEYS[0]]['config']
N_AGENTS = int(REFERENCE_CONFIG.get('companies', {}).get('n_agents', 8))
N_BOTS = int(REFERENCE_CONFIG.get('companies', {}).get('n_bot_agents', 0))
N_TOTAL = N_AGENTS + N_BOTS
N_YEARS = int(REFERENCE_CONFIG.get('environment', {}).get('n_years', 12))
print(f"\nReference config: n_agents={N_AGENTS}, n_bot_agents={N_BOTS},
↳n_years={N_YEARS}")

# Archetype/reward labelling (default config: 4 archetypes × {financial, ESG}).
ARCHETYPES = ['Coal-heavy', 'Gas-dominant', 'Transitioner', 'Green-leader']
def _agent_archetype(idx0):
    return ARCHETYPES[idx0 // 2] if idx0 // 2 < len(ARCHETYPES) else
↳f'Custom-{idx0//2}'
def _agent_weighting(idx0):
    # Even indices (A1, A3, ...) are pure financial; odd are balanced ESG. (Per_
↳README.)
    return 'Financial' if idx0 % 2 == 0 else 'Balanced ESG'

AGENT_LABELS = [f'A{i+1} ({_agent_archetype(i)}, {_agent_weighting(i)})' for i_
↳in range(N_AGENTS)]
print("Agent roster (1-based):")
for lbl in AGENT_LABELS:
    print(f"  {lbl}")

```

Usable default-config runs: 4

default_seeds/default/seed=1729	(120000 episodes, 1440000 year-rows)
default_seeds/default/seed=5041	(120000 episodes, 1440000 year-rows)
default_seeds/default/seed=6561	(120000 episodes, 1440000 year-rows)
default_seeds/default/seed=8191	(120000 episodes, 1440000 year-rows)

Reference config: n_agents=8, n_bot_agents=0, n_years=12

Agent roster (1-based):

```

A1 (Coal-heavy, Financial)
A2 (Coal-heavy, Balanced ESG)
A3 (Gas-dominant, Financial)
A4 (Gas-dominant, Balanced ESG)
A5 (Transitioner, Financial)
A6 (Transitioner, Balanced ESG)
A7 (Green-leader, Financial)
A8 (Green-leader, Balanced ESG)

```

1.5.1 5a. Helpers

A handful of helpers reused across §RQ1–§RQ3. We intentionally treat every loaded run as one seed of the same default-config population (no variant axis) — the helpers therefore aggregate over ALL_RUN_KEYS without any variant grouping.

```

[112]: ROLL = 50                # episode-axis smoothing window
TAIL_EPISODES = 5000          # converged-window definition for steady-state metrics

def agent_cols(df, prefix, max_idx=N_AGENTS):
    """Return columns 'prefix_A1' ... 'prefix_A{max_idx}' that exist in df,
    ↪sorted by agent index."""
    rx = re.compile(rf'{re.escape(prefix)}_A(\d+)\$')
    out = []
    for c in df.columns:
        m = rx.match(c)
        if m and int(m.group(1)) <= max_idx:
            out.append((int(m.group(1)), c))
    return [c for _, c in sorted(out)]

def per_run_series(metric_fn, source='ep'):
    """Return dict {run_key: pd.Series indexed by episode} from each run."""
    out = {}
    for k, run in runs.items():
        df = run['yr_df'] if source == 'yr' else run['ep_df']
        if source == 'run':
            s = metric_fn(run)
        else:
            s = metric_fn(df)
        if s is None or len(s) == 0:
            continue
        s = s.copy()
        if source == 'ep' and 'episode' in run['ep_df'].columns:
            s.index = run['ep_df']['episode']
        out[k] = s
    return out

def stack_runs(metric_fn, source='ep'):
    """Return wide DataFrame of episode-indexed series, one column per run."""

```

```

series = per_run_series(metric_fn, source=source)
if not series:
    return pd.DataFrame()
return pd.concat(series, axis=1).sort_index()

def fan_chart(df, ax, color='steelblue', label='runs', smooth=ROLL):
    """Plot median + ±1 std fan + thin per-run lines for a wide DataFrame."""
    if df.empty:
        ax.text(0.5, 0.5, 'no data', ha='center', va='center', transform=ax.
↳transAxes)
        return
    sm = df.rolling(smooth, min_periods=1).mean()
    median = sm.median(axis=1)
    std = sm.std(axis=1)
    for col in sm.columns:
        ax.plot(sm.index, sm[col].values, color=color, lw=0.6, alpha=0.30)
    ax.fill_between(sm.index, (median - std).values, (median + std).values,
                    color=color, alpha=0.20, label=f'{label} ±1 std (n={sm.
↳shape[1]})')
    ax.plot(sm.index, median.values, color=color, lw=2.2, label=f'{label}↳
↳median')
    ax.grid(True, alpha=0.3); ax.legend(fontsize=8, loc='best')

def tail_episodes(ep_df, n=TAIL_EPISODES):
    if ep_df.empty: return ep_df
    return ep_df.tail(min(n, len(ep_df)))

def tail_year_rows(run, n=TAIL_EPISODES):
    ep = tail_episodes(run['ep_df'], n)
    if ep.empty or 'episode' not in run['yr_df'].columns:
        return run['yr_df'].head(0)
    return run['yr_df'][run['yr_df']['episode'].isin(ep['episode'].unique())]

```

1.6 §RQ1 — Stable & Behaviourally Credible Simulation

Three sub-checks:

1. **Convergence** — system reward, composite quality score, and PPO actor/critic losses settle by training end.
2. **Cross-seed reproducibility** — fan charts on headline metrics across loaded seeds; the spread of converged-window means quantifies how reliably the same equilibrium is reached.
3. **Behavioural credibility** — final-window clearing price sits in the EU-ETS-plausible range (~€60–110 per the README’s design notes), compliance rate plateaus high, and warning-flag counts in the converged window are not pathological.

1.6.1 RQ1.1 — Convergence dynamics (rewards, quality score, losses)

```
[113]: def _system_reward(df):
        cols = agent_cols(df, 'reward')
        return df[cols].sum(axis=1) if cols else pd.Series(dtype=float)

def _system_base_reward(df):
    cols = agent_cols(df, 'reward_base')
    return df[cols].sum(axis=1) if cols else pd.Series(dtype=float)

def _quality(df):
    return df['quality_score'] if 'quality_score' in df.columns else pd.
    ↪Series(dtype=float)

def _mean_actor_loss(df):
    cols = agent_cols(df, 'actor_loss')
    return df[cols].mean(axis=1) if cols else pd.Series(dtype=float)

def _mean_critic_loss(df):
    cols = agent_cols(df, 'critic_loss')
    return df[cols].mean(axis=1) if cols else pd.Series(dtype=float)

fig, axes = plt.subplots(2, 2, figsize=(16, 9))
fan_chart(stack_runs(_system_reward), axes[0, 0], color='steelblue',
    ↪label='system reward')
axes[0, 0].set(title='System total reward ( $\Sigma$  reward_A*)', xlabel='Episode',
    ↪ylabel='reward')

fan_chart(stack_runs(_quality), axes[0, 1], color='seagreen',
    ↪label='quality_score')
axes[0, 1].set(title='Composite quality score', xlabel='Episode', ylabel='Q
    ↪[-5, 5]')
axes[0, 1].set_ylim(0, 3)

fan_chart(stack_runs(_mean_actor_loss), axes[1, 0], color='firebrick',
    ↪label='actor loss')
axes[1, 0].set(title='Mean actor loss across agents', xlabel='Episode',
    ↪ylabel='loss')

fan_chart(stack_runs(_mean_critic_loss), axes[1, 1], color='darkorange',
    ↪label='critic loss')
axes[1, 1].set(title='Mean critic loss across agents', xlabel='Episode',
    ↪ylabel='loss')

plt.suptitle('RQ1.1 - Training-time convergence (median  $\pm$ 1 std across loaded
    ↪seeds)',
             fontsize=13, y=1.02)
```

```
plt.tight_layout(); plt.show()
```

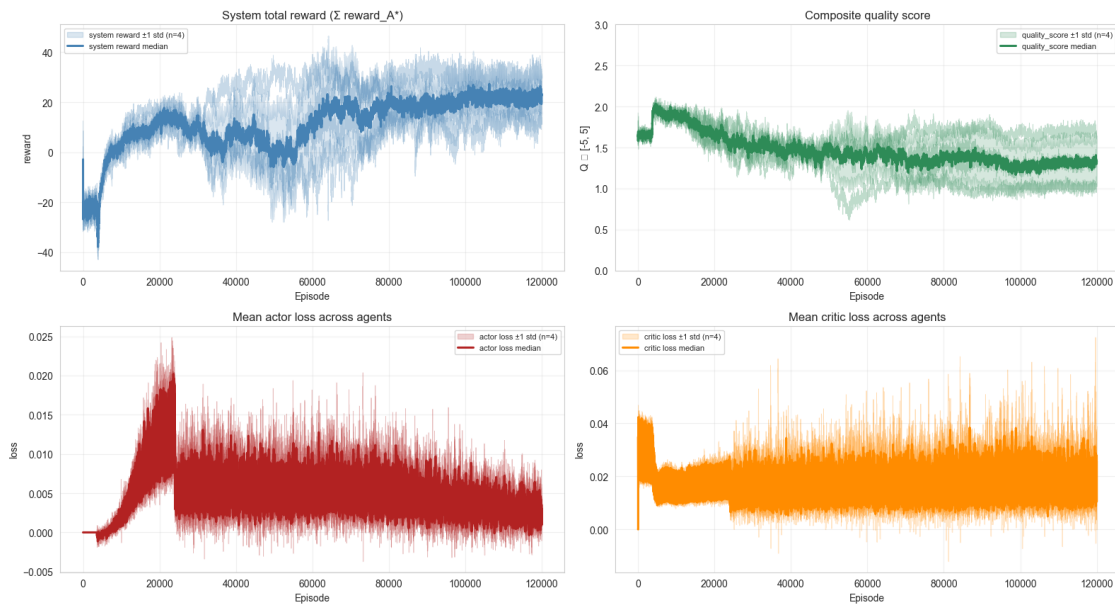
C:\Users\danie\AppData\Local\Temp\ipykernel_23912\3832536781.py:36: UserWarning: Glyph 8712 (\N{ELEMENT OF}) missing from font(s) Arial.

```
plt.tight_layout(); plt.show()
```

C:\Users\danie\AppData\Roaming\Python\Python312\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8712 (\N{ELEMENT OF}) missing from font(s) Arial.

```
fig.canvas.print_figure(bytes_io, **kw)
```

RQ1.1 — Training-time convergence (median ± 1 std across loaded seeds)



1.6.2 RQ1.2 — Cross-seed reproducibility on headline metrics

```
[114]: # Per-seed converged-window means for a small set of headline metrics.
def _converged_metrics_for(run):
    ep = tail_episodes(run['ep_df'])
    yr = tail_year_rows(run)
    out = {}

    rew_cols = agent_cols(ep, 'reward')
    out['system_reward'] = float(ep[rew_cols].sum(axis=1).mean()) if
↪rew_cols else np.nan
    out['quality_score'] = float(ep['quality_score'].mean()) if
↪'quality_score' in ep.columns else np.nan

    if 'clearing_price' in yr.columns:
        out['mean_clearing_price'] = float(yr['clearing_price'].mean())
```

```

elif 'clearing_price_last' in ep.columns:
    out['mean_clearing_price'] = float(ep['clearing_price_last'].mean())
else:
    out['mean_clearing_price'] = np.nan

sf_cols = agent_cols(yr, 'shortfall')
out['compliance_rate'] = float((yr[sf_cols] <= 1e-6).mean(axis=1).mean())
if sf_cols else np.nan

pen_cols = agent_cols(ep, 'penalty')
out['system_penalty'] = float(ep[pen_cols].sum(axis=1).mean()) if pen_cols
else np.nan

inv_cols = agent_cols(ep, 'invest_cost')
out['system_invest'] = float(ep[inv_cols].sum(axis=1).mean()) if inv_cols
else np.nan

gf_cols = agent_cols(ep, 'green_frac')
out['mean_final_green_frac'] = float(ep[gf_cols].mean(axis=1).mean()) if gf_cols
else np.nan

if 'secondary_volume' in ep.columns:
    out['secondary_volume'] = float(ep['secondary_volume'].mean())
else:
    out['secondary_volume'] = np.nan

return out

per_seed_rows = []
for k, run in runs.items():
    row = {'sweep': k[0], 'variant': k[1], 'seed': k[2]}
    row.update(_converged_metrics_for(run))
    per_seed_rows.append(row)
per_seed_df = pd.DataFrame(per_seed_rows)
print(f"Per-seed converged-window means (last {TAIL_EPISODES} episodes):")
display(per_seed_df.round(3))

agg_cols = ['system_reward', 'quality_score', 'mean_clearing_price',
            'compliance_rate',
            'system_penalty', 'system_invest', 'mean_final_green_frac',
            'secondary_volume']
summary = per_seed_df[agg_cols].agg(['mean', 'std', 'min', 'max']).round(3)
summary.loc['cv (%)'] = (summary.loc['std'] / summary.loc['mean'].abs() * 100).round(1)
print(f"\nCross-seed summary (n={len(per_seed_df)}):")
summary

```

Per-seed converged-window means (last 5000 episodes):

	sweep	variant	seed	system_reward	quality_score	mean_clearing_price	compliance_rate	system_penalty	system_invest	mean_final_green_frac	secondary_volume
0	default_seeds	default	1729	18.922	1.716	68.					
	↵606	0.874	1928.342	5157.227	0.735						
	↵	44.044									
1	default_seeds	default	8191	30.526	1.319	47.					
	↵860	0.850	2623.184	6430.930	0.765						
	↵	39.854									
2	default_seeds	default	6561	10.914	1.325	57.					
	↵204	0.791	3916.688	6815.107	0.771						
	↵	27.048									
3	default_seeds	default	5041	24.533	1.011	46.					
	↵418	0.786	4282.045	5129.795	0.747						
	↵	29.961									

Cross-seed summary (n=4):

```
[114]:
```

	system_reward	quality_score	mean_clearing_price	compliance_rate
system_penalty	system_invest	mean_final_green_frac	secondary_volume	
mean	21.224	1.343	55.022	0.825
3187.565	5883.265	0.754	35.227	
std	8.348	0.289	10.241	0.044
1100.557	868.546	0.016	8.037	
min	10.914	1.011	46.418	0.786
1928.342	5129.795	0.735	27.048	
max	30.526	1.716	68.606	0.874
4282.045	6815.107	0.771	44.044	
cv (%)	39.300	21.500	18.600	5.300
34.500	14.800	2.100	22.800	

Reading the table. A coefficient of variation (cv (%)) below ~10% on the headline outcomes (mean clearing price, compliance rate, mean final green fraction) is the bar we use to call the simulation *reproducible* across seeds — anything significantly larger means the RQ2 / RQ3 claims need to be hedged seed-by-seed.

1.6.3 RQ1.3 — Behavioural credibility in the converged window

```
[115]: # (a) Final-window clearing-price distribution; (b) per-seed compliance plateau.
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

ax = axes[0]
boxes = []
labels = []
for k, run in runs.items():
```

```

yr = tail_year_rows(run)
if 'clearing_price' not in yr.columns or yr.empty:
    continue
boxes.append(yr['clearing_price'].dropna().values)
labels.append(f"{k[0][:8]}/s{k[2]}")
if boxes:
    ax.boxplot(boxes, tick_labels=labels, showfliers=False)
    ax.axhspan(60, 110, color='green', alpha=0.10, label='EU-ETS plausible band_
↳( €60-110/t)')
    ax.set(title='Converged-window clearing price by seed', ylabel='€/tCO ',
↳xlabel='Run')
    ax.legend(fontsize=8); ax.tick_params(axis='x', rotation=30)
else:
    ax.text(0.5, 0.5, 'No clearing_price column in any year_log', ha='center',
↳va='center', transform=ax.transAxes)

ax = axes[1]
for k, run in runs.items():
    sf_cols = agent_cols(run['yr_df'], 'shortfall')
    if not sf_cols or 'episode' not in run['yr_df'].columns:
        continue
    rate = (run['yr_df'][sf_cols] <= 1e-6).mean(axis=1).
↳groupby(run['yr_df']['episode']).mean()
    rate_smooth = rate.rolling(ROLL, min_periods=1).mean()
    ax.plot(rate_smooth.index, rate_smooth.values, lw=1.0, alpha=0.85,
        label=f"{k[0][:8]}/s{k[2]}")
ax.set(title='Compliance rate over training (mean across (agent, year) cells)',
    xlabel='Episode', ylabel='compliance rate [0,1]')
ax.set_ylim(0.0, 1.02); ax.grid(True, alpha=0.3); ax.legend(fontsize=8,
↳loc='best')

plt.suptitle('RQ1.3 - Behavioural credibility: realistic price band +
↳compliance plateau',
    fontsize=13, y=1.02)
plt.tight_layout(); plt.show()

```

C:\Users\danie\AppData\Local\Temp\ipykernel_23912\2922376889.py:36: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

plt.tight_layout(); plt.show()

C:\Users\danie\AppData\Local\Temp\ipykernel_23912\2922376889.py:36: UserWarning: Glyph 8712 (\N{ELEMENT OF}) missing from font(s) Arial.

plt.tight_layout(); plt.show()

C:\Users\danie\AppData\Roaming\Python\Python312\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

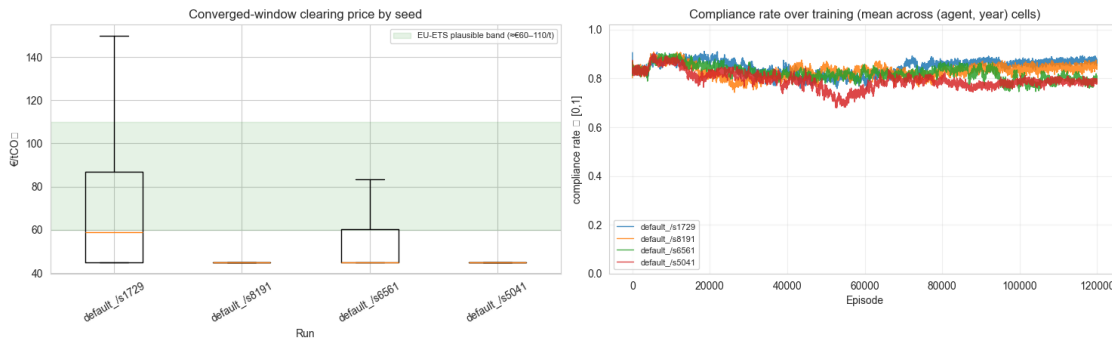
fig.canvas.print_figure(bytes_io, **kw)

C:\Users\danie\AppData\Roaming\Python\Python312\site-

packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8712 (\N{ELEMENT OF}) missing from font(s) Arial.

```
fig.canvas.print_figure(bytes_io, **kw)
```

RQ1.3 — Behavioural credibility: realistic price band + compliance plateau



1.6.4 RQ1.4 — Pathology check: warnings in the converged window

```
[116]: # Sum the warn_* columns across the converged window for each run.
WARN_PREFIX = 'warn_'
warn_rows = []
warn_columns_seen = set()
for k, run in runs.items():
    ep = tail_episodes(run['ep_df'])
    warn_cols = [c for c in ep.columns if c.startswith(WARN_PREFIX)]
    warn_columns_seen.update(warn_cols)
    if not warn_cols:
        continue
    row = {'run': f"{k[0][:8]}/s{k[2]}"}
    # Per-episode mean warning count (interpretable as "how often per episode
    # does this warn fire").
    row.update({c: float(ep[c].mean()) for c in warn_cols})
    warn_rows.append(row)

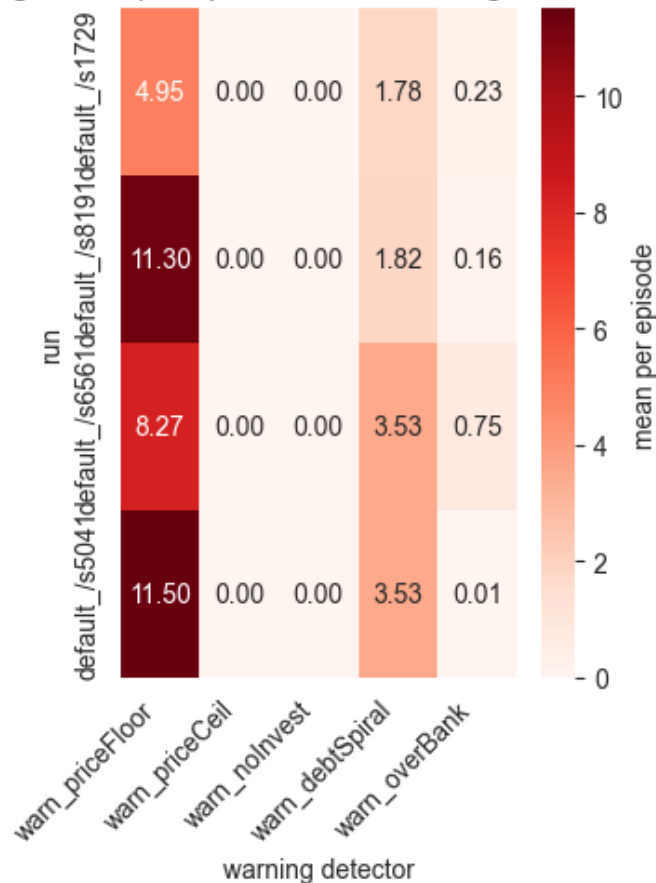
if warn_rows:
    warn_df = pd.DataFrame(warn_rows).set_index('run').fillna(0.0)
    # Drop columns that are exactly zero across all runs.
    warn_df = warn_df.loc[:, (warn_df > 0).any(axis=0)]
    fig, ax = plt.subplots(figsize=(min(14, 1.5 + 0.45 * len(warn_df.columns)),
    4 + 0.3 * len(warn_df)))
    if warn_df.empty:
        ax.text(0.5, 0.5, 'No non-zero warnings in the converged window - clean
run.',
                ha='center', va='center', transform=ax.transAxes, fontsize=12)
    ax.axis('off')
```

```

else:
    sns.heatmap(warn_df.round(2), annot=True, cmap='Reds', fmt='.2f',
    ↳cbar_kws={'label': 'mean per episode'}, ax=ax)
    ax.set(title=f'Mean warning-flag counts per episode in the converged_
    ↳window (last {TAIL_EPISODES} eps)',
           xlabel='warning detector', ylabel='run')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout(); plt.show()
    print(f'Inspected {len(warn_columns_seen)} warning columns; showing only_
    ↳the {warn_df.shape[1]} with > 0 events.')
else:
    print('No warn_* columns present in episode logs.')

```

Mean warning-flag counts per episode in the converged window (last 5000 eps)



Inspected 6 warning columns; showing only the 5 with > 0 events.

1.6.5 RQ1.5 — Whole-training overview (all loaded runs)

A single 2×2 panel summarising the entire training trajectory across *every* loaded run (i.e. before the §RQ2 default-seeds filter kicks in). Top-left: system reward; top-right: final-year clearing price (per-episode `clearing_price` at the maximum simulation year); bottom-left: compliance rate; bottom-right: mean green generation share. The shaded band on each panel is the **min/max envelope across the loaded runs** at every smoothed episode (i.e. it is bounded by the data and cannot ever cross the auction reserve price), with the solid line as the cross-run median.

The clearing-price panel adds two horizontal references taken from `configs/default.yaml`: the auction reserve (floor) price and the final-year penalty rate (`penalty.rate × (1 + inflation_rate)^(N_YEARS - 1)`). The green-share panel marks the cross-agent mean of the *initial* (year-0) green fractions read from `companies.initial_mix × technologies.is_green`. Compliance and green-share use a tight padded y-axis so changes during training are easy to read.

```
[147]: # 2x2 whole-training overview across all loaded runs.
# Episode-axis fan charts (median + min/max band across runs, smoothed by ROLL).

def _final_year_clearing_by_ep(run):
    """Per-episode clearing price in the final simulation year."""
    yr = run['yr_df']
    needed = {'clearing_price', 'episode', 'year'}
    if not needed.issubset(yr.columns):
        return pd.Series(dtype=float)
    idx = yr.groupby('episode')['year'].idxmax()
    s = yr.loc[idx].set_index('episode')['clearing_price']
    return s.sort_index()

def _compliance_rate_by_ep(run):
    yr = run['yr_df']
    sf_cols = agent_cols(yr, 'shortfall')
    if not sf_cols or 'episode' not in yr.columns:
        return pd.Series(dtype=float)
    return (yr[sf_cols] <= 1e-6).mean(axis=1).groupby(yr['episode']).mean()

def _mean_green_frac_by_ep(df):
    cols = agent_cols(df, 'green_frac')
    return df[cols].mean(axis=1) if cols else pd.Series(dtype=float)

# Year-0 (start-of-simulation) mean green fraction from the default config.
_mix = REFERENCE_CONFIG.get('companies', {}).get('initial_mix', []) or []
_is_green = REFERENCE_CONFIG.get('technologies', {}).get('is_green', []) or []
_green_idx = [i for i, g in enumerate(_is_green) if g]
if _mix and _green_idx:
    _agent_start_green = [sum(row[i] for i in _green_idx) for row in _mix[:
↪N_AGENTS]]
    START_GREEN_FRAC = float(np.mean(_agent_start_green))
else:
```

```

START_GREEN_FRAC = float('nan')

def _minmax_fan(df_wide, ax, *, color, label, smooth=ROLL):
    """Plot median + min/max band (across runs) of `df_wide` on `ax`,
    plus thin per-run lines. Min/max keeps the band inside the data."""
    if df_wide.empty:
        ax.text(0.5, 0.5, 'no data', ha='center', va='center', transform=ax.
        ↪transAxes)
        return
    sm = df_wide.rolling(smooth, min_periods=1).mean()
    med = sm.median(axis=1)
    lo = sm.min(axis=1)
    hi = sm.max(axis=1)
    n = sm.shape[1]
    for col in sm.columns:
        ax.plot(sm.index, sm[col].values, color=color, lw=0.6, alpha=0.30)
    ax.fill_between(sm.index, lo.values, hi.values, color=color, alpha=0.20,
                    label=f'{label} min/max (n={n})')
    ax.plot(sm.index, med.values, color=color, lw=2.2, label=f'{label} median')
    ax.grid(True, alpha=0.3)
    ax.legend(fontsize=8, loc='best')

def _padded_ylim_minmax(df_wide, *, smooth=ROLL, pad_frac=0.10, include=None,
                        lo_clip=None, hi_clip=None):
    """Tight padded (ymin, ymax) around the smoothed min/max envelope."""
    if df_wide.empty:
        return None
    sm = df_wide.rolling(smooth, min_periods=1).mean()
    lo = float(sm.min(axis=1).min())
    hi = float(sm.max(axis=1).max())
    if include is not None and not np.isnan(include):
        lo = min(lo, float(include))
        hi = max(hi, float(include))
    span = max(hi - lo, 1e-9)
    pad = pad_frac * span
    ylo, yhi = lo - pad, hi + pad
    if lo_clip is not None:
        ylo = max(ylo, lo_clip)
    if hi_clip is not None:
        yhi = min(yhi, hi_clip)
    return (ylo, yhi)

# Reserve (auction floor) price and final-year penalty rate from the
# default config - used as horizontal reference lines on the clearing-
# price panel, and reported in the companion table.
_ets_cfg = REFERENCE_CONFIG.get('ets', {}) or {}
RESERVE_PRICE = float(_ets_cfg.get('reserve_price', float('nan')))
```

```

_pen_cfg = REFERENCE_CONFIG.get('penalty', {}) or {}
_pen_base = float(_pen_cfg.get('rate', float('nan')))
_pen_infl = float(_pen_cfg.get('inflation_rate', 0.0) or 0.0)
# Final simulation year is index N_YEARS - 1 (year 0 == base year).
FINAL_YEAR_PENALTY_RATE = (
    _pen_base * (1.0 + _pen_infl) ** max(N_YEARS - 1, 0)
    if not np.isnan(_pen_base) else float('nan')
)

# Episode-indexed wide frames (one column per run).
overview_panels = {
    'System reward': (stack_runs(_system_reward,
    ↪source='ep'), 'reward', 'steelblue'),
    'Final-year clearing price': (stack_runs(_final_year_clearing_by_ep,
    ↪source='run'), '€/tCO ', 'firebrick'),
    'Compliance rate': (stack_runs(_compliance_rate_by_ep,
    ↪source='run'), 'compliance [0,1]', 'seagreen'),
    'Mean green generation share': (stack_runs(_mean_green_frac_by_ep,
    ↪source='ep'), 'green frac [0,1]', 'darkorange'),
}

fig, axes = plt.subplots(2, 2, figsize=(15, 8))
for ax, (title, (df_wide, ylabel, color)) in zip(axes.ravel(), overview_panels.
    ↪items()):
    _minmax_fan(df_wide, ax, color=color, label=title)
    ax.set(title=title, xlabel='Episode', ylabel=ylabel)
    if title == 'Final-year clearing price':
        _refs = [v for v in (RESERVE_PRICE, FINAL_YEAR_PENALTY_RATE) if not np.
        ↪isnan(v)]
        ylim = _padded_ylim_minmax(df_wide, pad_frac=0.08,
            include=None)

        if ylim is not None:
            lo, hi = ylim
            for r in _refs:
                lo = min(lo, r); hi = max(hi, r)
            ax.set_ylim(lo, hi)
        if not np.isnan(RESERVE_PRICE):
            ax.axhline(RESERVE_PRICE, color='black', lw=1.0, linestyle='--',
            ↪alpha=0.7,
                label=f'reserve price ({RESERVE_PRICE:.1f} €/t)')
        if not np.isnan(FINAL_YEAR_PENALTY_RATE):
            ax.axhline(FINAL_YEAR_PENALTY_RATE, color='dimgray', lw=1.0,
            ↪linestyle=':', alpha=0.8,
                label=f'final-year penalty rate_
            ↪({FINAL_YEAR_PENALTY_RATE:.1f} €/t)')
        if _refs:

```

```

        ax.legend(fontsize=8, loc='best')
    elif title == 'Compliance rate':
        ylim = _padded_ylim_minmax(df_wide, pad_frac=0.15, lo_clip=0.0,
        ↪hi_clip=1.02)
        if ylim is not None:
            ax.set_ylim(*ylim)
    elif title == 'Mean green generation share':
        ylim = _padded_ylim_minmax(df_wide, pad_frac=0.15,
        ↪include=START_GREEN_FRAC,
                                lo_clip=0.0, hi_clip=1.0)
        if ylim is not None:
            ax.set_ylim(*ylim)
        if not np.isnan(START_GREEN_FRAC):
            ax.axhline(START_GREEN_FRAC, color='black', lw=1.0, linestyle='--',
            ↪alpha=0.7,
                        label=f'year-0 default mix ({START_GREEN_FRAC:.3f})')
        ax.legend(fontsize=8, loc='best')

plt.suptitle(f'RQ1.5 - Whole-training overview across {len(runs)} loaded run(s)',
        ↪'
            f'(median + min/max across runs, ROLL={ROLL})', fontsize=13, y=1.
        ↪02)
plt.tight_layout(); plt.show()

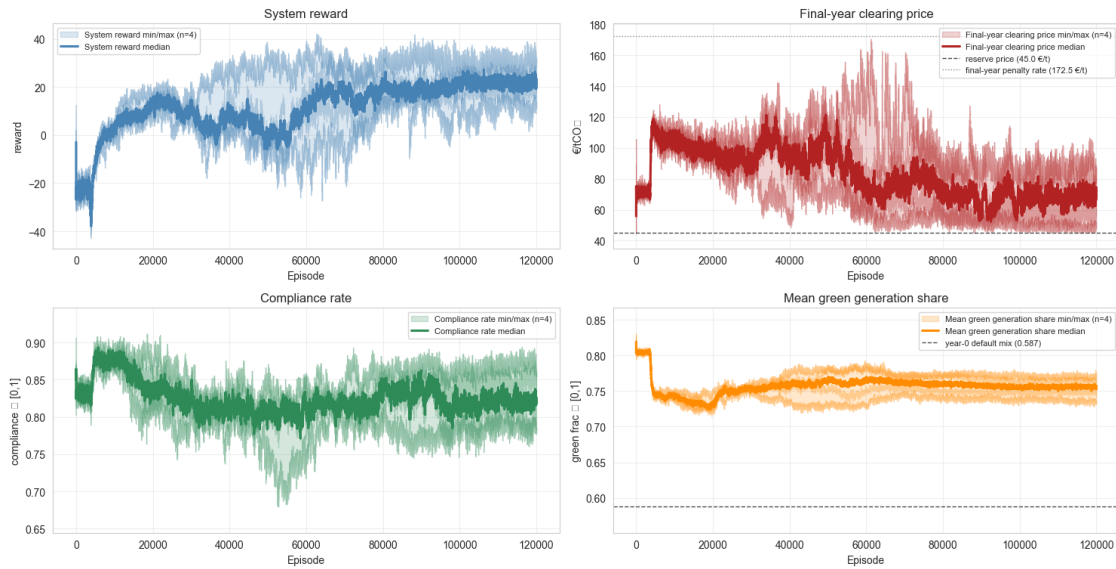
```

```

C:\Users\danie\AppData\Local\Temp\ipykernel_23912\2901333879.py:133:
UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.
    plt.tight_layout(); plt.show()
C:\Users\danie\AppData\Local\Temp\ipykernel_23912\2901333879.py:133:
UserWarning: Glyph 8712 (\N{ELEMENT OF}) missing from font(s) Arial.
    plt.tight_layout(); plt.show()
C:\Users\danie\AppData\Roaming\Python\Python312\site-
packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT
TWO}) missing from font(s) Arial.
    fig.canvas.print_figure(bytes_io, **kw)
C:\Users\danie\AppData\Roaming\Python\Python312\site-
packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8712 (\N{ELEMENT
OF}) missing from font(s) Arial.
    fig.canvas.print_figure(bytes_io, **kw)

```

RQ1.5 — Whole-training overview across 4 loaded run(s) (median + min/max across runs, ROLL=50)



[146]: *# Companion data table for the RQ1.5 overview figure: absolute values
of the smoothed median series (the line drawn in each panel) reduced
to the headline scalars used in thesis prose.*

```
_overview_rows = []
for title, (df_wide, ylabel, _color) in overview_panels.items():
    if df_wide.empty:
        _overview_rows.append({
            'panel': title, 'unit': ylabel, 'n_runs': 0, 'episodes': 0,
            'mean': np.nan, 'median': np.nan, 'std': np.nan,
            'min': np.nan, 'max': np.nan,
            'first': np.nan, 'last': np.nan,
        })
        continue
    sm = df_wide.rolling(ROLL, min_periods=1).mean()
    med = sm.median(axis=1)
    _overview_rows.append({
        'panel': title, 'unit': ylabel,
        'n_runs': int(sm.shape[1]), 'episodes': int(len(med)),
        'mean': float(med.mean()), 'median': float(med.median()),
        'std': float(med.std()), 'min': float(med.min()), 'max': float(med.
        ↪max()),
        'first': float(med.iloc[0]), 'last': float(med.iloc[-1]),
    })
overview_df = pd.DataFrame(_overview_rows)
print('Whole-training overview - summary of the smoothed median series shown in ↪
    ↪each panel:')
```

```
display(overview_df.round(4))
print('Reference lines used in the figure:')
print(f'  Year-0 default-mix mean green fraction: {START_GREEN_FRAC:.4f}')
print(f'  Auction reserve (floor) price:           {RESERVE_PRICE:.2f} \u20ac/t')
print(f'  Final-year penalty rate (year {N_YEARS - 1}, base {_pen_base:.2f} @_{_pen_infl:.1%}/yr): '
      f'{FINAL_YEAR_PENALTY_RATE:.2f} \u20ac/t')
```

Whole-training overview - summary of the smoothed median series shown in each panel:

			panel		unit	n_runs	episodes	mean	
	median	std	min	max	first	last			
0									
1									
2									
3									

Reference lines used in the figure:

Year-0 default-mix mean green fraction: 0.5875

Auction reserve (floor) price: 45.00 €/t

Final-year penalty rate (year 11, base 138.75 @ 2.0%/yr): 172.52 €/t

RQ1 summary. The convergence panels (1.1) plus the cross-seed table (1.2) and credibility plots (1.3 / 1.4) together provide the evidence base for Sub-RQ 1: with the default config, the simulation converges, reproduces a plausible EU-ETS price band and compliance plateau across loaded seeds, and the converged window is not dominated by pathology warnings.

1.7 §RQ2 — Regulatory-Mechanism Behaviour Within the Default Config

Scope caveat. Real *sensitivity* analysis (cap LRF / MSR on/off / penalty level) requires sweep variants and is deferred to a separate experiments notebook. What we *can* show with default-config data is a descriptive characterisation of how the regulatory mechanisms operate in equilibrium: how the cap maps to the clearing price, how the MSR withholds and releases, when the penalty actually binds, and how quickly the system decarbonises. This is the baseline that the sensitivity analysis will perturb.

Four panels:

- **2.1** — Cap → price linkage by year (mean across episodes/seeds in the converged window).
- **2.2** — TNAC trajectory and MSR reserve / flow behaviour.
- **2.3** — Penalty incidence: how often does it bind, and how big is it when it does?
- **2.4** — Decarbonisation pace: green-share trajectory and MAC-driven reductions.

1.7.1 RQ2.1 — Cap and clearing-price trajectory by year

```
[119]: # Average across all seeds and converged-window episodes.
year_frames = []
for k, run in runs.items():
    yr = tail_year_rows(run)
    if yr.empty: continue
    keep = ['episode', 'year', 'cap', 'clearing_price', 'auction_volume',
    ↪ 'tnac',
            'msr_reserve', 'msr_withhold_this_year', 'msr_release_this_year',
            'msr_total_cancelled', 'secondary_price']
    keep = [c for c in keep if c in yr.columns]
    if 'year' not in keep: continue
    df = yr[keep].copy(); df['__seed__'] = k[2]
    year_frames.append(df)
if not year_frames:
    raise RuntimeError('No year_log data available - RQ2 requires year-level
    ↪ logs.')
yr_pool = pd.concat(year_frames, ignore_index=True)

by_year = yr_pool.groupby('year').agg(['mean', 'std'])

fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))

ax = axes[0]
if ('cap', 'mean') in by_year.columns:
    ax.plot(by_year.index, by_year[('cap', 'mean')], lw=2, color='black',
    ↪ label='cap (Mt, mean)')
    ax.set_ylabel('cap (Mt)', color='black')
    ax2 = ax.twinx()
    if ('clearing_price', 'mean') in by_year.columns:
        ax2.plot(by_year.index, by_year[('clearing_price', 'mean')], lw=2,
        ↪ color='firebrick',
                label='clearing price (€/t)')
        ax2.fill_between(by_year.index,
                        by_year[('clearing_price', 'mean')] -
    ↪ by_year[('clearing_price', 'std')],
                        by_year[('clearing_price', 'mean')] +
    ↪ by_year[('clearing_price', 'std')],
                        color='firebrick', alpha=0.15)
        ax2.set_ylabel('clearing price (€/t)', color='firebrick')
    ax.set(title='Cap vs clearing price by simulation year (converged window)',
    ↪ xlabel='year')
    ax.grid(True, alpha=0.3)

ax = axes[1]
if ('auction_volume', 'mean') in by_year.columns:
```

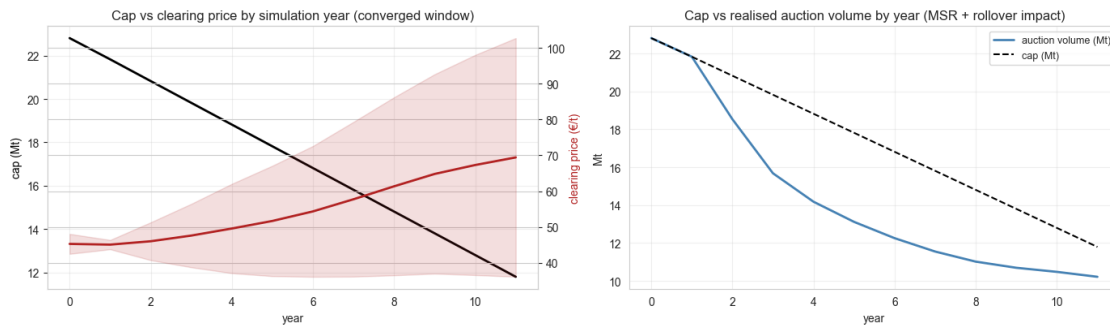
```

    ax.plot(by_year.index, by_year[('auction_volume', 'mean')], lw=2,
    ↪color='steelblue', label='auction volume (Mt)')
if ('cap', 'mean') in by_year.columns:
    ax.plot(by_year.index, by_year[('cap', 'mean')], lw=1.5, color='black',
    ↪linestyle='--', label='cap (Mt)')
ax.set(title='Cap vs realised auction volume by year (MSR + rollover impact)',
       xlabel='year', ylabel='Mt')
ax.legend(fontsize=9); ax.grid(True, alpha=0.3)

plt.suptitle('RQ2.1 - Regulatory cap schedule and its realised market impact',
            fontsize=13, y=1.02)
plt.tight_layout(); plt.show()

```

RQ2.1 — Regulatory cap schedule and its realised market impact



1.7.2 RQ2.2 — TNAC, MSR reserve, and MSR flow

```

[120]: fig, axes = plt.subplots(1, 3, figsize=(18, 4.5))

ax = axes[0]
if ('tnac', 'mean') in by_year.columns:
    ax.plot(by_year.index, by_year[('tnac', 'mean')], lw=2, color='darkblue')
    ax.fill_between(by_year.index,
                    by_year[('tnac', 'mean')] - by_year[('tnac', 'std')],
                    by_year[('tnac', 'mean')] + by_year[('tnac', 'std')],
                    color='darkblue', alpha=0.15)
    ax.set(title='TNAC trajectory (Total Number of Allowances in Circulation)',
           xlabel='year', ylabel='Mt')
    ax.grid(True, alpha=0.3)

ax = axes[1]
if ('msr_reserve', 'mean') in by_year.columns:
    ax.plot(by_year.index, by_year[('msr_reserve', 'mean')], lw=2,
    ↪color='purple', label='MSR reserve')
if ('msr_total_cancelled', 'mean') in by_year.columns:

```

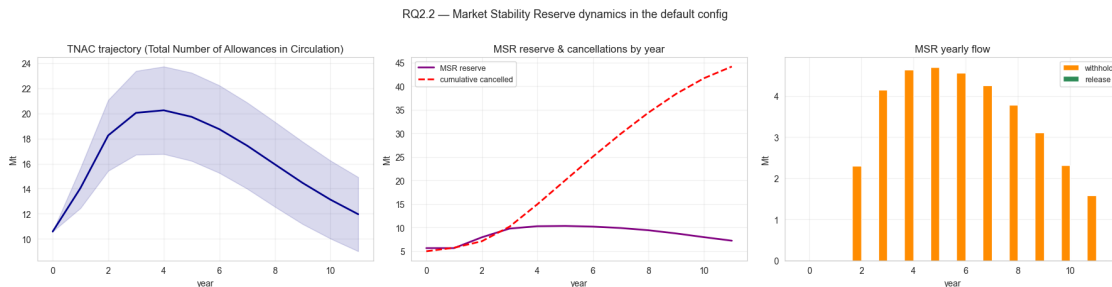
```

    ax.plot(by_year.index, by_year[('msr_total_cancelled', 'mean')], lw=2,
            color='red',
            linestyle='--', label='cumulative cancelled')
ax.set(title='MSR reserve & cancellations by year', xlabel='year', ylabel='Mt')
ax.legend(fontsize=9); ax.grid(True, alpha=0.3)

ax = axes[2]
years = by_year.index.values
if ('msr_withhold_this_year', 'mean') in by_year.columns:
    wh = by_year[('msr_withhold_this_year', 'mean')]
    ax.bar(years - 0.18, wh, width=0.35, color='darkorange', label='withhold')
if ('msr_release_this_year', 'mean') in by_year.columns:
    rel = by_year[('msr_release_this_year', 'mean')]
    ax.bar(years + 0.18, rel, width=0.35, color='seagreen', label='release')
ax.axhline(0, color='black', lw=0.8)
ax.set(title='MSR yearly flow', xlabel='year', ylabel='Mt')
ax.legend(fontsize=9); ax.grid(True, alpha=0.3)

plt.suptitle('RQ2.2 - Market Stability Reserve dynamics in the default config',
            fontsize=13, y=1.02)
plt.tight_layout(); plt.show()

```



1.7.3 RQ2.3 — Penalty incidence and magnitude

```

[121]: # (a) Share of (agent, year) cells with shortfall > 0; (b) mean per-cell
        penalty when it bites.
share_rows = []
for k, run in runs.items():
    yr = tail_year_rows(run)
    sf_cols = agent_cols(yr, 'shortfall')
    pen_cols = agent_cols(yr, 'penalty')
    if not sf_cols: continue
    sf = yr[sf_cols].values.flatten()
    pen = yr[pen_cols].values.flatten() if pen_cols else None
    nc_mask = sf > 1e-6

```

```

share_rows.append({
    'run': f'{k[0][:8]}/s{k[2]}',
    'pct_cells_short': 100.0 * nc_mask.mean() if sf.size else np.nan,
    'mean_shortfall_when_short_Mt': float(sf[nc_mask].mean()) if nc_mask.
    any() else 0.0,
    'mean_penalty_when_short_Meur': float(pen[nc_mask].mean()) if (pen is
    not None and nc_mask.any()) else 0.0,
})
share_df = pd.DataFrame(share_rows)
print(f'Penalty incidence in the converged window (last {TAIL_EPISODES}
    episodes):')
display(share_df.round(3))

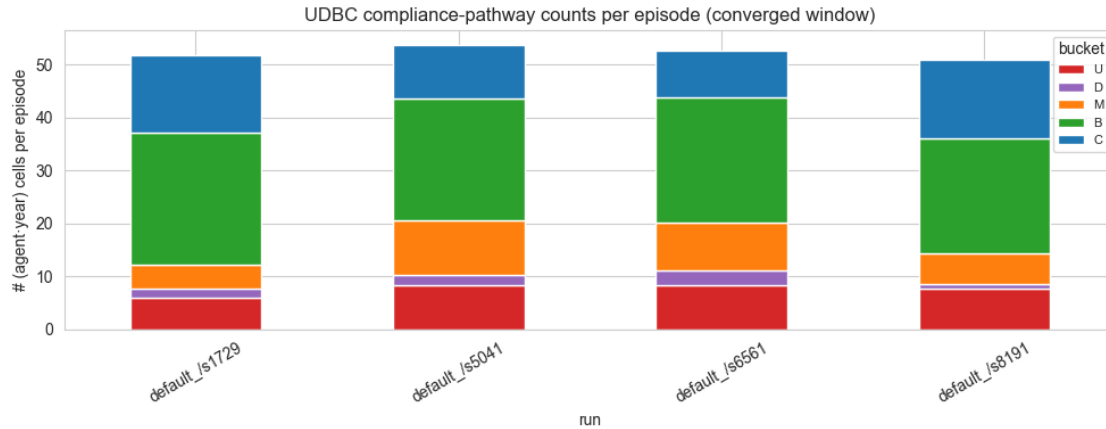
# UDBC mix at the system level (averaged across seeds).
ud_rows = []
for k, run in runs.items():
    ep = tail_episodes(run['ep_df'])
    for b in ['U', 'D', 'M', 'B', 'C']:
        cols = agent_cols(ep, f'udbc_{b}_total')
        if cols:
            ud_rows.append({'run': f'{k[0][:8]}/s{k[2]}', 'bucket': b,
                            'mean_total_per_episode': float(ep[cols].
                                sum(axis=1).mean())})
if ud_rows:
    ud_df = pd.DataFrame(ud_rows).pivot(index='run', columns='bucket',
    values='mean_total_per_episode').fillna(0.0)
    ud_df = ud_df[[c for c in ['U', 'D', 'M', 'B', 'C'] if c in ud_df.columns]]
    fig, ax = plt.subplots(figsize=(10, 4))
    ud_df.plot(kind='bar', stacked=True, ax=ax,
                color=['#d62728', '#9467bd', '#ff7f0e', '#2ca02c', '#1f77b4'])
    ax.set(title='UDBC compliance-pathway counts per episode (converged
    window)',
            ylabel='# (agent·year) cells per episode', xlabel='run')
    ax.legend(title='bucket', fontsize=8); ax.tick_params(axis='x', rotation=30)
    plt.tight_layout(); plt.show()
    print('U = pure under-bid; D = pure debt-cascade; M = mixed; B =
    bank-covered; C = secondary-covered.')

```

Penalty incidence in the converged window (last 5000 episodes):

	run	pct_cells_short	mean_shortfall_when_short_Mt	mean_penalty_when_short_Meur
0	default_/s1729	12.624	0.994	
	159.120			
1	default_/s8191	15.043	1.141	
	181.641			

2	default_/s6561	20.943	1.225	□
	↪ 194.810			
3	default_/s5041	21.411	1.324	□
	↪ 208.325			



U = pure under-bid; D = pure debt-cascade; M = mixed; B = bank-covered; C = secondary-covered.

1.7.4 RQ2.4 — Decarbonisation pace

```
[122]: # (a) Mean green_frac by year across episodes/seeds; (b) MAC reduction by year.
yr_pool2 = []
for k, run in runs.items():
    yr = tail_year_rows(run)
    keep = ['year']
    gf_cols = agent_cols(yr, 'green_frac')
    mac_cols = agent_cols(yr, 'mac_reduction')
    em_cols = agent_cols(yr, 'emissions')
    if 'year' not in yr.columns: continue
    df = yr[['year']].copy()
    if gf_cols: df['green_frac_mean'] = yr[gf_cols].mean(axis=1)
    if mac_cols: df['mac_reduction_total'] = yr[mac_cols].sum(axis=1)
    if em_cols: df['emissions_total'] = yr[em_cols].sum(axis=1)
    yr_pool2.append(df)
if yr_pool2:
    pool = pd.concat(yr_pool2, ignore_index=True).groupby('year').agg(['mean',
    ↪ 'std'])

    fig, axes = plt.subplots(1, 3, figsize=(18, 4.5))
    ax = axes[0]
    if ('green_frac_mean', 'mean') in pool.columns:
```

```

    ax.plot(pool.index, pool[('green_frac_mean', 'mean')], lw=2,
    ↪color='seagreen')
    ax.fill_between(pool.index,
                    pool[('green_frac_mean', 'mean')] -
    ↪pool[('green_frac_mean', 'std')],
                    pool[('green_frac_mean', 'mean')] +
    ↪pool[('green_frac_mean', 'std')],
                    color='seagreen', alpha=0.15)
    ax.set(title='Mean green generation share by year', xlabel='year',
    ↪ylabel='green frac [0,1]')
    ax.set_ylim(0, 1); ax.grid(True, alpha=0.3)

    ax = axes[1]
    if ('emissions_total', 'mean') in pool.columns:
        ax.plot(pool.index, pool[('emissions_total', 'mean')], lw=2,
    ↪color='black')
        ax.set(title='System total emissions by year', xlabel='year',
    ↪ylabel='Mt')
        ax.grid(True, alpha=0.3)

    ax = axes[2]
    if ('mac_reduction_total', 'mean') in pool.columns:
        ax.bar(pool.index, pool[('mac_reduction_total', 'mean')],
    ↪color='steelblue')
        ax.set(title='MAC fuel-switching reductions by year', xlabel='year',
    ↪ylabel='Mt abated')
        ax.grid(True, alpha=0.3)

    plt.suptitle('RQ2.4 - Decarbonisation pace within the default config',
                 fontsize=13, y=1.02)
    plt.tight_layout(); plt.show()

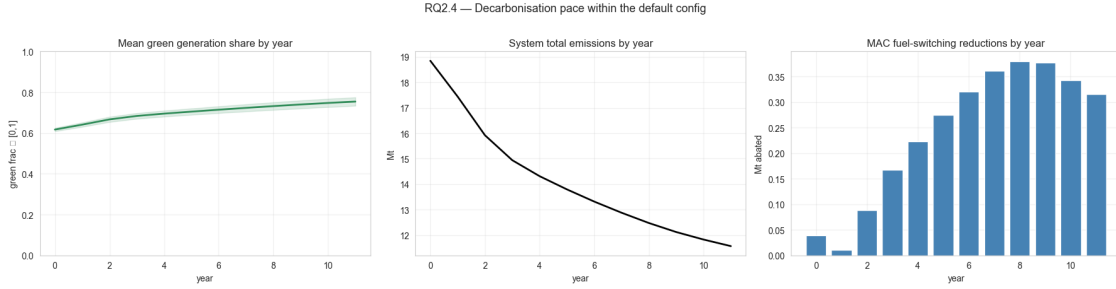
```

C:\Users\danie\AppData\Local\Temp\ipykernel_23912\1425922483.py:43: UserWarning: Glyph 8712 (\N{ELEMENT OF}) missing from font(s) Arial.

```
plt.tight_layout(); plt.show()
```

C:\Users\danie\AppData\Roaming\Python\Python312\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8712 (\N{ELEMENT OF}) missing from font(s) Arial.

```
fig.canvas.print_figure(bytes_io, **kw)
```



RQ2 summary. Within the default config, RQ2.1–2.4 give the *descriptive* baseline of how cap, MSR, penalty, and emission outcomes move together. The sensitivity question — *would these trajectories look qualitatively different under a tighter LRF, or with the MSR off?* — is what the cross-config experiments notebook will answer.

1.8 §RQ3 — Strategic Behaviour and Aggregation

The default config crosses 4 archetypes (coal-heavy, gas-dominant, transitioner, green-leader) with 2 reward weightings (pure financial, balanced ESG). That gives us the financial-vs-environmental contrast that RQ3 asks about, *inside one config*.

Five panels:

- **3.1** — Bidding behaviour by archetype $\times$ weighting.
- **3.2** — Investment behaviour by archetype $\times$ weighting.
- **3.3** — Secondary-market role by archetype.
- **3.4** — UDBC compliance pathway by archetype.
- **3.5** — Reward decomposition (financial vs ESG vs penalty) by weighting.

1.8.1 RQ3.1 — Bidding behaviour by archetype $\times$ reward-weighting

```
[123]: # Per-agent converged-window means, then group by (archetype, weighting).
def _per_agent_table(run, episode_cols, year_cols):
    """Return list of dicts (one per agent) with mean values from converged_
    ↪window."""
    ep = tail_episodes(run['ep_df'])
    yr = tail_year_rows(run)
    rows = []
    for i in range(N_AGENTS):
        row = {'agent_idx': i, 'archetype': _agent_archetype(i), 'weighting': _
        ↪_agent_weighting(i)}
        for prefix in episode_cols:
            col = f'{prefix}_A{i+1}'
            row[prefix] = float(ep[col].mean()) if col in ep.columns else np.nan
        for prefix in year_cols:
            col = f'{prefix}_A{i+1}'
```

```

        row[prefix] = float(yr[col].mean()) if col in yr.columns else np.nan
    rows.append(row)
    return rows

agent_rows = []
for k, run in runs.items():
    for r in _per_agent_table(
        run,
        episode_cols=['bid_price', 'avg_bid_mult', 'avg_bid_coverage',
                      'sec_buy_vol', 'sec_sell_vol', 'sec_buy_avg_px',
↪ 'sec_sell_avg_px',
                      'sec_buy_years', 'sec_sell_years',
                      'invest_cost', 'green_frac', 'delta_green',
                      'mean_alloc', 'penalty', 'shortfall', 'reward',
↪ 'reward_base',
                      'inv_onshore_share', 'inv_offshore_share',
↪ 'inv_solar_share'],
        year_cols=['mac_reduction', 'mac_cost'],
    ):
        r.update({'sweep': k[0], 'seed': k[2]})
        agent_rows.append(r)
agent_df = pd.DataFrame(agent_rows)
print(f'Per-(run, agent) rows: {len(agent_df)}')

fig, axes = plt.subplots(1, 3, figsize=(18, 4.5))
order = [(arch, w) for arch in ARCHETYPES for w in ['Financial', 'Balanced',
↪ ESG']]
labels = [f'{a}\n{w}' for a, w in order]

def _bar(ax, col, title, ylabel):
    means = []; stds = []
    for arch, w in order:
        sub = agent_df[(agent_df['archetype'] == arch) & (agent_df['weighting']
↪ == w)][col]
        means.append(sub.mean()); stds.append(sub.std())
    x = np.arange(len(order))
    colors = ['#d62728' if w == 'Financial' else '#2ca02c' for _, w in order]
    ax.bar(x, means, yerr=stds, color=colors, capsize=3, alpha=0.85)
    ax.set_xticks(x); ax.set_xticklabels(labels, fontsize=8, rotation=30,
↪ ha='right')
    ax.set(title=title, ylabel=ylabel); ax.grid(True, alpha=0.3, axis='y')

_bar(axes[0], 'bid_price', 'Mean Phase-1 bid price', '€/tCO2')
_bar(axes[1], 'avg_bid_mult', 'Mean qty multiplier on `estimate_need`,
↪ 'multiplier')

```



```

_bar(axes[2], 'avg_bid_coverage', 'Mean coverage = alloc / need', 'ratio_
↳(clipped to 1)')
plt.suptitle('RQ3.1 - Bidding behaviour: red = pure financial reward; green =
↳balanced ESG',
            fontsize=12, y=1.04)
plt.tight_layout(); plt.show()

```

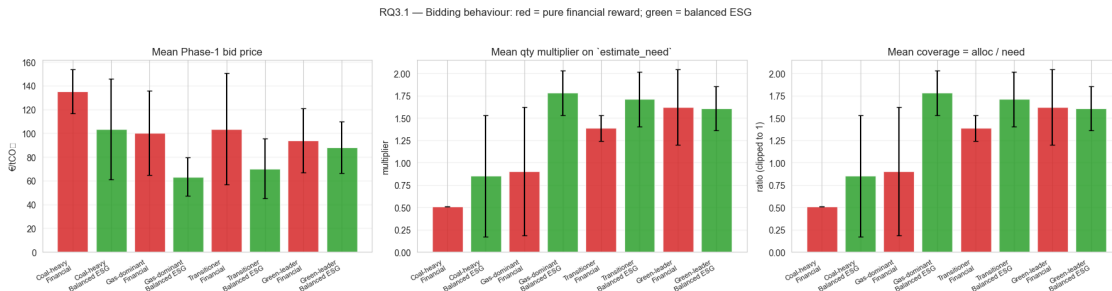
Per-(run, agent) rows: 32

C:\Users\danie\AppData\Local\Temp\ipykernel_23912\309587121.py:55: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

```

plt.tight_layout(); plt.show()
C:\Users\danie\AppData\Roaming\Python\Python312\site-
packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT
TWO}) missing from font(s) Arial.
fig.canvas.print_figure(bytes_io, **kw)

```



1.8.2 RQ3.2 — Investment behaviour

```

[124]: fig, axes = plt.subplots(1, 3, figsize=(18, 4.5))
_bar(axes[0], 'invest_cost', 'Mean episode invest cost', 'M€')
_bar(axes[1], 'green_frac', 'End-of-episode green fraction', 'green frac
↳[0,1]')
_bar(axes[2], 'delta_green', 'Δ green fraction over the episode', 'Δ green
↳frac')
plt.suptitle('RQ3.2 - Investment outcomes: red = pure financial; green =
↳balanced ESG',
            fontsize=12, y=1.04)
plt.tight_layout(); plt.show()

# Tech mix by archetype × weighting (stacked bar).
tech_means = (agent_df.groupby(['archetype', 'weighting'])
              [['inv_onshore_share', 'inv_offshore_share', 'inv_solar_share']]
              ↳mean())
fig, ax = plt.subplots(figsize=(10, 4.5))
tech_means.plot(kind='bar', stacked=True, ax=ax,

```

```

color=['#1f77b4', '#17becf', '#ffbf00'])
ax.set(title='Tech mix of episode investments by archetype × weighting',
       ylabel='share of invested MW', xlabel='')
ax.legend(['onshore wind', 'offshore wind', 'solar'], fontsize=9)
plt.xticks(rotation=30, ha='right')
plt.tight_layout(); plt.show()

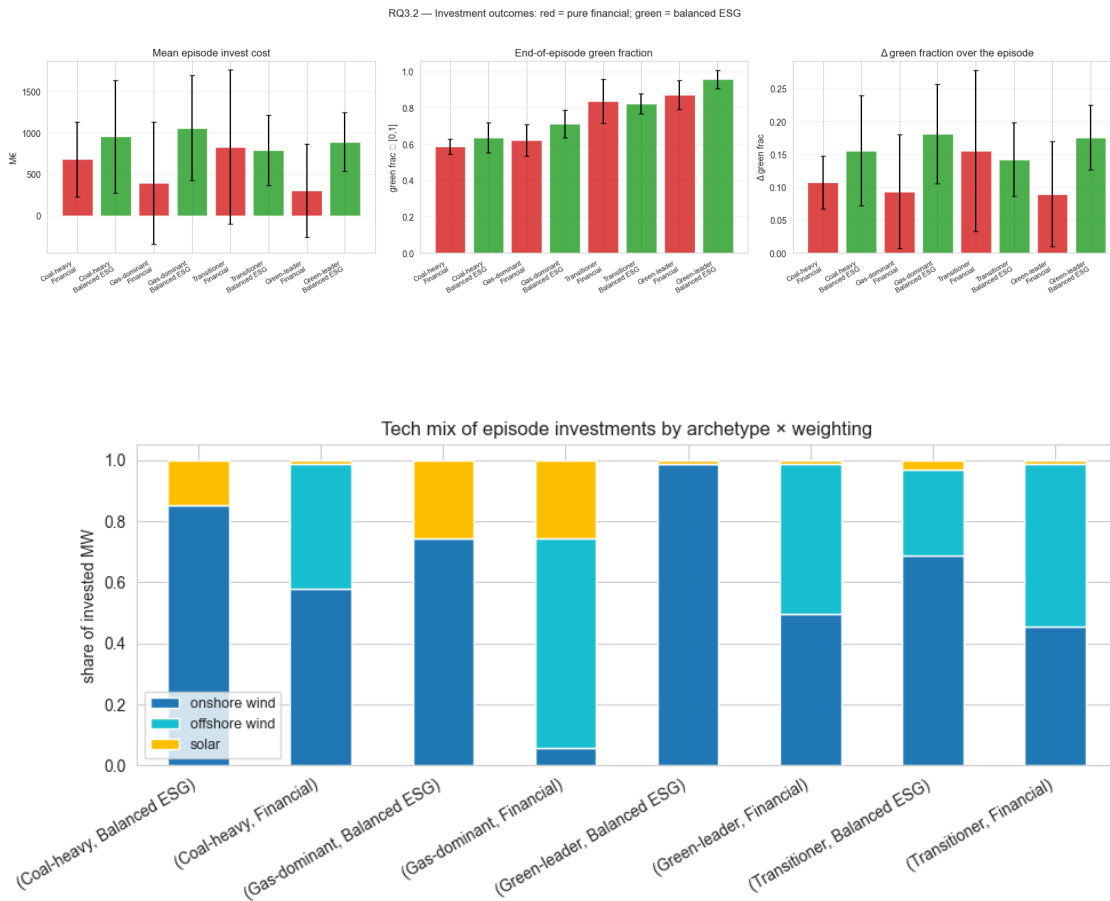
```

C:\Users\danie\AppData\Local\Temp\ipykernel_23912\1602023298.py:7: UserWarning: Glyph 8712 (\N{ELEMENT OF}) missing from font(s) Arial.

```
plt.tight_layout(); plt.show()
```

C:\Users\danie\AppData\Roaming\Python\Python312\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8712 (\N{ELEMENT OF}) missing from font(s) Arial.

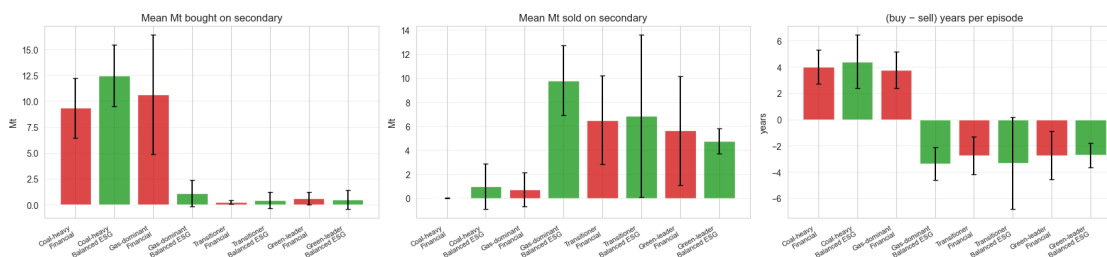
```
fig.canvas.print_figure(bytes_io, **kw)
```



1.8.3 RQ3.3 — Secondary-market role

```
[125]: fig, axes = plt.subplots(1, 3, figsize=(18, 4.5))
_bar(axes[0], 'sec_buy_vol', 'Mean Mt bought on secondary', 'Mt')
_bar(axes[1], 'sec_sell_vol', 'Mean Mt sold on secondary', 'Mt')
# Net buyer/seller years (out of 12).
def _net_buyer_years(row):
    by = row.get('sec_buy_years', np.nan); sy = row.get('sec_sell_years', np.
    ↪nan)
    if pd.isna(by) or pd.isna(sy): return np.nan
    return by - sy
agent_df['net_buyer_years'] = agent_df.apply(_net_buyer_years, axis=1)
_bar(axes[2], 'net_buyer_years', '(buy - sell) years per episode', 'years')
plt.suptitle('RQ3.3 - Who buys, who sells: secondary-market role by archetype ×
    ↪weighting',
            fontsize=12, y=1.04)
plt.tight_layout(); plt.show()
```

RQ3.3 — Who buys, who sells: secondary-market role by archetype × weighting



1.8.4 RQ3.4 — Compliance pathway (UDBC) by archetype × weighting

```
[126]: # Mean per-agent UDBC totals → grouped by archetype × weighting → stacked bar
    ↪of shares.
udbc_buckets = ['U', 'D', 'M', 'B', 'C']
udbc_rows = []
for k, run in runs.items():
    ep = tail_episodes(run['ep_df'])
    for i in range(N_AGENTS):
        row = {'archetype': _agent_archetype(i), 'weighting':
        ↪_agent_weighting(i)}
        for b in udbc_buckets:
            col = f'udbc_{b}_total_A{i+1}'
            row[b] = float(ep[col].mean()) if col in ep.columns else 0.0
        udbc_rows.append(row)
udbc_df = pd.DataFrame(udbc_rows)
udbc_grouped = udbc_df.groupby(['archetype', 'weighting'])[udbc_buckets].mean()
```

```

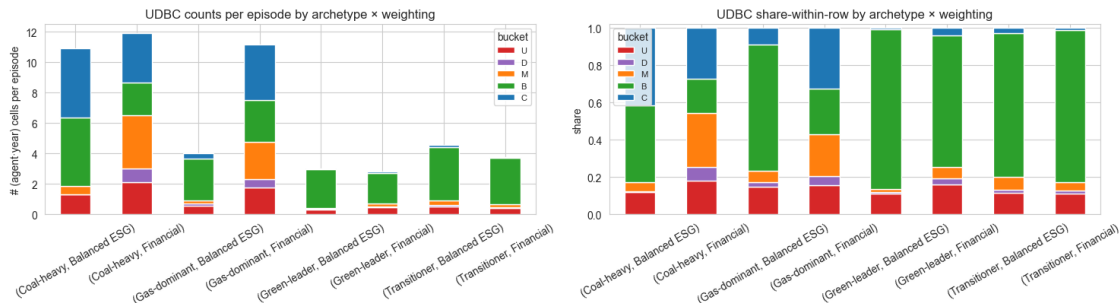
udbc_share = udbc_grouped.div(udbc_grouped.sum(axis=1).replace(0, np.nan),
    ↪axis=0).fillna(0.0)

fig, axes = plt.subplots(1, 2, figsize=(15, 4.5))
udbc_grouped.plot(kind='bar', stacked=True, ax=axes[0],
    color=['#d62728', '#9467bd', '#ff7f0e', '#2ca02c', '#1f77b4'])
axes[0].set(title='UDBC counts per episode by archetype × weighting',
    ylabel='# (agent·year) cells per episode', xlabel='')
axes[0].legend(title='bucket', fontsize=8)
axes[0].tick_params(axis='x', rotation=30)

udbc_share.plot(kind='bar', stacked=True, ax=axes[1],
    color=['#d62728', '#9467bd', '#ff7f0e', '#2ca02c', '#1f77b4'])
axes[1].set(title='UDBC share-within-row by archetype × weighting',
    ylabel='share', xlabel='')
axes[1].set_ylim(0, 1.02)
axes[1].legend(title='bucket', fontsize=8)
axes[1].tick_params(axis='x', rotation=30)
plt.suptitle('RQ3.4 - Compliance pathway by archetype × reward-weighting',
    fontsize=12, y=1.04)
plt.tight_layout(); plt.show()

```

RQ3.4 — Compliance pathway by archetype × reward-weighting



1.8.5 RQ3.5 — Reward decomposition: financial vs balanced-ESG agents

```

[127]: # At the (run, agent) level we have reward_base = financial - penalty + ESG
    ↪(pre-shaping).
# We don't have the financial / ESG / penalty buckets cleanly split in the
    ↪episode CSV (they're
# combined into reward_base), so we use what IS exposed: penalty_A{i},
    ↪invest_cost_A{i},
# reward_A{i}, reward_base_A{i}.

```

```

decomp = agent_df.groupby(['archetype', 'weighting'])[['reward', 'reward_base',
    ↪ 'penalty', 'invest_cost']].mean()
decomp['shaping'] = decomp['reward'] - decomp['reward_base']
decomp_print = decomp[['reward', 'reward_base', 'shaping', 'penalty',
    ↪ 'invest_cost']].round(2)
print('Per-archetype × weighting mean reward decomposition (M€-scale,
    ↪ post-normalisation units):')
display(decomp_print)

fig, ax = plt.subplots(figsize=(11, 4.5))
order = [(a, w) for a in ARCHETYPES for w in ['Financial', 'Balanced ESG']]
labels = [f'{a}\n{w}' for a, w in order]
x = np.arange(len(order))
width = 0.22

rew_b   = [decomp.loc[(a, w), 'reward_base'] for a, w in order]
shape_  = [decomp.loc[(a, w), 'shaping']      for a, w in order]
pen      = [decomp.loc[(a, w), 'penalty']      for a, w in order]
inv      = [decomp.loc[(a, w), 'invest_cost']  for a, w in order]

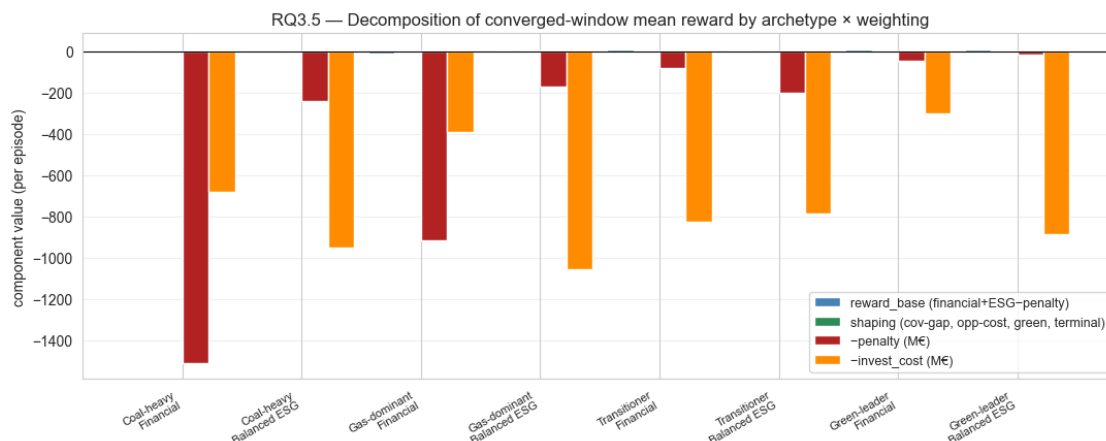
ax.bar(x - 1.5 * width, rew_b, width, label='reward_base
    ↪ (financial+ESG-penalty)', color='steelblue')
ax.bar(x - 0.5 * width, shape_, width, label='shaping (cov-gap, opp-cost,
    ↪ green, terminal)', color='seagreen')
ax.bar(x + 0.5 * width, -np.asarray(pen), width, label='-penalty (M€)',
    ↪ color='firebrick')
ax.bar(x + 1.5 * width, -np.asarray(inv), width, label='-invest_cost (M€)',
    ↪ color='darkorange')
ax.axhline(0, color='black', lw=0.8)
ax.set_xticks(x); ax.set_xticklabels(labels, fontsize=8, rotation=30,
    ↪ ha='right')
ax.set(title='RQ3.5 - Decomposition of converged-window mean reward by
    ↪ archetype × weighting',
        ylabel='component value (per episode)')
ax.legend(fontsize=9, loc='best'); ax.grid(True, alpha=0.3, axis='y')
plt.tight_layout(); plt.show()

```

Per-archetype × weighting mean reward decomposition (M€-scale, post-normalisation units):

		reward	reward_base	shaping	penalty	invest_cost
archetype	weighting					
Coal-heavy	Balanced ESG	-1.88	-2.40	0.51	243.23	951.54
	Financial	-7.33	-7.80	0.47	1510.65	679.35
Gas-dominant	Balanced ESG	5.16	4.63	0.52	174.14	1056.19
	Financial	-9.98	-10.47	0.49	915.77	392.30
Green-leader	Balanced ESG	10.21	9.68	0.53	15.30	888.53

	Financial	10.35	9.82	0.53	47.53	301.89
Transitioner	Balanced ESG	4.94	4.42	0.52	200.55	786.14
	Financial	9.76	9.23	0.52	80.39	827.32



Aggregation note. The system-level outcomes that RQ3 asks about — total clearing price, system green fraction, total penalty volume — are exactly what RQ1.2 / RQ1.3 / RQ2.1 already report. The missing piece RQ3 adds is *attribution*: which archetypes drive those aggregates? The §RQ3 panels above answer that — and in particular, panels 3.1 / 3.4 / 3.5 contrast the pure-financial agent (red) against its balanced-ESG twin (green) within each archetype, which is the direct test of “how do financial vs environmental objectives shape strategy”.

1.8.6 RQ4 — Compliance pressure, debt cascade, and credit stress

These panels read from the v8.6.1 expanded year-log columns. Cells silently no-op on logs from older versions (every column access is guarded by `if col in df.columns`). The four sub-questions are:

- **RQ4.1** — Does coverage gap ($\max(0, \text{need} - \text{alloc})$ Mt) shrink as agents learn to size their auction bids? Is the residual gap eaten by the secondary market or does it propagate as carry-forward debt?
- **RQ4.2** — Auction concentration: does the system stabilise around a diffuse allocation, or does a small subset of bidders capture most volume (HHI / max-agent-share)?
- **RQ4.3** — Secondary-market liquidity: how many agents participate on each side, and what fraction of intent gets executed?
- **RQ4.4** — Credit stress in the converged window: peak emergency-loan balance and peak carry-forward debt by archetype × reward-weighting.

1.8.7 RQ4.1 — Coverage gap & carry-forward trajectory

```
[128]: # RQ4.1 - Per-year coverage gap and end-of-year carry-forward (debt cascade).
# Cross-seed median + ±1 std fan, computed from the year_log only - works on
# any v8.6.1+ run. Earlier logs silently produce empty figures.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def _stack_year_metric(prefix, max_idx=N_TOTAL, agg="sum"):
    """Return wide DataFrame indexed by year with one column per run, where
    each value is the cross-agent (sum/mean) of `{prefix}_A{i}` for that
    year - averaged across episodes within the run."""
    out = {}
    for k, run in runs.items():
        yr = run["yr_df"]
        if "year" not in yr.columns:
            continue
        cols = [f"{prefix}_A{i+1}" for i in range(max_idx)
                 if f"{prefix}_A{i+1}" in yr.columns]
        if not cols:
            continue
        # Per-year mean across episodes of the cross-agent agg.
        if agg == "sum":
            per_year = yr.groupby("year")[cols].mean().sum(axis=1)
        else:
            per_year = yr.groupby("year")[cols].mean().mean(axis=1)
        out[k] = per_year
    return pd.concat(out, axis=1).sort_index() if out else pd.DataFrame()

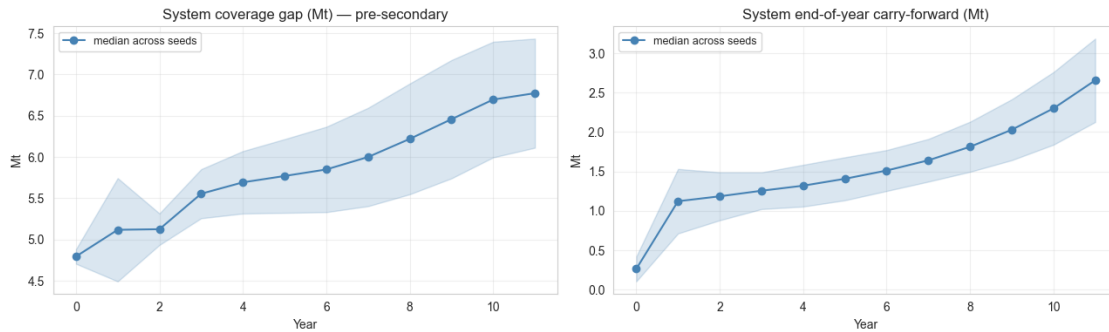
cov_gap_df = _stack_year_metric("coverage_gap") # system-sum (Mt)
cf_end_df = _stack_year_metric("carry_forward_end") # system-sum (Mt)
cf_start_df = _stack_year_metric("carry_forward_start") # system-sum (Mt)

if cov_gap_df.empty and cf_end_df.empty:
    print("[RQ4.1] coverage_gap_A* / carry_forward_*_A* columns missing - pre-8.
    ↪6.1 log.")
else:
    fig, axes = plt.subplots(1, 2, figsize=(13, 4))
    for ax, df, ttl in [
        (axes[0], cov_gap_df, "System coverage gap (Mt) - pre-secondary"),
        (axes[1], cf_end_df, "System end-of-year carry-forward (Mt)"),
    ]:
        if df.empty:
            ax.set_title(ttl + "\n(missing)"); continue
        med = df.median(axis=1); std = df.std(axis=1)
```

```

    ax.fill_between(med.index, med - std, med + std, alpha=0.2,
↳color="steelblue")
    ax.plot(med.index, med, "o-", color="steelblue", label="median across_
↳seeds")
    ax.set_xlabel("Year"); ax.set_ylabel("Mt"); ax.set_title(ttl)
    ax.grid(alpha=0.3); ax.legend(loc="best", fontsize=9)
    plt.tight_layout(); plt.show()

```



1.8.8 RQ4.2 — Auction concentration (HHI, max-agent share)

```

[129]: # RQ4.2 - Auction concentration trajectory. Reads `auction_hhi` and
# `auction_max_agent_share` (year-level scalars added in v8.6.1).
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def _stack_year_scalar(col):
    out = {}
    for k, run in runs.items():
        yr = run["yr_df"]
        if "year" not in yr.columns or col not in yr.columns:
            continue
        out[k] = yr.groupby("year")[col].mean()
    return pd.concat(out, axis=1).sort_index() if out else pd.DataFrame()

hhi_df = _stack_year_scalar("auction_hhi")
mas_df = _stack_year_scalar("auction_max_agent_share")
fail_df = _stack_year_scalar("auction_failed")
unsold_df = _stack_year_scalar("auction_unsold")

if hhi_df.empty and mas_df.empty:
    print("[RQ4.2] auction_hhi / auction_max_agent_share missing - pre-8.6.1_
↳log.")
else:

```

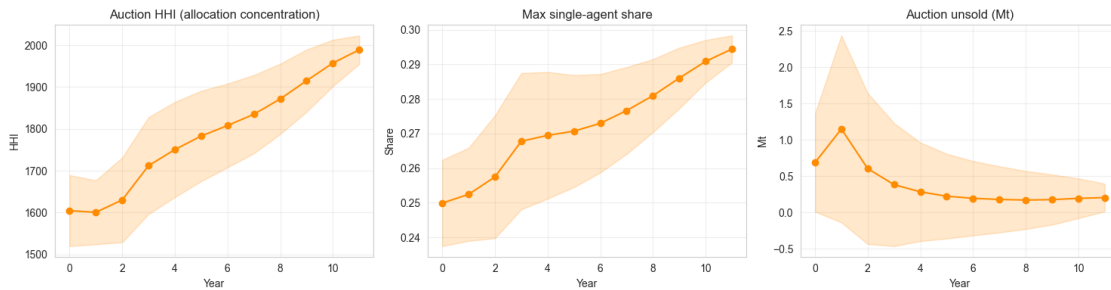


```

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, df, ttl, ylabel in [
    (axes[0], hhi_df, "Auction HHI (allocation concentration)", "HHI"),
    (axes[1], mas_df, "Max single-agent share", "Share"),
    (axes[2], unsold_df, "Auction unsold (Mt)", "Mt"),
]:
    if df.empty:
        ax.set_title(ttl + "\n(missing)"); continue
    med = df.median(axis=1); std = df.std(axis=1)
    ax.fill_between(med.index, med - std, med + std, alpha=0.2,
        color="darkorange")
    ax.plot(med.index, med, "o-", color="darkorange")
    ax.set_xlabel("Year"); ax.set_ylabel(ylabel); ax.set_title(ttl)
    ax.grid(alpha=0.3)
plt.tight_layout(); plt.show()

if not fail_df.empty:
    n_failures = int(np.nansum(fail_df.values))
    print(f"Total auction-failure (year, seed) cells across loaded runs: {n_failures}")

```



Total auction-failure (year, seed) cells across loaded runs: 0

1.8.9 RQ4.3 — Secondary-market liquidity (intent vs execution)

```

[130]: # RQ4.3 - Secondary-market participation: how many agents posted intent
# vs how many actually traded? A wide intent-execution gap means agents
# repeatedly try to clear at uncrossable prices.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

cols = {
    "buyers_intent": "secondary_n_buyers_intent",
    "sellers_intent": "secondary_n_sellers_intent",
    "buyers_executed": "secondary_n_buyers_executed",

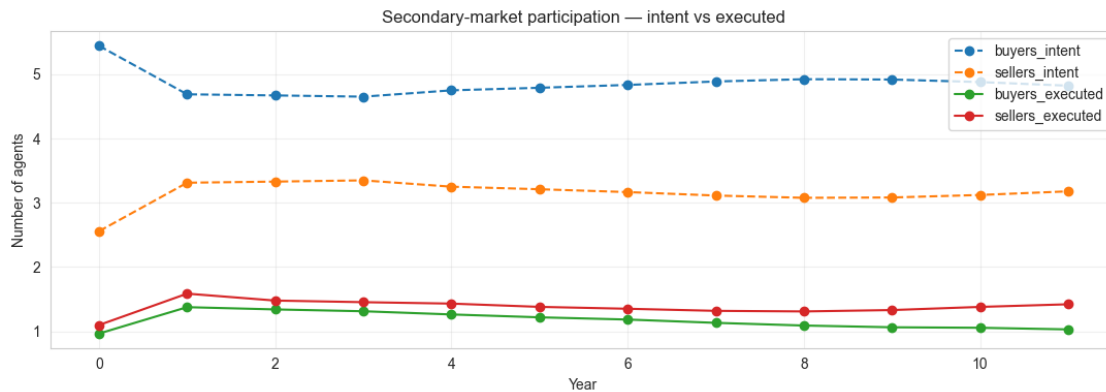
```

```

    "sellers_executed": "secondary_n_sellers_executed",
}
present = {label: col for label, col in cols.items() if any(col in run["yr_df"].
    ↪ columns for run in runs.values())}

if not present:
    print("[RQ4.3] secondary_n*_intent/executed columns missing - pre-8.6.1_
    ↪ log.")
else:
    fig, ax = plt.subplots(figsize=(11, 4))
    for label, col in present.items():
        rows = []
        for k, run in runs.items():
            yr = run["yr_df"]
            if col not in yr.columns or "year" not in yr.columns:
                continue
            rows.append(yr.groupby("year")[col].mean())
        if not rows:
            continue
        s = pd.concat(rows, axis=1).median(axis=1)
        ls = "-" if "executed" in label else "--"
        ax.plot(s.index, s, ls + "o", label=label)
    ax.set_xlabel("Year"); ax.set_ylabel("Number of agents")
    ax.set_title("Secondary-market participation - intent vs executed")
    ax.legend(); ax.grid(alpha=0.3)
    plt.tight_layout(); plt.show()

```



1.8.10 RQ4.4 — Converged-window credit stress by archetype × weighting

```

[131]: # RQ4.4 - Per-(archetype, weighting) cell of the mean peak emergency-loan
# balance and peak carry-forward debt in the converged window. Sourced from
# the per-episode `peak_loan_outstanding_A{i}` / `peak_carry_forward_A{i}`

```

```

# (v8.6.1+); pre-8.6.1 logs lack these columns and produce an empty table.
import numpy as np
import pandas as pd

def _converged_credit_metrics():
    rows = []
    for k, run in runs.items():
        ep_tail = tail_episodes(run["ep_df"])
        cfg = run["config"]
        for i in range(N_AGENTS):
            arche = ARCHETYPES[i // 2] if i // 2 < len(ARCHETYPES) else
↪f"A{i+1}"
            w_cost, w_green = (cfg.get("companies", {}).get("reward_weights"),
↪or [[1, 0]] * N_AGENTS)[i]
            weighting = "Financial" if w_green < 0.25 else ("ESG" if w_green >
↪0.75 else "Balanced")
            row = {"sweep": k[0], "variant": k[1], "seed": k[2],
                    "archetype": arche, "weighting": weighting, "agent": i + 1}
            for stub in ["peak_loan_outstanding", "peak_carry_forward",
                        "final_treasury_reserve"]:
                col = f"{stub}_A{i+1}"
                row[stub] = float(ep_tail[col].mean()) if col in ep_tail.
↪columns else np.nan
            rows.append(row)
    return pd.DataFrame(rows)

credit_df = _converged_credit_metrics()
if credit_df.empty or credit_df[["peak_loan_outstanding",
↪"peak_carry_forward"]].isna().all().all():
    print("[RQ4.4] peak_loan_outstanding_A* / peak_carry_forward_A* missing -
↪pre-8.6.1 log.")
else:
    pivot_loan = credit_df.pivot_table(
        index="archetype", columns="weighting", values="peak_loan_outstanding",
↪aggfunc="mean"
    ).round(3)
    pivot_cf = credit_df.pivot_table(
        index="archetype", columns="weighting", values="peak_carry_forward",
↪aggfunc="mean"
    ).round(3)
    print("Mean peak emergency-loan balance (M€) - converged window:")
    display(pivot_loan)
    print("\nMean peak end-of-year carry-forward (Mt) - converged window:")
    display(pivot_cf)

```

Mean peak emergency-loan balance (M€) - converged window:

weighting archetype	Balanced	Financial
Coal-heavy	0.0	0.000
Gas-dominant	0.0	0.005
Green-leader	0.0	0.000
Transitioner	0.0	0.000

Mean peak end-of-year carry-forward (Mt) - converged window:

weighting archetype	Balanced	Financial
Coal-heavy	0.790	2.025
Gas-dominant	0.528	1.403
Green-leader	0.072	0.170
Transitioner	0.535	0.271

1.8.11 RQ4.5 — Why agents go in-compliant: per-cell root-cause attribution

The UDBC mix in §RQ3.4 / §RQ2.3 already tells us *how often* each combinatorial state (under-bid / debt / mixed / bank-covered / secondary-covered) occurs. What it does not isolate is the **proximate cause** of each non-compliant cell: did the agent bid too cheaply and lose the auction, did it ask for too few tonnes (`bid_qty_mult × estimate_need` short of realised `need`), did the auction itself clear oversubscribed, was it carrying inherited debt forward, or did it submit a buy intent on the secondary that nobody filled?

This panel pulls every non-compliant cell (`episode`, `year`, `agent`) from the converged-tail window only and runs each through a prioritised waterfall:

1. **Inherited debt** — `alloc emiss AND cf_start shortfall`. The agent was auction-compliant *this year*; the deficit is fully explained by debt carried forward from earlier years.
2. **Underbid (price)** — `bid_price < 99.5 % × clearing_price`. The agent was priced out of the marginal allocation tranche.
3. **Undersized bid (quantity)** — `(bid_qty_mult × estimate_need)` falls short of the realised need by more than half of the shortfall. The agent asked for too few tonnes regardless of price.
4. **Oversubscribed auction** — `auction_total_demand / auction_volume > 1.05`. Even bidding at clearing, the agent was rationed by total demand exceeding supply.
5. **Secondary illiquidity** — agent submitted a positive buy intent (`sec_qty_action > 0`) but the realised buy fell more than 25 % of the shortfall short of intent.
6. **Mixed / cascade** — fallback bucket; no single cause dominates.

The taxonomy is intentionally simple and prioritised — each cell is assigned exactly one dominant cause. The downstream tables also report median diagnostic-feature values both per-cause and contrasting compliant vs non-compliant cells, so the *strength* of each signal is visible too.

```
[132]: # RQ4.5 - Per-cell root-cause attribution for non-compliant (agent, year)
# cells in the converged tail window. Cells silently no-op on logs lacking
# the v8.6.1+ year-log columns.
import numpy as np
```

```

import pandas as pd
import matplotlib.pyplot as plt

EPS_MT = 1e-6
CAUSE_ORDER = ['Inherited debt', 'Underbid (price)', 'Undersized bid (qty)',
               'Oversubscribed auction', 'Secondary illiquidity', 'Mixed / cascade']
CAUSE_COLORS = {
    'Inherited debt': '#9467bd',
    'Underbid (price)': '#d62728',
    'Undersized bid (qty)': '#ff7f0e',
    'Oversubscribed auction': '#1f77b4',
    'Secondary illiquidity': '#17becf',
    'Mixed / cascade': '#7f7f7f',
}

def _classify_row(sf, cf_start, alloc, emiss, bid_p, clear_p,
                 undersized_qty, oversub, sec_unmet):
    if sf <= EPS_MT:
        return None
    cf = max(0.0, float(cf_start))
    if alloc + EPS_MT >= emiss and cf >= sf - EPS_MT:
        return 'Inherited debt'
    if bid_p > 0 and clear_p > 0 and bid_p < 0.995 * clear_p:
        return 'Underbid (price)'
    if undersized_qty > 0.5 * sf:
        return 'Undersized bid (qty)'
    if oversub > 1.05:
        return 'Oversubscribed auction'
    if sec_unmet > 0.25 * sf:
        return 'Secondary illiquidity'
    return 'Mixed / cascade'

nc_rows = []
for k, run in runs.items():
    yr = tail_year_rows(run)
    if yr.empty or 'year' not in yr.columns:
        continue
    n = len(yr)
    cp_arr = yr['clearing_price'].to_numpy(float) if 'clearing_price' in yr.columns else np.full(n, np.nan)
    atd = yr['auction_total_demand'].to_numpy(float) if 'auction_total_demand' in yr.columns else np.full(n, np.nan)
    avol = yr['auction_volume'].to_numpy(float) if 'auction_volume' in yr.columns else np.full(n, np.nan)

```

```

ns_exec = yr['secondary_n_sellers_executed'].to_numpy(float) if_
↪ 'secondary_n_sellers_executed' in yr.columns else np.full(n, np.nan)
yr_arr = yr['year'].to_numpy(int)
ep_arr = yr['episode'].to_numpy(int) if 'episode' in yr.columns else np.
↪ zeros(n, dtype=int)
oversub_y = np.where(avol > 1e-9, atd / avol, np.nan)

for i in range(N_AGENTS):
    cols_req = [f'shortfall_A{i+1}', f'alloc_A{i+1}', f'emissions_A{i+1}',
                f'carry_forward_start_A{i+1}', f'bid_price_A{i+1}',
                f'estimate_need_A{i+1}', f'bid_qty_mult_A{i+1}',
                f'bank_start_A{i+1}', f'trade_qty_A{i+1}',
                f'sec_qty_action_A{i+1}']
    if not all(c in yr.columns for c in cols_req):
        continue
    sf = yr[f'shortfall_A{i+1}'].to_numpy(float)
    nc_mask = sf > EPS_MT
    if not nc_mask.any():
        continue
    alloc = yr[f'alloc_A{i+1}'].to_numpy(float)
    emiss = yr[f'emissions_A{i+1}'].to_numpy(float)
    cf_st = yr[f'carry_forward_start_A{i+1}'].to_numpy(float)
    bidp = yr[f'bid_price_A{i+1}'].to_numpy(float)
    en = yr[f'estimate_need_A{i+1}'].to_numpy(float)
    qm = yr[f'bid_qty_mult_A{i+1}'].to_numpy(float)
    bank = yr[f'bank_start_A{i+1}'].to_numpy(float)
    trq = yr[f'trade_qty_A{i+1}'].to_numpy(float)
    sqa = yr[f'sec_qty_action_A{i+1}'].to_numpy(float)
    cf_e = yr[f'carry_forward_end_A{i+1}'].to_numpy(float) if_
↪ f'carry_forward_end_A{i+1}' in yr.columns else np.full(n, np.nan)
    loan = yr[f'loan_outstanding_A{i+1}'].to_numpy(float) if_
↪ f'loan_outstanding_A{i+1}' in yr.columns else np.full(n, np.nan)
    tdr = yr[f'treasury_drawn_A{i+1}'].to_numpy(float) if_
↪ f'treasury_drawn_A{i+1}' in yr.columns else np.full(n, np.nan)
    cgap = yr[f'coverage_gap_A{i+1}'].to_numpy(float) if_
↪ f'coverage_gap_A{i+1}' in yr.columns else np.full(n, np.nan)

    # need_realised = emissions + inherited debt the agent had to cover.
    need_realised = emiss + np.maximum(0.0, cf_st)
    bid_qty_target = np.maximum(0.0, qm) * np.maximum(0.0, en)
    undersized_qty = np.maximum(0.0, need_realised - bid_qty_target)
    sec_intent_buy = np.maximum(0.0, sqa)
    sec_bought = np.maximum(0.0, trq)
    sec_unmet = np.maximum(0.0, sec_intent_buy - sec_bought)

    for j in np.where(nc_mask)[0]:

```

```

cp_j = float(cp_arr[j]) if np.isfinite(cp_arr[j]) else 0.0
os_j = float(oversub_y[j]) if np.isfinite(oversub_y[j]) else 0.0
cause = _classify_row(
    sf=float(sf[j]), cf_start=float(cf_st[j]),
    alloc=float(alloc[j]), emiss=float(emiss[j]),
    bid_p=float(bidp[j]), clear_p=cp_j,
    undersized_qty=float(undersized_qty[j]),
    oversub=os_j, sec_unmet=float(sec_unmet[j]),
)
nc_rows.append({
    'run': f'{k[1]}/s{k[2]}',
    'episode': int(ep_arr[j]),
    'year': int(yr_arr[j]),
    'agent': i + 1,
    'archetype': _agent_archetype(i),
    'weighting': _agent_weighting(i),
    'shortfall_Mt': float(sf[j]),
    'inherited_debt_Mt': float(cf_st[j]),
    'alloc_Mt': float(alloc[j]),
    'emissions_Mt': float(emiss[j]),
    'bid_price': float(bidp[j]),
    'clearing_price': cp_j,
    'bid_qty_mult': float(qm[j]),
    'estimate_need_Mt': float(en[j]),
    'undersized_qty_Mt': float(undersized_qty[j]),
    'oversubscription_ratio': os_j,
    'bank_start_Mt': float(bank[j]),
    'sec_buy_intent_Mt': float(sec_intent_buy[j]),
    'secondary_buy_unmet_Mt': float(sec_unmet[j]),
    'n_sellers_executed': float(ns_exec[j]) if np.
↳isfinite(ns_exec[j]) else np.nan,
    'carry_forward_end_Mt': float(cf_e[j]),
    'loan_outstanding_Meur': float(loan[j]),
    'treasury_drawn_Meur': float(tdr[j]),
    'coverage_gap_Mt': float(cgap[j]),
    'cause': cause,
})

incomp_df = pd.DataFrame(nc_rows)

if incomp_df.empty:
    print(' [RQ4.5] No non-compliant (agent, year) cells in the converged window_
↳- '
        'agents are fully compliant. Skipping attribution panels.')
else:
    incomp_df['cause'] = pd.Categorical(incomp_df['cause'],_
↳categories=CAUSE_ORDER, ordered=True)

```

```

# (a) Counts and tonne-weighted shares per cause.
cause_counts = (incomp_df.groupby('cause', observed=True)
                 .agg(n_cells=('shortfall_Mt', 'size'),
                     sum_shortfall_Mt=('shortfall_Mt', 'sum'),
                     median_shortfall_Mt=('shortfall_Mt',
→ 'median')))
                 .reindex(CAUSE_ORDER).fillna(0.0))
total_cells = max(1, int(cause_counts['n_cells'].sum()))
total_tons = max(1e-9, float(cause_counts['sum_shortfall_Mt'].sum()))
cause_counts['cell_share_pct'] = 100.0 * cause_counts['n_cells'] /
→ total_cells
cause_counts['tonne_share_pct'] = 100.0 * cause_counts['sum_shortfall_Mt'] /
→ total_tons

print(f'[RQ4.5] Non-compliant (agent, year, episode) cells in converged_
→ window: '
      f'{len(incomp_df):,}')
print(f'[RQ4.5]  $\Sigma$  shortfall across the window: '
      f'{incomp_df["shortfall_Mt"].sum():.2f} Mt')
display(cause_counts.round(3))

fig, axes = plt.subplots(1, 2, figsize=(14, 4.6))
colors = [CAUSE_COLORS[c] for c in CAUSE_ORDER]
axes[0].bar(CAUSE_ORDER, cause_counts['cell_share_pct'].values,
→ color=colors)
axes[0].set(title='Share of non-compliant cells by dominant cause',
            ylabel='% of (agent, year, episode) cells')
axes[0].tick_params(axis='x', rotation=25); axes[0].grid(alpha=0.3,
→ axis='y')

axes[1].bar(CAUSE_ORDER, cause_counts['tonne_share_pct'].values,
→ color=colors)
axes[1].set(title='Share of total shortfall tonnes by dominant cause',
            ylabel='% of  $\Sigma$  shortfall (Mt)')
axes[1].tick_params(axis='x', rotation=25); axes[1].grid(alpha=0.3,
→ axis='y')
plt.suptitle('RQ4.5 - Why agents go non-compliant in the converged window',
            y=1.03, fontsize=13)
plt.tight_layout(); plt.show()

# (b) Median diagnostic feature values per cause - what discriminates them.
diag_cols = ['shortfall_Mt', 'inherited_debt_Mt',
             'bid_price', 'clearing_price', 'bid_qty_mult',
             'undersized_qty_Mt', 'oversubscription_ratio',
             'bank_start_Mt', 'sec_buy_intent_Mt',

```



```

        'secondary_buy_unmet_Mt', 'loan_outstanding_Meur']
diag_cols = [c for c in diag_cols if c in incom_df.columns]
cause_medians = (incom_df.groupby('cause', observed=True)[diag_cols]
                  .median().reindex(CAUSE_ORDER))
print('[RQ4.5] Median feature value per dominant cause:')
display(cause_medians.round(3))

# (c) Per-archetype × weighting cause split.
aw_pivot = (incom_df.groupby(['archetype', 'weighting', 'cause'],
↪observed=True)
            .size().unstack('cause', fill_value=0))
aw_share = aw_pivot.div(aw_pivot.sum(axis=1).replace(0, np.nan), axis=0).
↪fillna(0.0)
aw_share = aw_share.reindex(columns=[c for c in CAUSE_ORDER if c in
↪aw_share.columns])
if not aw_share.empty:
    fig, ax = plt.subplots(figsize=(11, 4.2))
    aw_share.plot(kind='bar', stacked=True, ax=ax,
                  color=[CAUSE_COLORS[c] for c in aw_share.columns])
    ax.set(title='RQ4.5 - Cause mix by archetype × weighting (share within
↪row)',
           ylabel='Share of non-compliant cells', ylim=(0, 1.0))
    ax.legend(fontsize=8, loc='center left', bbox_to_anchor=(1.02, 0.5))
    ax.tick_params(axis='x', rotation=20); ax.grid(alpha=0.3, axis='y')
    plt.tight_layout(); plt.show()

# (d) Compliant vs non-compliant feature contrast on the same tail window.
comp_pieces = []
for k, run in runs.items():
    yr = tail_year_rows(run)
    if yr.empty: continue
    for i in range(N_AGENTS):
        sf_col = f'shortfall_A{i+1}'
        if sf_col not in yr.columns: continue
        sf = yr[sf_col].to_numpy(float)
        cmask = sf <= EPS_MT
        if not cmask.any(): continue
        row = {}
        for stub, src_col in [
            ('inherited_debt_Mt', f'carry_forward_start_A{i+1}'),
            ('bank_start_Mt', f'bank_start_A{i+1}'),
            ('bid_qty_mult', f'bid_qty_mult_A{i+1}'),
            ('bid_price', f'bid_price_A{i+1}'),
            ('coverage_gap_Mt', f'coverage_gap_A{i+1}'),
        ]:
            row[stub] = (yr.loc[cmask, src_col].to_numpy(float)

```

```

        if src_col in yr.columns else np.full(int(cmask.
↪sum()), np.nan))
        row['clearing_price'] = (yr.loc[cmask, 'clearing_price'].
↪to_numpy(float)

        if 'clearing_price' in yr.columns
        else np.full(int(cmask.sum()), np.nan))
        comp_pieces.append(pd.DataFrame(row))
    if comp_pieces:
        comp_df = pd.concat(comp_pieces, ignore_index=True)
        comp_df['bid_to_clearing'] = (comp_df['bid_price']
        / comp_df['clearing_price']).replace(0,
↪np.nan))
        nc_with = incmp_df.copy()
        nc_with['bid_to_clearing'] = (nc_with['bid_price']
        / nc_with['clearing_price']).replace(0,
↪np.nan))
        contrast_cols = ['inherited_debt_Mt', 'bank_start_Mt',
        'bid_qty_mult', 'bid_to_clearing', 'coverage_gap_Mt']
        rows = []
        for c in contrast_cols:
            cv = comp_df[c].replace([np.inf, -np.inf], np.nan).dropna() if c in
↪comp_df.columns else pd.Series(dtype=float)
            nv = nc_with[c].replace([np.inf, -np.inf], np.nan).dropna() if c in
↪nc_with.columns else pd.Series(dtype=float)
            rows.append({
                'feature': c,
                'compliant_median': float(cv.median()) if len(cv) else np.
↪nan,
                'non_compliant_median': float(nv.median()) if len(nv) else np.
↪nan,
                'delta_(nc - c)': (float(nv.median() - cv.median())
        if len(cv) and len(nv) else np.nan),
            })
        print(['RQ4.5] Compliant vs non-compliant cell medians (key
↪discriminators):'])
        display(pd.DataFrame(rows).round(3))

```

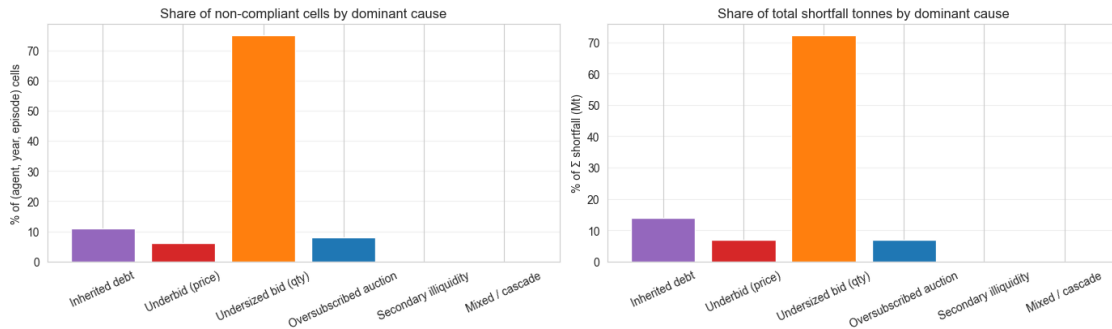
[RQ4.5] Non-compliant (agent, year, episode) cells in converged window: 336,101

[RQ4.5] Σ shortfall across the window: 401,810.42 Mt

	n_cells	sum_shortfall_Mt	median_shortfall_Mt	
cell_share_pct	tonne_share_pct			
cause				
Inherited debt	36726	55893.162	1.666	
↪10.927	13.910			
Underbid (price)	20785	28219.957	0.750	
↪6.184	7.023			

Undersized bid (qty)	251702	289880.351	1.102	␣
↳74.889	72.144			
Oversubscribed auction	26839	27800.081	0.714	␣
↳7.985	6.919			
Secondary illiquidity	14	6.650	0.422	␣
↳0.004	0.002			
Mixed / cascade	35	10.223	0.242	␣
↳0.010	0.003			

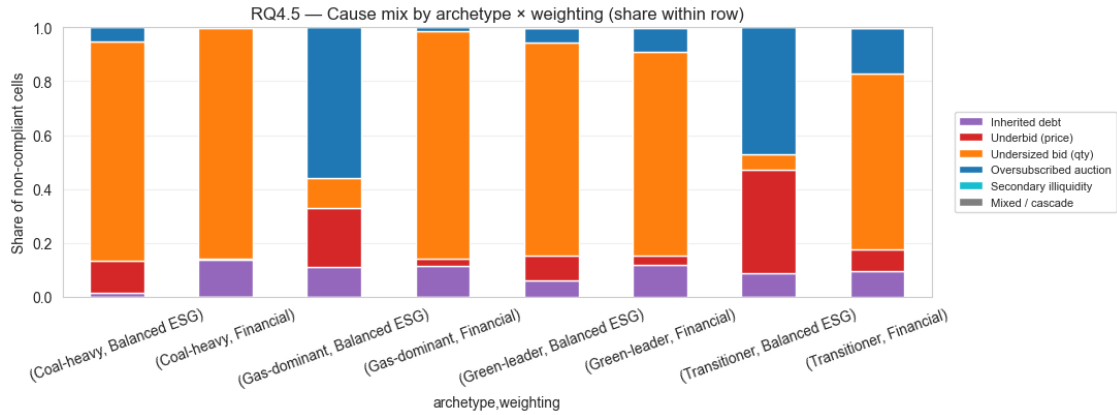
RQ4.5 — Why agents go non-compliant in the converged window



[RQ4.5] Median feature value per dominant cause:

	shortfall_Mt	inherited_debt_Mt	bid_price	␣
↳clearing_price	bid_qty_mult	undersized_qty_Mt	oversubscription_ratio	␣
↳bank_start_Mt	sec_buy_intent_Mt	secondary_buy_unmet_Mt	\	
cause				
Inherited debt	1.666	1.977	165.621	61.
↳210	0.500	1.713	1.535	0.000
↳	3.0	3.0		
Underbid (price)	0.750	0.000	45.000	96.
↳802	2.000	0.000	1.755	0.268
↳	3.0	3.0		
Undersized bid (qty)	1.102	0.088	138.222	45.
↳000	0.500	1.543	1.238	0.000
↳	3.0	3.0		
Oversubscribed auction	0.714	0.000	45.000	45.
↳000	2.000	0.000	1.427	0.173
↳	3.0	3.0		
Secondary illiquidity	0.422	0.000	45.000	45.
↳000	1.074	0.037	1.036	0.000
↳	3.0	3.0		
Mixed / cascade	0.242	0.000	45.000	45.
↳000	1.234	0.000	1.036	0.000
↳	0.0	0.0		

loan_outstanding_Meur	
cause	
Inherited debt	0.0
Underbid (price)	0.0
Undersized bid (qty)	0.0
Oversubscribed auction	0.0
Secondary illiquidity	0.0
Mixed / cascade	0.0



[RQ4.5] Compliant vs non-compliant cell medians (key discriminators):

	feature	compliant_median	non_compliant_median	delta_(nc - c)
0	inherited_debt_Mt	0.000	0.186	0.186
1	bank_start_Mt	1.958	0.000	-1.958
2	bid_qty_mult	1.821	0.500	-1.321
3	bid_to_clearing	1.555	2.276	0.721
4	coverage_gap_Mt	0.000	2.101	2.101

RQ4 summary. RQ4.1–4.3 give the run-level mechanics: did the agents close the coverage gap, did the auction stay diffuse, and did the secondary market actually clear the residual? RQ4.4 attributes the residual debt and emergency-loan usage to the (archetype × weighting) cells — the financial vs ESG split is the most informative cut here, since coal-heavy financial agents are the ones for whom the gap-penalty reward (driven by **effective_penalty_rate**) actively dominates the bid head.

1.9 Year-over-Year Dynamics in the Converged Window — Cross-Sweep Comparison

All preceding sections pool every loaded run into a single default-config population. This section instead splits by **sweep** and overlays every loaded sweep on the same axes, so cross-sweep differences become visible at a glance. The x-axis is always the simulation year (1..N_YEARS) and every series

is restricted to the converged tail window (PELT and plateau diagnostics confirm convergence well within the final 10 000 episodes; the default tail of 200 episodes is enough to capture the steady-state shape, but the knob can be raised below).

Each panel shows the cross-sweep median (within a sweep) plus a ± 1 std fan, so the spread reflects seed dispersion within a sweep, while the vertical separation between coloured medians reflects the cross-sweep effect.

```
[133]: # Cross-sweep year-over-year helpers.
#
# YOY_TAIL_EPISODES bounds how many of the final episodes per run feed
# the year-axis aggregations below. 200 is the working default; raising
# it adds samples but does not change the steady-state shape because the
# runs are already converged at that point (PELT/plateau diagnostics).
import itertools
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display

YOY_TAIL_EPISODES = 200
YOY_SWEEP_FILTER = ['default_seeds']

# Stable sweep ordering and colour map so every panel below uses the
# same colour for a given sweep.
AVAILABLE_SWEEP_NAMES = sorted({k[0] for k in runs.keys()})
if YOY_SWEEP_FILTER:
    SWEEP_NAMES = [sw for sw in AVAILABLE_SWEEP_NAMES if sw in YOY_SWEEP_FILTER]
    if not SWEEP_NAMES:
        raise ValueError(
            f'YOY_SWEEP_FILTER={YOY_SWEEP_FILTER} matched no loaded sweeps. '
            f'Available: {AVAILABLE_SWEEP_NAMES}'
        )
else:
    SWEEP_NAMES = AVAILABLE_SWEEP_NAMES
_palette = plt.get_cmap('tab10').colors
SWEEP_COLORS = {sw: _palette[i % len(_palette)] for i, sw in
    enumerate(SWEEP_NAMES)}
print(f'YoY converged-window section - tail = {YOY_TAIL_EPISODES} episodes per_
    run')
print(f'Available sweeps: {AVAILABLE_SWEEP_NAMES}')
print(f'Sweeps overlaid: {SWEEP_NAMES}')
for sw in SWEEP_NAMES:
    n_runs = sum(1 for k in runs if k[0] == sw)
    print(f' {sw}: {n_runs} run(s)')

def _yoy_year_rows(run, n=YOY_TAIL_EPISODES):
```

```

"""Return year_log rows whose episode is in the last `n` episodes."""
ep = run['ep_df']
yr = run['yr_df']
if ep.empty or yr.empty or 'episode' not in yr.columns or 'episode' not in_
↳ ep.columns:
    return yr.head(0)
last_eps = ep['episode'].tail(min(n, len(ep))).unique()
return yr[yr['episode'].isin(last_eps)]

def _per_sweep_year_stack(metric_fn):
    """For each sweep, build a wide DataFrame indexed by year with one
    column per (variant, seed). `metric_fn(yr_rows)` must return a
    pd.Series indexed by year (or None to skip the run)."""
    out = {}
    for sw in SWEEP_NAMES:
        cols = {}
        for k, run in runs.items():
            if k[0] != sw:
                continue
            yr = _yoy_year_rows(run)
            if yr.empty or 'year' not in yr.columns:
                continue
            s = metric_fn(yr)
            if s is None or len(s) == 0:
                continue
            cols[(k[1], k[2])] = s
        if cols:
            out[sw] = pd.concat(cols, axis=1).sort_index()
    return out

def _overlay_yoy(ax, sweep_data, ylabel, title, *, show_band=True, marker='o'):
    """Overlay one median + ±1 std fan per sweep on `ax`."""
    if not sweep_data:
        ax.text(0.5, 0.5, 'no data', ha='center', va='center', transform=ax.
↳ transAxes)
        ax.set_title(title); ax.set_ylabel(ylabel); ax.set_xlabel('Year')
        return
    for sw in SWEEP_NAMES:
        df = sweep_data.get(sw)
        if df is None or df.empty:
            continue
        color = SWEEP_COLORS[sw]
        med = df.median(axis=1)
        std = df.std(axis=1)
        n = df.shape[1]

```

```

        if show_band and n > 1:
            ax.fill_between(med.index, (med - std).values, (med + std).values,
                           color=color, alpha=0.15)
            ax.plot(med.index, med.values, marker=marker, lw=2.0, color=color,
                   label=f'{sw} (n={n})')
    ax.set_title(title); ax.set_xlabel('Year'); ax.set_ylabel(ylabel)
    ax.grid(alpha=0.3)
    ax.legend(fontsize=8, loc='best')

def _agent_cols_in(yr_df, prefix, max_idx=N_TOTAL):
    """Return existing `{prefix}_A{i}` columns in yr_df, sorted by i."""
    return [f'{prefix}_A{i+1}' for i in range(max_idx)
            if f'{prefix}_A{i+1}' in yr_df.columns]

def _system_sum_by_year(yr_df, prefix, max_idx=N_TOTAL):
    """Per-year mean across episodes of the cross-agent SUM of {prefix}_A*."""
    cols = _agent_cols_in(yr_df, prefix, max_idx)
    if not cols:
        return None
    return yr_df.groupby('year')[cols].mean().sum(axis=1)

def _agent_mean_by_year(yr_df, prefix, max_idx=N_TOTAL):
    """Per-year mean across episodes of the cross-agent MEAN of {prefix}_A*."""
    cols = _agent_cols_in(yr_df, prefix, max_idx)
    if not cols:
        return None
    return yr_df.groupby('year')[cols].mean().mean(axis=1)

def _scalar_mean_by_year(yr_df, col):
    if col not in yr_df.columns:
        return None
    return yr_df.groupby('year')[col].mean()

def _scalar_std_by_year(yr_df, col):
    """Cross-episode std of `col` per year - the within-run volatility."""
    if col not in yr_df.columns:
        return None
    return yr_df.groupby('year')[col].std()

def _yoy_panel_tables(panels, *, units=None):
    """Build companion data tables for a 2x2 YoY figure.

```

Parameters

`panels` : list of (title, sweep_data)
 *`sweep\_data` is the dict returned by `\_per\_sweep\_year\_stack`,
 i.e. `{sweep_name: wide_df indexed by year, columns = (variant,
 seed)}`.*
`units` : optional dict {title: unit_str}, embedded in the printout.

Returns

(per_year_df, summary_df)
 *`per\_year\_df` - one row per (panel, sweep, year) with the
 median (the plotted line), the cross-seed std (the band) and the
 cross-seed min/max/n.
 `summary\_df` - one row per (panel, sweep) summarising the
 per-year median series across years (mean / median / std / min /
 max), i.e. the scalar headline numbers for the panel.*
"""
py_rows, ag_rows = [], []
for title, sd in panels:
 unit = (units or {}).get(title, '')
 if not sd:
 ag_rows.append({'panel': title, 'unit': unit, 'sweep': '(no data)',
 'n_seeds': 0, 'years': 0,
 'mean': np.nan, 'median': np.nan,
 'std': np.nan, 'min': np.nan, 'max': np.nan})
 continue
 for sw, df in sd.items():
 med = df.median(axis=1)
 std = df.std(axis=1)
 mn = df.min(axis=1)
 mx = df.max(axis=1)
 n = df.shape[1]
 for y in med.index:
 py_rows.append({
 'panel': title, 'unit': unit, 'sweep': sw,
 'year': int(y),
 'median_across_seeds': med.loc[y],
 'std_across_seeds': std.loc[y],
 'min_across_seeds': mn.loc[y],
 'max_across_seeds': mx.loc[y],
 'n_seeds': n,
 })
 ag_rows.append({
 'panel': title, 'unit': unit, 'sweep': sw,
 'n_seeds': n, 'years': int(len(med)),


```

        'mean': med.mean(), 'median': med.median(),
        'std': med.std(), 'min': med.min(), 'max': med.max(),
    })
    py_df = pd.DataFrame(py_rows)
    ag_df = pd.DataFrame(ag_rows)
    return py_df, ag_df

def _show_yoy_tables(pannels, *, units=None, float_fmt='{:,.4g}'):
    """Compute and display the per-year and across-year summary tables
    for a 2x2 YoY figure. Returns the two DataFrames so calling cells
    can re-use them if needed."""
    py_df, ag_df = _yoy_panel_tables(pannels, units=units)
    with pd.option_context('display.float_format', float_fmt.format,
                           'display.max_rows', 200):
        print(f'Across-year summary (median series across_
↪{YOY_TAIL_EPISODES}-episode tail, per sweep):')
        display(ag_df)
        print('Per-year detail (median is the plotted line; std is the band;
↪min/max are cross-seed extremes):')
        display(py_df)
    return py_df, ag_df

```

YoY converged-window section - tail = 200 episodes per run
 Available sweeps: ['default_seeds']
 Sweeps overlaid: ['default_seeds']
 default_seeds: 4 run(s)

1.9.1 Price dynamics over the simulation year

Clearing price and secondary price (level + cross-seed spread per sweep), price realism measured by the clearing/fundamental-anchor ratio (1 means the auction tracks the fundamental), and the within-run cross-episode std of the clearing price (a proxy for year-conditional price volatility within the converged window).

```

[134]: fig, axes = plt.subplots(2, 2, figsize=(14, 8))

    _overlay_yoy(
        axes[0, 0],
        _per_sweep_year_stack(lambda yr: _scalar_mean_by_year(yr,
↪'clearing_price')),
        ylabel='€/tonne', title='Auction clearing price')

    _overlay_yoy(
        axes[0, 1],
        _per_sweep_year_stack(lambda yr: _scalar_mean_by_year(yr,
↪'secondary_price')),

```

```

ylabel='€/tonne', title='Secondary-market price')

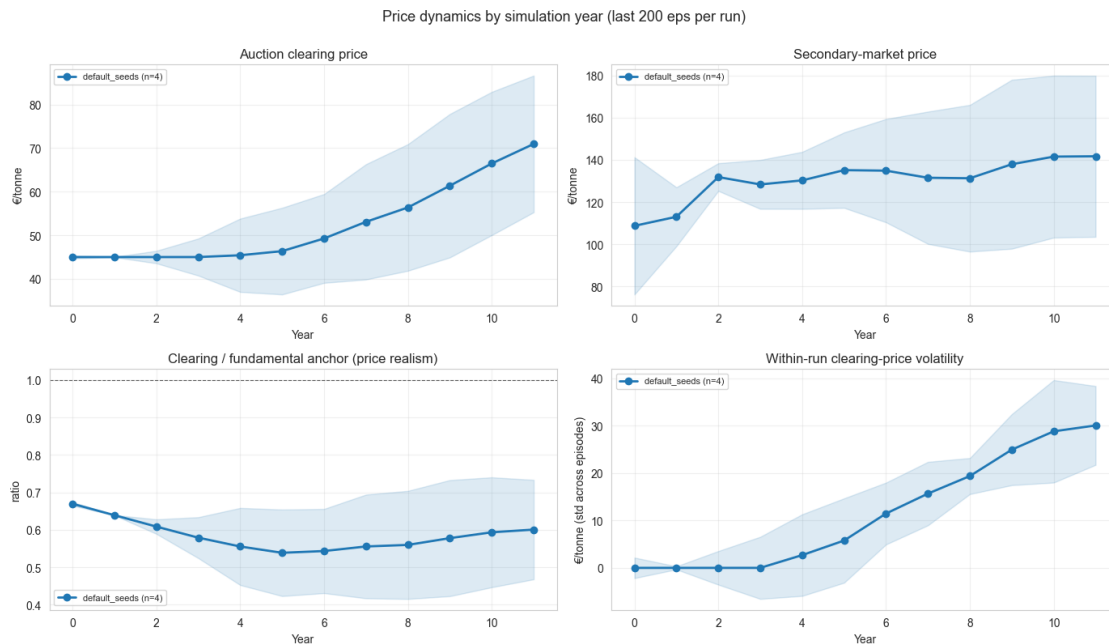
def _clearing_over_anchor(yr):
    if 'clearing_price' not in yr.columns or 'fundamental_anchor' not in yr.
    columns:
        return None
    g = yr.groupby('year')[['clearing_price', 'fundamental_anchor']].mean()
    anchor = g['fundamental_anchor'].replace(0, np.nan)
    return (g['clearing_price'] / anchor)

_overlay_yoy(
    axes[1, 0],
    _per_sweep_year_stack(_clearing_over_anchor),
    ylabel='ratio', title='Clearing / fundamental anchor (price realism)'
    axes[1, 0].axhline(1.0, color='black', lw=0.8, linestyle='--', alpha=0.6)

_overlay_yoy(
    axes[1, 1],
    _per_sweep_year_stack(lambda yr: _scalar_std_by_year(yr, 'clearing_price')),
    ylabel='€/tonne (std across episodes)',
    title='Within-run clearing-price volatility')

plt.suptitle(f'Price dynamics by simulation year (last {YOY_TAIL_EPISODES} eps_
    ↳per run)',
            fontsize=13, y=1.00)
plt.tight_layout(); plt.show()

```



```
[135]: # Companion data tables for the price-dynamics figure above.
# Each row of the per-year table corresponds to one plotted point;
# the across-year summary collapses the median series into mean/std/
# min/max so the thesis text can quote both shape and headline numbers.

def _clearing_over_anchor_tbl(yr):
    if 'clearing_price' not in yr.columns or 'fundamental_anchor' not in yr.
    ↪columns:
        return None
    g = yr.groupby('year')[['clearing_price', 'fundamental_anchor']].mean()
    anchor = g['fundamental_anchor'].replace(0, np.nan)
    return g['clearing_price'] / anchor

_panels_price = [
    ('Auction clearing price',
     _per_sweep_year_stack(lambda yr: _scalar_mean_by_year(yr,
     ↪'clearing_price'))),
    ('Secondary-market price',
     _per_sweep_year_stack(lambda yr: _scalar_mean_by_year(yr,
     ↪'secondary_price'))),
    ('Clearing / fundamental anchor (price realism)',
     _per_sweep_year_stack(_clearing_over_anchor_tbl)),
    ('Within-run clearing-price volatility',
     _per_sweep_year_stack(lambda yr: _scalar_std_by_year(yr,
     ↪'clearing_price'))),
]
_units_price = {
    'Auction clearing price': '€/tonne',
    'Secondary-market price': '€/tonne',
    'Clearing / fundamental anchor (price realism)': 'ratio',
    'Within-run clearing-price volatility': '€/tonne (std across episodes)',
}
_show_yoy_tables(_panels_price, units=_units_price)
```

Across-year summary (median series across 200-episode tail, per sweep):

					panel				unit
↪	sweep	n_seeds	years	mean	median	std	min	max	
0					Auction clearing price				€/tonne
↪	default_seeds	4	12	52.44	47.81	9.303	45	70.94	
1					Secondary-market price				€/tonne
↪	default_seeds	4	12	130.5	131.7	10.13	108.8	141.7	
2					Clearing / fundamental anchor (price realism)				ratio
↪	default_seeds	4	12	0.5853	0.5786	0.03962	0.5388	0.6696	
3					Within-run clearing-price volatility	€/tonne (std across episodes)			
↪	default_seeds	4	12	11.57	8.62	11.85	0	30.07	

Per-year detail (median is the plotted line; std is the band; min/max are cross-

seed extremes):

			panel		unit
	sweep	year	median_across_seeds	std_across_seeds	min_across_seeds
	↪ max_across_seeds	n_seeds			↪
0			Auction clearing price		€/tonne
	↪ default_seeds	0	45	0.3285	45
	↪	45.66	4		↪
1			Auction clearing price		€/tonne
	↪ default_seeds	1	45	0.03264	45
	↪	45.07	4		↪
2			Auction clearing price		€/tonne
	↪ default_seeds	2	45	1.435	45
	↪	47.87	4		↪
3			Auction clearing price		€/tonne
	↪ default_seeds	3	45	4.248	45
	↪	53.5	4		↪
4			Auction clearing price		€/tonne
	↪ default_seeds	4	45.41	8.41	45
	↪	62.08	4		↪
5			Auction clearing price		€/tonne
	↪ default_seeds	5	46.36	9.94	45.23
	↪	65.78	4		↪
6			Auction clearing price		€/tonne
	↪ default_seeds	6	49.26	10.18	45.85
	↪	67.84	4		↪
7			Auction clearing price		€/tonne
	↪ default_seeds	7	53.08	13.22	46.96
	↪	75.9	4		↪
8			Auction clearing price		€/tonne
	↪ default_seeds	8	56.37	14.52	50.08
	↪	80.83	4		↪
9			Auction clearing price		€/tonne
	↪ default_seeds	9	61.36	16.45	47.88
	↪	84.71	4		↪
10			Auction clearing price		€/tonne
	↪ default_seeds	10	66.46	16.43	46.79
	↪	81.88	4		↪
11			Auction clearing price		€/tonne
	↪ default_seeds	11	70.94	15.67	49.45
	↪	81.26	4		↪
12			Secondary-market price		€/tonne
	↪ default_seeds	0	108.8	32.5	56.46
	↪	130.4	4		↪
13			Secondary-market price		€/tonne
	↪ default_seeds	1	113.1	13.96	93.22
	↪	125.9	4		↪

14			Secondary-market price		€/tonne
↪	default_seeds	2	131.9	6.561	127.6
↪	141	4			
15			Secondary-market price		€/tonne
↪	default_seeds	3	128.4	11.59	120.3
↪	147.3	4			
16			Secondary-market price		€/tonne
↪	default_seeds	4	130.3	13.51	114.4
↪	147.5	4			
17			Secondary-market price		€/tonne
↪	default_seeds	5	135.1	17.86	115.1
↪	156.4	4			
18			Secondary-market price		€/tonne
↪	default_seeds	6	134.9	24.47	106.2
↪	155.8	4			
19			Secondary-market price		€/tonne
↪	default_seeds	7	131.5	31.33	109.3
↪	179	4			
20			Secondary-market price		€/tonne
↪	default_seeds	8	131.3	34.74	103.3
↪	181.9	4			
21			Secondary-market price		€/tonne
↪	default_seeds	9	137.9	40.01	100.5
↪	175	4			
22			Secondary-market price		€/tonne
↪	default_seeds	10	141.5	38.36	95.33
↪	184.4	4			
23			Secondary-market price		€/tonne
↪	default_seeds	11	141.7	38.14	102.1
↪	183.6	4			
24	Clearing / fundamental anchor (price realism)				ratio
↪	default_seeds	0	0.6696	0.004888	0.6696
↪	0.6794	4			
25	Clearing / fundamental anchor (price realism)				ratio
↪	default_seeds	1	0.6392	0.0004637	0.6392
↪	0.6402	4			
26	Clearing / fundamental anchor (price realism)				ratio
↪	default_seeds	2	0.6088	0.01942	0.6088
↪	0.6477	4			
27	Clearing / fundamental anchor (price realism)				ratio
↪	default_seeds	3	0.5793	0.05469	0.5793
↪	0.6886	4			
28	Clearing / fundamental anchor (price realism)				ratio
↪	default_seeds	4	0.5557	0.1029	0.5507
↪	0.7597	4			

29	Clearing / fundamental anchor (price realism)				ratio	
↪	default_seeds	5	0.5388	0.1155	0.5258	↪
↪		0.7646	4			
30	Clearing / fundamental anchor (price realism)				ratio	
↪	default_seeds	6	0.5436	0.1124	0.506	↪
↪		0.7486	4			
31	Clearing / fundamental anchor (price realism)				ratio	
↪	default_seeds	7	0.5558	0.1384	0.4917	↪
↪		0.7948	4			
32	Clearing / fundamental anchor (price realism)				ratio	
↪	default_seeds	8	0.5599	0.1442	0.4975	↪
↪		0.8029	4			
33	Clearing / fundamental anchor (price realism)				ratio	
↪	default_seeds	9	0.578	0.155	0.4511	↪
↪		0.798	4			
34	Clearing / fundamental anchor (price realism)				ratio	
↪	default_seeds	10	0.5936	0.1467	0.418	↪
↪		0.7314	4			
35	Clearing / fundamental anchor (price realism)				ratio	
↪	default_seeds	11	0.6008	0.1327	0.4188	↪
↪		0.6882	4			
36	Within-run clearing-price volatility		€/tonne (std across episodes)			
↪	default_seeds	0	0	2.167	0	↪
↪		4.334	4			
37	Within-run clearing-price volatility		€/tonne (std across episodes)			
↪	default_seeds	1	0	0.3261	0	↪
↪		0.6522	4			
38	Within-run clearing-price volatility		€/tonne (std across episodes)			
↪	default_seeds	2	0	3.546	0	↪
↪		7.092	4			
39	Within-run clearing-price volatility		€/tonne (std across episodes)			
↪	default_seeds	3	0	6.555	0	↪
↪		13.11	4			
40	Within-run clearing-price volatility		€/tonne (std across episodes)			
↪	default_seeds	4	2.692	8.614	0.06493	↪
↪		18.54	4			
41	Within-run clearing-price volatility		€/tonne (std across episodes)			
↪	default_seeds	5	5.769	8.906	2.428	↪
↪		21.78	4			
42	Within-run clearing-price volatility		€/tonne (std across episodes)			
↪	default_seeds	6	11.47	6.529	5.214	↪
↪		20.12	4			
43	Within-run clearing-price volatility		€/tonne (std across episodes)			
↪	default_seeds	7	15.68	6.699	9.412	↪
↪		24.88	4			

44	Within-run clearing-price volatility	€/tonne (std across episodes)	
↪ default_seeds	8	19.38	3.808 15.89
↪	24.94 4		
45	Within-run clearing-price volatility	€/tonne (std across episodes)	
↪ default_seeds	9	24.96	7.501 12.01
↪	29.29 4		
46	Within-run clearing-price volatility	€/tonne (std across episodes)	
↪ default_seeds	10	28.83	10.82 7.542
↪	29.76 4		
47	Within-run clearing-price volatility	€/tonne (std across episodes)	
↪ default_seeds	11	30.07	8.318 15.96
↪	35.35 4		

```
[135]: (
    panel
    unit sweep year median_across_seeds std_across_seeds
    min_across_seeds max_across_seeds n_seeds
    0 Auction clearing price
    €/tonne default_seeds 0 45.000000 0.328457
    45.000000 45.656914 4
    1 Auction clearing price
    €/tonne default_seeds 1 45.000000 0.032644
    45.000000 45.065289 4
    2 Auction clearing price
    €/tonne default_seeds 2 45.000000 1.435179
    45.000000 47.870358 4
    3 Auction clearing price
    €/tonne default_seeds 3 45.000000 4.248381
    45.000000 53.496761 4
    4 Auction clearing price
    €/tonne default_seeds 4 45.412453 8.410115
    45.004589 62.082619 4
    5 Auction clearing price
    €/tonne default_seeds 5 46.358322 9.940073
    45.232697 65.778030 4
    6 Auction clearing price
    €/tonne default_seeds 6 49.261395 10.184593
    45.854626 67.838104 4
    7 Auction clearing price
    €/tonne default_seeds 7 53.075989 13.220366
    46.955879 75.900421 4
    8 Auction clearing price
    €/tonne default_seeds 8 56.372330 14.517346
    50.082939 80.828064 4
    9 Auction clearing price
    €/tonne default_seeds 9 61.358124 16.450266
    47.883511 84.714737 4
    10 Auction clearing price
```

€/tonne	default_seeds	10	66.456329	16.425957
46.791126	81.875145	4		
11			Auction clearing price	
€/tonne	default_seeds	11	70.940369	15.667480
49.446930	81.258179	4		
12			Secondary-market price	
€/tonne	default_seeds	0	108.759842	32.500149
56.463219	130.422653	4		
13			Secondary-market price	
€/tonne	default_seeds	1	113.057976	13.958501
93.221443	125.861595	4		
14			Secondary-market price	
€/tonne	default_seeds	2	131.853210	6.560907
127.639328	141.045715	4		
15			Secondary-market price	
€/tonne	default_seeds	3	128.376404	11.588679
120.303947	147.282333	4		
16			Secondary-market price	
€/tonne	default_seeds	4	130.283112	13.512214
114.448372	147.516876	4		
17			Secondary-market price	
€/tonne	default_seeds	5	135.137177	17.863943
115.110939	156.388840	4		
18			Secondary-market price	
€/tonne	default_seeds	6	134.893707	24.468582
106.188759	155.782806	4		
19			Secondary-market price	
€/tonne	default_seeds	7	131.526337	31.327133
109.314209	179.022461	4		
20			Secondary-market price	
€/tonne	default_seeds	8	131.293762	34.741020
103.256271	181.947205	4		
21			Secondary-market price	
€/tonne	default_seeds	9	137.931152	40.006596
100.527412	175.043091	4		
22			Secondary-market price	
€/tonne	default_seeds	10	141.538177	38.357258
95.328514	184.355652	4		
23			Secondary-market price	
€/tonne	default_seeds	11	141.694397	38.140228
102.062218	183.555084	4		
24	Clearing / fundamental anchor (price realism)			
ratio	default_seeds	0	0.669643	0.004888
0.669643	0.679418	4		
25	Clearing / fundamental anchor (price realism)			
ratio	default_seeds	1	0.639241	0.000464
0.639241	0.640168	4		

26	Clearing / fundamental anchor (price realism)			
ratio default_seeds	2	0.608828	0.019417	
0.608828	0.647663	4		
27	Clearing / fundamental anchor (price realism)			
ratio default_seeds	3	0.579258	0.054687	
0.579258	0.688632	4		
28	Clearing / fundamental anchor (price realism)			
ratio default_seeds	4	0.555689	0.102910	
0.550698	0.759673	4		
29	Clearing / fundamental anchor (price realism)			
ratio default_seeds	5	0.538849	0.115539	
0.525765	0.764575	4		
30	Clearing / fundamental anchor (price realism)			
ratio default_seeds	6	0.543595	0.112386	
0.506002	0.748587	4		
31	Clearing / fundamental anchor (price realism)			
ratio default_seeds	7	0.555774	0.138434	
0.491688	0.794775	4		
32	Clearing / fundamental anchor (price realism)			
ratio default_seeds	8	0.559949	0.144201	
0.497476	0.802868	4		
33	Clearing / fundamental anchor (price realism)			
ratio default_seeds	9	0.578008	0.154965	
0.451074	0.798032	4		
34	Clearing / fundamental anchor (price realism)			
ratio default_seeds	10	0.593627	0.146726	
0.417966	0.731357	4		
35	Clearing / fundamental anchor (price realism)			
ratio default_seeds	11	0.600840	0.132698	
0.418798	0.688228	4		
36	Within-run clearing-price volatility	€/tonne (std across		
episodes)	default_seeds	0	0.000000	2.166894
0.000000	4.333788	4		
37	Within-run clearing-price volatility	€/tonne (std across		
episodes)	default_seeds	1	0.000000	0.326084
0.000000	0.652167	4		
38	Within-run clearing-price volatility	€/tonne (std across		
episodes)	default_seeds	2	0.000000	3.545781
0.000000	7.091562	4		
39	Within-run clearing-price volatility	€/tonne (std across		
episodes)	default_seeds	3	0.000000	6.554780
0.000000	13.109561	4		
40	Within-run clearing-price volatility	€/tonne (std across		
episodes)	default_seeds	4	2.691767	8.614256
0.064932	18.543728	4		
41	Within-run clearing-price volatility	€/tonne (std across		
episodes)	default_seeds	5	5.769314	8.906443

2.427817	21.782230	4						
42	Within-run clearing-price volatility €/tonne (std across episodes)							
default_seeds	6	11.470092	6.529057					
5.214242	20.124584	4						
43	Within-run clearing-price volatility €/tonne (std across episodes)							
default_seeds	7	15.679464	6.698502					
9.412136	24.884315	4						
44	Within-run clearing-price volatility €/tonne (std across episodes)							
default_seeds	8	19.384602	3.808328					
15.885598	24.941202	4						
45	Within-run clearing-price volatility €/tonne (std across episodes)							
default_seeds	9	24.959063	7.501497					
12.009020	29.287025	4						
46	Within-run clearing-price volatility €/tonne (std across episodes)							
default_seeds	10	28.826925	10.820453					
7.541983	29.755806	4						
47	Within-run clearing-price volatility €/tonne (std across episodes)							
default_seeds	11	30.068550	8.318158					
15.955498	35.345726	4,						

sweep	n_seeds	years	panel					unit
			mean	median	std	min	max	
0			Auction clearing price					€/tonne
default_seeds	4	12	52.436275	47.809860	9.303043	45.000000		
70.940369								
1			Secondary-market price					€/tonne
default_seeds	4	12	130.528763	131.689774	10.126948	108.759842		
141.694397								
2	Clearing / fundamental anchor (price realism)							ratio
default_seeds	4	12	0.585275	0.578633	0.039618	0.538849		
0.669643								
3	Within-run clearing-price volatility €/tonne (std across episodes)							
default_seeds	4	12	11.570815	8.619703	11.851970	0.000000		
30.068550)								

1.9.2 Compliance and penalty pressure over the simulation year

System-aggregate coverage gap (Mt remaining after the auction), end-of-year carry-forward (debt cascade indicator), the share of (agent, year) cells with a non-zero shortfall (incidence), and the system-aggregate penalty paid per year.

```
[136]: fig, axes = plt.subplots(2, 2, figsize=(14, 8))

_overlay_yoy(
    axes[0, 0],
    _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'coverage_gap')),
    ylabel='Mt', title='System coverage gap (pre-secondary)')
```

```

_overlay_yoy(
    axes[0, 1],
    _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr,
↳ 'carry_forward_end')),
    ylabel='Mt', title='End-of-year carry-forward (system)')

def _shortfall_incidence(yr):
    cols = _agent_cols_in(yr, 'shortfall')
    if not cols:
        return None
    long = yr.melt(id_vars=['year'], value_vars=cols, value_name='sf')
    long['flag'] = (long['sf'].fillna(0) > 0).astype(float)
    return long.groupby('year')['flag'].mean() # share of (ep, agent) cells

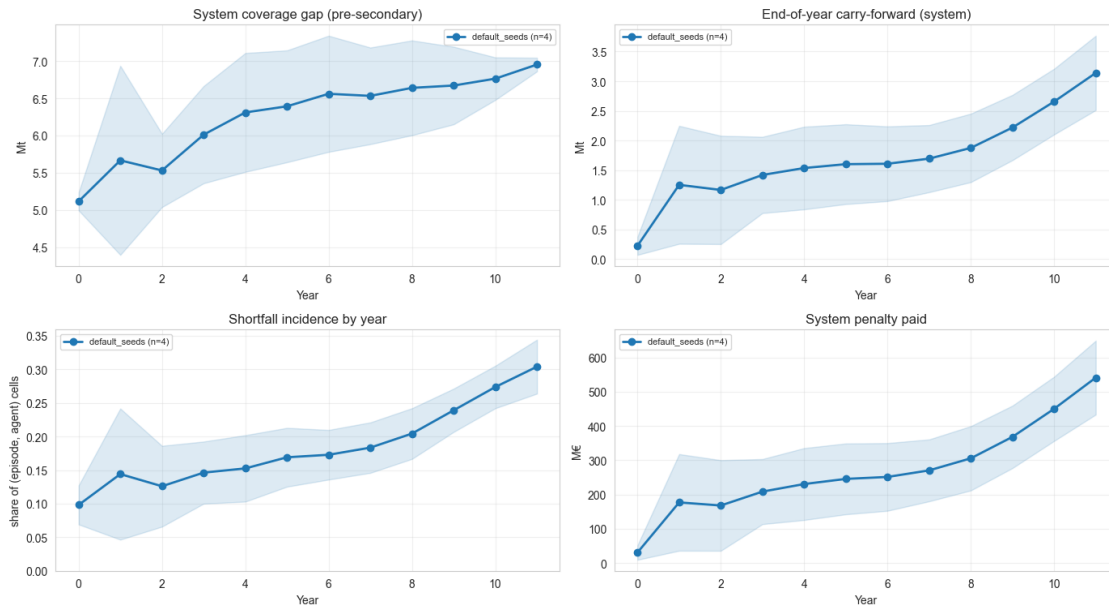
_overlay_yoy(
    axes[1, 0],
    _per_sweep_year_stack(_shortfall_incidence),
    ylabel='share of (episode, agent) cells',
    title='Shortfall incidence by year')
axes[1, 0].set_ylim(bottom=0)

_overlay_yoy(
    axes[1, 1],
    _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'penalty')),
    ylabel='M€', title='System penalty paid')

plt.suptitle(f'Compliance pressure by simulation year (last {YOY_TAIL_EPISODES}
↳ eps per run)',
            fontsize=13, y=1.00)
plt.tight_layout(); plt.show()

```

Compliance pressure by simulation year (last 200 eps per run)



```
[137]: # Companion data tables for the compliance / penalty-pressure figure.
def _shortfall_incidence_tbl(yr):
    cols = _agent_cols_in(yr, 'shortfall')
    if not cols:
        return None
    long = yr.melt(id_vars=['year'], value_vars=cols, value_name='sf')
    long['flag'] = (long['sf'].fillna(0) > 0).astype(float)
    return long.groupby('year')['flag'].mean()

_panels_compliance = [
    ('System coverage gap (pre-secondary)',
     _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'coverage_gap'))),
    ('End-of-year carry-forward (system)',
     _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr,
     ↪ 'carry_forward_end'))),
    ('Shortfall incidence by year',
     _per_sweep_year_stack(_shortfall_incidence_tbl)),
    ('System penalty paid',
     _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'penalty'))),
]

_units_compliance = {
    'System coverage gap (pre-secondary)': 'Mt',
    'End-of-year carry-forward (system)': 'Mt',
    'Shortfall incidence by year': 'share of (episode, agent) cells',
    'System penalty paid': 'M€',
}
```

```
_show_yoy_tables(_panels_compliance, units=_units_compliance)
```

Across-year summary (median series across 200-episode tail, per sweep):

				panel				unit	
↪sweep	n_seeds	years	mean	median	std	min	max		
0	System coverage gap (pre-secondary)							Mt	↪
↪default_seeds	4	12	6.262	6.462	0.5628	5.116	6.953		
1	End-of-year carry-forward (system)							Mt	↪
↪default_seeds	4	12	1.699	1.604	0.7437	0.2222	3.14		
2	Shortfall incidence by year share of (episode, agent) cells								↪
↪default_seeds	4	12	0.1845	0.1709	0.06107	0.09813	0.3041		
3	System penalty paid							M€	↪
↪default_seeds	4	12	270.6	248.4	134.6	30.83	541		

Per-year detail (median is the plotted line; std is the band; min/max are cross-seed extremes):

			panel		unit	
↪ sweep	year	median_across_seeds	std_across_seeds	min_across_seeds		↪
↪max_across_seeds	n_seeds					
0	System coverage gap (pre-secondary)				Mt	↪
↪default_seeds	0	5.116	0.1219	5.011		↪
↪	5.303	4				
1	System coverage gap (pre-secondary)				Mt	↪
↪default_seeds	1	5.666	1.269	4.201		↪
↪	7.287	4				
2	System coverage gap (pre-secondary)				Mt	↪
↪default_seeds	2	5.531	0.492	4.971		↪
↪	5.935	4				
3	System coverage gap (pre-secondary)				Mt	↪
↪default_seeds	3	6.011	0.6528	5.123		↪
↪	6.704	4				
4	System coverage gap (pre-secondary)				Mt	↪
↪default_seeds	4	6.31	0.7978	4.953		↪
↪	6.802	4				
5	System coverage gap (pre-secondary)				Mt	↪
↪default_seeds	5	6.392	0.7509	5.073		↪
↪	6.787	4				
6	System coverage gap (pre-secondary)				Mt	↪
↪default_seeds	6	6.56	0.7796	5.119		↪
↪	6.788	4				
7	System coverage gap (pre-secondary)				Mt	↪
↪default_seeds	7	6.532	0.6486	5.414		↪
↪	6.85	4				
8	System coverage gap (pre-secondary)				Mt	↪
↪default_seeds	8	6.64	0.6362	5.512		↪
↪	6.958	4				

9	System coverage gap (pre-secondary)				Mt	┐
	└default_seeds	9	6.671	0.5209		5.856 ┐
	└	7.101	4			
10	System coverage gap (pre-secondary)				Mt	┐
	└default_seeds	10	6.763	0.2855		6.377 ┐
	└	7.073	4			
11	System coverage gap (pre-secondary)				Mt	┐
	└default_seeds	11	6.953	0.09165		6.859 ┐
	└	7.067	4			
12	End-of-year carry-forward (system)				Mt	┐
	└default_seeds	0	0.2222	0.1535		0.1389 ┐
	└	0.4835	4			
13	End-of-year carry-forward (system)				Mt	┐
	└default_seeds	1	1.251	0.9954		0.4106 ┐
	└	2.78	4			
14	End-of-year carry-forward (system)				Mt	┐
	└default_seeds	2	1.165	0.9142		0.5744 ┐
	└	2.648	4			
15	End-of-year carry-forward (system)				Mt	┐
	└default_seeds	3	1.418	0.6436		0.8919 ┐
	└	2.259	4			
16	End-of-year carry-forward (system)				Mt	┐
	└default_seeds	4	1.535	0.6974		0.8564 ┐
	└	2.208	4			
17	End-of-year carry-forward (system)				Mt	┐
	└default_seeds	5	1.6	0.6738		0.8631 ┐
	└	2.136	4			
18	End-of-year carry-forward (system)				Mt	┐
	└default_seeds	6	1.607	0.6309		0.9859 ┐
	└	2.259	4			
19	End-of-year carry-forward (system)				Mt	┐
	└default_seeds	7	1.694	0.5665		1.06 ┐
	└	2.382	4			
20	End-of-year carry-forward (system)				Mt	┐
	└default_seeds	8	1.875	0.5778		1.167 ┐
	└	2.466	4			
21	End-of-year carry-forward (system)				Mt	┐
	└default_seeds	9	2.218	0.5508		1.437 ┐
	└	2.586	4			
22	End-of-year carry-forward (system)				Mt	┐
	└default_seeds	10	2.656	0.5548		1.756 ┐
	└	3.039	4			
23	End-of-year carry-forward (system)				Mt	┐
	└default_seeds	11	3.14	0.6273		1.998 ┐
	└	3.396	4			

24	Shortfall incidence by year share of (episode, agent) cells				▢
↳ default_seeds	0	0.09813	0.02907	0.06875	▢
↳	0.1338	4			
25	Shortfall incidence by year share of (episode, agent) cells				▢
↳ default_seeds	1	0.1441	0.09766	0.04688	▢
↳	0.2838	4			
26	Shortfall incidence by year share of (episode, agent) cells				▢
↳ default_seeds	2	0.1259	0.06018	0.065	▢
↳	0.2106	4			
27	Shortfall incidence by year share of (episode, agent) cells				▢
↳ default_seeds	3	0.1462	0.04625	0.09875	▢
↳	0.2069	4			
28	Shortfall incidence by year share of (episode, agent) cells				▢
↳ default_seeds	4	0.1525	0.0495	0.08687	▢
↳	0.1969	4			
29	Shortfall incidence by year share of (episode, agent) cells				▢
↳ default_seeds	5	0.1691	0.0438	0.1006	▢
↳	0.1931	4			
30	Shortfall incidence by year share of (episode, agent) cells				▢
↳ default_seeds	6	0.1728	0.03696	0.1144	▢
↳	0.1931	4			
31	Shortfall incidence by year share of (episode, agent) cells				▢
↳ default_seeds	7	0.1834	0.03759	0.1331	▢
↳	0.2225	4			
32	Shortfall incidence by year share of (episode, agent) cells				▢
↳ default_seeds	8	0.2044	0.03775	0.1675	▢
↳	0.2481	4			
33	Shortfall incidence by year share of (episode, agent) cells				▢
↳ default_seeds	9	0.2391	0.03226	0.21	▢
↳	0.2744	4			
34	Shortfall incidence by year share of (episode, agent) cells				▢
↳ default_seeds	10	0.2737	0.03166	0.245	▢
↳	0.3137	4			
35	Shortfall incidence by year share of (episode, agent) cells				▢
↳ default_seeds	11	0.3041	0.04017	0.2606	▢
↳	0.3569	4			
36	System penalty paid			M€	▢
↳ default_seeds	0	30.83	21.29	19.27	▢
↳	67.09	4			
37	System penalty paid			M€	▢
↳ default_seeds	1	177	140.9	58.22	▢
↳	393.6	4			
38	System penalty paid			M€	▢
↳ default_seeds	2	168.1	132.1	82.91	▢
↳	382.7	4			

39	System penalty paid			M€	
↪ default_seeds	3	208.5	94.94		131
↪	333	4			
40	System penalty paid			M€	
↪ default_seeds	4	230.6	104.8		128.6
↪	332	4			
41	System penalty paid			M€	
↪ default_seeds	5	245.6	103.3		131.7
↪	327.2	4			
42	System penalty paid			M€	
↪ default_seeds	6	251.3	98.73		153.9
↪	353	4			
43	System penalty paid			M€	
↪ default_seeds	7	270.4	90.25		169.4
↪	380.1	4			
44	System penalty paid			M€	
↪ default_seeds	8	305.6	93.75		190.4
↪	400.6	4			
45	System penalty paid			M€	
↪ default_seeds	9	368.2	91.08		239.5
↪	430.3	4			
46	System penalty paid			M€	
↪ default_seeds	10	450	93.67		297.6
↪	514.6	4			
47	System penalty paid			M€	
↪ default_seeds	11	541	107.8		345.4
↪	586.1	4			

[137]:	(	panel	unit
sweep	year	median_across_seeds	std_across_seeds
max_across_seeds	n_seeds		
0	System coverage gap (pre-secondary)		Mt
default_seeds	0	5.116022	0.121915
5.302558	4		5.010922
1	System coverage gap (pre-secondary)		Mt
default_seeds	1	5.666437	1.269411
7.286503	4		4.201180
2	System coverage gap (pre-secondary)		Mt
default_seeds	2	5.530775	0.492005
5.935021	4		4.971396
3	System coverage gap (pre-secondary)		Mt
default_seeds	3	6.011251	0.652823
6.703637	4		5.122776
4	System coverage gap (pre-secondary)		Mt
default_seeds	4	6.309737	0.797794
6.802486	4		4.952747
5	System coverage gap (pre-secondary)		Mt

default_seeds	5	6.392026	0.750940	5.072672
6.786938	4			
6	System coverage gap (pre-secondary)			Mt
default_seeds	6	6.559943	0.779591	5.118542
6.787553	4			
7	System coverage gap (pre-secondary)			Mt
default_seeds	7	6.532311	0.648577	5.413662
6.850002	4			
8	System coverage gap (pre-secondary)			Mt
default_seeds	8	6.639850	0.636204	5.512203
6.957816	4			
9	System coverage gap (pre-secondary)			Mt
default_seeds	9	6.671042	0.520936	5.855717
7.101438	4			
10	System coverage gap (pre-secondary)			Mt
default_seeds	10	6.763318	0.285530	6.377172
7.073412	4			
11	System coverage gap (pre-secondary)			Mt
default_seeds	11	6.952510	0.091654	6.859487
7.066913	4			
12	End-of-year carry-forward (system)			Mt
default_seeds	0	0.222211	0.153472	0.138875
0.483510	4			
13	End-of-year carry-forward (system)			Mt
default_seeds	1	1.251398	0.995398	0.410648
2.779818	4			
14	End-of-year carry-forward (system)			Mt
default_seeds	2	1.165136	0.914185	0.574382
2.648426	4			
15	End-of-year carry-forward (system)			Mt
default_seeds	3	1.418232	0.643587	0.891879
2.258808	4			
16	End-of-year carry-forward (system)			Mt
default_seeds	4	1.534831	0.697397	0.856428
2.207970	4			
17	End-of-year carry-forward (system)			Mt
default_seeds	5	1.600306	0.673813	0.863097
2.136389	4			
18	End-of-year carry-forward (system)			Mt
default_seeds	6	1.607293	0.630893	0.985907
2.258587	4			
19	End-of-year carry-forward (system)			Mt
default_seeds	7	1.693518	0.566485	1.059673
2.381708	4			
20	End-of-year carry-forward (system)			Mt
default_seeds	8	1.874557	0.577825	1.167292
2.466290	4			

21	End-of-year carry-forward (system)			Mt
default_seeds	9	2.218000	0.550796	1.437149
2.586451	4			
22	End-of-year carry-forward (system)			Mt
default_seeds	10	2.656165	0.554779	1.756419
3.038503	4			
23	End-of-year carry-forward (system)			Mt
default_seeds	11	3.140377	0.627325	1.998199
3.396197	4			
24	Shortfall incidence by year	share of (episode, agent)	cells	
default_seeds	0	0.098125	0.029072	0.068750
0.133750	4			
25	Shortfall incidence by year	share of (episode, agent)	cells	
default_seeds	1	0.144062	0.097655	0.046875
0.283750	4			
26	Shortfall incidence by year	share of (episode, agent)	cells	
default_seeds	2	0.125938	0.060184	0.065000
0.210625	4			
27	Shortfall incidence by year	share of (episode, agent)	cells	
default_seeds	3	0.146250	0.046248	0.098750
0.206875	4			
28	Shortfall incidence by year	share of (episode, agent)	cells	
default_seeds	4	0.152500	0.049501	0.086875
0.196875	4			
29	Shortfall incidence by year	share of (episode, agent)	cells	
default_seeds	5	0.169063	0.043799	0.100625
0.193125	4			
30	Shortfall incidence by year	share of (episode, agent)	cells	
default_seeds	6	0.172812	0.036963	0.114375
0.193125	4			
31	Shortfall incidence by year	share of (episode, agent)	cells	
default_seeds	7	0.183438	0.037587	0.133125
0.222500	4			
32	Shortfall incidence by year	share of (episode, agent)	cells	
default_seeds	8	0.204375	0.037747	0.167500
0.248125	4			
33	Shortfall incidence by year	share of (episode, agent)	cells	
default_seeds	9	0.239063	0.032263	0.210000
0.274375	4			
34	Shortfall incidence by year	share of (episode, agent)	cells	
default_seeds	10	0.273750	0.031662	0.245000
0.313750	4			
35	Shortfall incidence by year	share of (episode, agent)	cells	
default_seeds	11	0.304063	0.040173	0.260625
0.356875	4			
36	System penalty paid			M€
default_seeds	0	30.831669	21.294420	19.268955

67.087540	4				
37		System penalty paid		M€	
default_seeds	1	176.981720	140.928757	58.219070	
393.580536	4				
38		System penalty paid		M€	
default_seeds	2	168.052780	132.137253	82.911125	
382.743530	4				
39		System penalty paid		M€	
default_seeds	3	208.531982	94.943047	131.007004	
333.039612	4				
40		System penalty paid		M€	
default_seeds	4	230.562531	104.848450	128.612701	
332.015991	4				
41		System penalty paid		M€	
default_seeds	5	245.571655	103.333069	131.698898	
327.175293	4				
42		System penalty paid		M€	
default_seeds	6	251.312851	98.732681	153.939545	
352.999878	4				
43		System penalty paid		M€	
default_seeds	7	270.444214	90.247757	169.413742	
380.116669	4				
44		System penalty paid		M€	
default_seeds	8	305.589172	93.749298	190.370575	
400.639648	4				
45		System penalty paid		M€	
default_seeds	9	368.171661	91.078644	239.459061	
430.313141	4				
46		System penalty paid		M€	
default_seeds	10	450.001831	93.674797	297.570679	
514.605347	4				
47		System penalty paid		M€	
default_seeds	11	541.008606	107.796700	345.409424	
586.131348	4,				

			panel			unit	
sweep	n_seeds	years	mean	median	std	min	max
0	System coverage gap (pre-secondary)						Mt
default_seeds	4	12	6.262102	6.462169	0.562847	5.116022	
6.952510							
1	End-of-year carry-forward (system)						Mt
default_seeds	4	12	1.698502	1.603799	0.743709	0.222211	
3.140377							
2	Shortfall incidence by year share of (episode, agent) cells						
default_seeds	4	12	0.184453	0.170937	0.061067	0.098125	
0.304063							
3	System penalty paid						M€
default_seeds	4	12	270.588409	248.442261	134.594803	30.831669	

541.008606)

1.9.3 Decarbonisation and investment over the simulation year

Cross-agent mean green generation share (does the fleet green up over the year axis?), system emissions (Mt), system invest cost (M€), and MAC fuel-switching reductions (Mt abated). All four panels read from per-agent year_log columns aggregated across the converged tail.

```
[138]: fig, axes = plt.subplots(2, 2, figsize=(14, 8))

    _overlay_yoy(
        axes[0, 0],
        _per_sweep_year_stack(lambda yr: _agent_mean_by_year(yr, 'green_frac')),
        ylabel='green frac [0, 1]', title='Mean green generation share')
    axes[0, 0].set_ylim(0, 1)

    _overlay_yoy(
        axes[0, 1],
        _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'emissions')),
        ylabel='Mt CO ', title='System emissions')

    _overlay_yoy(
        axes[1, 0],
        _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'invest_cost')),
        ylabel='M€', title='System investment spend')

    _overlay_yoy(
        axes[1, 1],
        _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'mac_reduction')),
        ylabel='Mt abated', title='MAC fuel-switching reductions')

    plt.suptitle(f'Decarbonisation & investment by simulation year (last_
        ↳{YOY_TAIL_EPISODES} eps per run)',
                fontsize=13, y=1.00)
    plt.tight_layout(); plt.show()
```

```
C:\Users\danie\AppData\Local\Temp\ipykernel_23912\1586708090.py:26: UserWarning:
Glyph 8712 (\N{ELEMENT OF}) missing from font(s) Arial.
```

```
    plt.tight_layout(); plt.show()
```

```
C:\Users\danie\AppData\Local\Temp\ipykernel_23912\1586708090.py:26: UserWarning:
Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.
```

```
    plt.tight_layout(); plt.show()
```

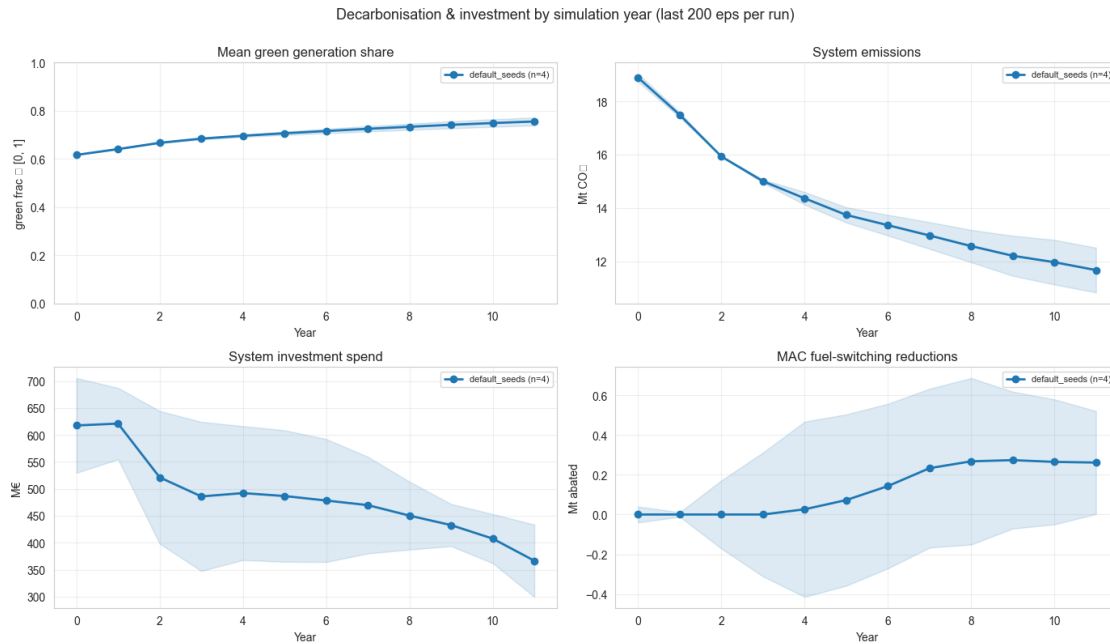
```
C:\Users\danie\AppData\Roaming\Python\Python312\site-
packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8712 (\N{ELEMENT
OF}) missing from font(s) Arial.
```

```
    fig.canvas.print_figure(bytes_io, **kw)
```

```
C:\Users\danie\AppData\Roaming\Python\Python312\site-
packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT
```

TWO}) missing from font(s) Arial.

```
fig.canvas.print_figure(bytes_io, **kw)
```



```
[139]: # Companion data tables for the decarbonisation / investment figure.
_panels_decarb = [
    ('Mean green generation share',
     _per_sweep_year_stack(lambda yr: _agent_mean_by_year(yr, 'green_frac'))),
    ('System emissions',
     _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'emissions'))),
    ('System investment spend',
     _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'invest_cost'))),
    ('MAC fuel-switching reductions',
     _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr,
     ↪ 'mac_reduction'))),
]
_units_decarb = {
    'Mean green generation share': 'green frac [0, 1]',
    'System emissions': 'Mt CO ',
    'System investment spend': 'M€',
    'MAC fuel-switching reductions': 'Mt abated',
}
_show_yoy_tables(_panels_decarb, units=_units_decarb)
```

Across-year summary (median series across 200-episode tail, per sweep):

		panel	unit	sweep	n_seeds	
↪ years	mean	median	std	min	max	

0	Mean green generation share	green frac	[0, 1]	default_seeds	4	┐
↪12	0.7029	0.7115	0.04368	0.6169	0.7558	
1	System emissions		Mt CO	default_seeds	4	┐
↪12	14.18	13.54	2.275	11.67	18.88	
2	System investment spend		M€	default_seeds	4	┐
↪12	485.8	482.1	74.9	366.5	621.1	
3	MAC fuel-switching reductions		Mt abated	default_seeds	4	┐
↪12	0.1284	0.1078	0.1232	0	0.2733	

Per-year detail (median is the plotted line; std is the band; min/max are cross-seed extremes):

	panel	unit	sweep	year		
↪median_across_seeds	std_across_seeds	min_across_seeds	max_across_seeds		┐	
↪n_seeds						
0	Mean green generation share	green frac	[0, 1]	default_seeds	0	┐
↪	0.6169	0.000319	0.6164	0.6172	4	
1	Mean green generation share	green frac	[0, 1]	default_seeds	1	┐
↪	0.6411	0.000771	0.6399	0.6418	4	
2	Mean green generation share	green frac	[0, 1]	default_seeds	2	┐
↪	0.6671	0.002288	0.6631	0.6682	4	
3	Mean green generation share	green frac	[0, 1]	default_seeds	3	┐
↪	0.6845	0.003558	0.6781	0.6861	4	
4	Mean green generation share	green frac	[0, 1]	default_seeds	4	┐
↪	0.6963	0.00532	0.6868	0.6988	4	
5	Mean green generation share	green frac	[0, 1]	default_seeds	5	┐
↪	0.7068	0.007118	0.6942	0.7095	4	
6	Mean green generation share	green frac	[0, 1]	default_seeds	6	┐
↪	0.7161	0.009248	0.7009	0.721	4	
7	Mean green generation share	green frac	[0, 1]	default_seeds	7	┐
↪	0.7253	0.01115	0.7077	0.732	4	
8	Mean green generation share	green frac	[0, 1]	default_seeds	8	┐
↪	0.7338	0.01295	0.7145	0.7428	4	
9	Mean green generation share	green frac	[0, 1]	default_seeds	9	┐
↪	0.742	0.01454	0.721	0.7528	4	
10	Mean green generation share	green frac	[0, 1]	default_seeds	10	┐
↪	0.7492	0.01585	0.7272	0.7621	4	
11	Mean green generation share	green frac	[0, 1]	default_seeds	11	┐
↪	0.7558	0.01651	0.7337	0.7701	4	
12	System emissions		Mt CO	default_seeds	0	┐
↪	18.88	0.1518	18.62	18.95	4	
13	System emissions		Mt CO	default_seeds	1	┐
↪	17.49	0.0923	17.39	17.59	4	
14	System emissions		Mt CO	default_seeds	2	┐
↪	15.93	0.01199	15.92	15.94	4	
15	System emissions		Mt CO	default_seeds	3	┐
↪	15	0.06191	14.92	15.06	4	

16	System emissions	Mt CO	default_seeds	4	┐
↪	14.36 0.2373	14.08	14.65	4	
17	System emissions	Mt CO	default_seeds	5	┐
↪	13.74 0.2867	13.63	14.25	4	
18	System emissions	Mt CO	default_seeds	6	┐
↪	13.35 0.3865	12.95	13.89	4	
19	System emissions	Mt CO	default_seeds	7	┐
↪	12.97 0.5019	12.29	13.48	4	
20	System emissions	Mt CO	default_seeds	8	┐
↪	12.57 0.607	11.76	13.22	4	
21	System emissions	Mt CO	default_seeds	9	┐
↪	12.2 0.7523	11.25	13.01	4	
22	System emissions	Mt CO	default_seeds	10	┐
↪	11.96 0.8367	10.79	12.72	4	
23	System emissions	Mt CO	default_seeds	11	┐
↪	11.67 0.8392	10.53	12.44	4	
24	System investment spend	M€	default_seeds	0	┐
↪	617.6 87.95	518.3	725.1	4	
25	System investment spend	M€	default_seeds	1	┐
↪	621.1 66.23	528.3	681.4	4	
26	System investment spend	M€	default_seeds	2	┐
↪	521.2 123.2	437.5	705.4	4	
27	System investment spend	M€	default_seeds	3	┐
↪	485.9 138.4	336	635.7	4	
28	System investment spend	M€	default_seeds	4	┐
↪	492.1 124.3	378.3	628.3	4	
29	System investment spend	M€	default_seeds	5	┐
↪	486.6 122	390.5	630.2	4	
30	System investment spend	M€	default_seeds	6	┐
↪	478.3 114.1	390.1	614	4	
31	System investment spend	M€	default_seeds	7	┐
↪	469.8 89.71	401.5	580.4	4	
32	System investment spend	M€	default_seeds	8	┐
↪	450.3 63.1	393.6	521.5	4	
33	System investment spend	M€	default_seeds	9	┐
↪	432.7 39.08	375.9	463.9	4	
34	System investment spend	M€	default_seeds	10	┐
↪	407.6 45.52	336.4	445	4	
35	System investment spend	M€	default_seeds	11	┐
↪	366.5 67.15	286.8	450.9	4	
36	MAC fuel-switching reductions	Mt abated	default_seeds	0	┐
↪	0 0.04006	0	0.08012	4	
37	MAC fuel-switching reductions	Mt abated	default_seeds	1	┐
↪	0 0.01124	0	0.02249	4	
38	MAC fuel-switching reductions	Mt abated	default_seeds	2	┐
↪	0 0.1713	0	0.3427	4	

39	MAC fuel-switching reductions		Mt abated	default_seeds	3	␣
↪	0	0.3114	0	0.6228	4	
40	MAC fuel-switching reductions		Mt abated	default_seeds	4	␣
↪	0.02631	0.4405	0	0.8975	4	
41	MAC fuel-switching reductions		Mt abated	default_seeds	5	␣
↪	0.07263	0.4306	0.01272	0.9082	4	
42	MAC fuel-switching reductions		Mt abated	default_seeds	6	␣
↪	0.1429	0.4137	0.03387	0.9186	4	
43	MAC fuel-switching reductions		Mt abated	default_seeds	7	␣
↪	0.2329	0.3992	0.0467	0.9303	4	
44	MAC fuel-switching reductions		Mt abated	default_seeds	8	␣
↪	0.2673	0.4184	0.0836	0.9976	4	
45	MAC fuel-switching reductions		Mt abated	default_seeds	9	␣
↪	0.2733	0.344	0.07721	0.8206	4	
46	MAC fuel-switching reductions		Mt abated	default_seeds	10	␣
↪	0.2645	0.3142	0.06647	0.7491	4	
47	MAC fuel-switching reductions		Mt abated	default_seeds	11	␣
↪	0.2612	0.2589	0.1044	0.6692	4	

[139]: (

	panel	unit	sweep	year
median_across_seeds	std_across_seeds	min_across_seeds	max_across_seeds	
n_seeds				
0	Mean green generation share	green frac	[0, 1]	default_seeds
0.616913	0.000319	0.616425	0.617167	4
1	Mean green generation share	green frac	[0, 1]	default_seeds
0.641060	0.000771	0.639935	0.641798	4
2	Mean green generation share	green frac	[0, 1]	default_seeds
0.667134	0.002288	0.663124	0.668162	4
3	Mean green generation share	green frac	[0, 1]	default_seeds
0.684523	0.003558	0.678104	0.686126	4
4	Mean green generation share	green frac	[0, 1]	default_seeds
0.696332	0.005320	0.686817	0.698770	4
5	Mean green generation share	green frac	[0, 1]	default_seeds
0.706792	0.007118	0.694237	0.709462	4
6	Mean green generation share	green frac	[0, 1]	default_seeds
0.716135	0.009248	0.700945	0.720995	4
7	Mean green generation share	green frac	[0, 1]	default_seeds
0.725305	0.011154	0.707730	0.731978	4
8	Mean green generation share	green frac	[0, 1]	default_seeds
0.733782	0.012954	0.714461	0.742778	4
9	Mean green generation share	green frac	[0, 1]	default_seeds
0.741958	0.014543	0.720958	0.752849	4
10	Mean green generation share	green frac	[0, 1]	default_seeds
0.749222	0.015854	0.727207	0.762101	4
11	Mean green generation share	green frac	[0, 1]	default_seeds
0.755758	0.016511	0.733707	0.770081	4
12	System emissions	Mt CO	default_seeds	0

18.883976	0.151826	18.615524	18.947956	4	
13	System emissions		Mt CO default_seeds	1	
17.488400	0.092298	17.387444	17.587048	4	
14	System emissions		Mt CO default_seeds	2	
15.927521	0.011986	15.915089	15.943642	4	
15	System emissions		Mt CO default_seeds	3	
15.001048	0.061909	14.920080	15.056665	4	
16	System emissions		Mt CO default_seeds	4	
14.359162	0.237347	14.076200	14.654884	4	
17	System emissions		Mt CO default_seeds	5	
13.736522	0.286672	13.628600	14.247948	4	
18	System emissions		Mt CO default_seeds	6	
13.351520	0.386541	12.945765	13.888060	4	
19	System emissions		Mt CO default_seeds	7	
12.965221	0.501863	12.287891	13.484884	4	
20	System emissions		Mt CO default_seeds	8	
12.567822	0.607030	11.761139	13.223429	4	
21	System emissions		Mt CO default_seeds	9	
12.204937	0.752304	11.249317	13.009091	4	
22	System emissions		Mt CO default_seeds	10	
11.963831	0.836730	10.791172	12.724990	4	
23	System emissions		Mt CO default_seeds	11	
11.668345	0.839226	10.529618	12.441605	4	
24	System investment spend		M€ default_seeds	0	
617.603760	87.951553	518.251221	725.071960	4	
25	System investment spend		M€ default_seeds	1	
621.070679	66.230843	528.289490	681.375000	4	
26	System investment spend		M€ default_seeds	2	
521.182495	123.150650	437.542053	705.413696	4	
27	System investment spend		M€ default_seeds	3	
485.893585	138.367416	336.043182	635.722168	4	
28	System investment spend		M€ default_seeds	4	
492.050201	124.308479	378.321930	628.262817	4	
29	System investment spend		M€ default_seeds	5	
486.627106	121.997475	390.524963	630.183105	4	
30	System investment spend		M€ default_seeds	6	
478.276978	114.079628	390.119598	614.016235	4	
31	System investment spend		M€ default_seeds	7	
469.758728	89.714790	401.523804	580.422852	4	
32	System investment spend		M€ default_seeds	8	
450.263550	63.101410	393.622955	521.507019	4	
33	System investment spend		M€ default_seeds	9	
432.667053	39.083500	375.947327	463.878601	4	
34	System investment spend		M€ default_seeds	10	
407.595154	45.519211	336.356873	445.003601	4	
35	System investment spend		M€ default_seeds	11	
366.527405	67.147881	286.800781	450.916992	4	

36	MAC fuel-switching reductions			Mt abated	default_seeds	0
0.000000	0.040058	0.000000		0.080117	4	
37	MAC fuel-switching reductions			Mt abated	default_seeds	1
0.000000	0.011245	0.000000		0.022490	4	
38	MAC fuel-switching reductions			Mt abated	default_seeds	2
0.000000	0.171330	0.000000		0.342660	4	
39	MAC fuel-switching reductions			Mt abated	default_seeds	3
0.000000	0.311420	0.000000		0.622840	4	
40	MAC fuel-switching reductions			Mt abated	default_seeds	4
0.026306	0.440485	0.000000		0.897519	4	
41	MAC fuel-switching reductions			Mt abated	default_seeds	5
0.072631	0.430575	0.012724		0.908150	4	
42	MAC fuel-switching reductions			Mt abated	default_seeds	6
0.142870	0.413710	0.033871		0.918606	4	
43	MAC fuel-switching reductions			Mt abated	default_seeds	7
0.232898	0.399209	0.046702		0.930306	4	
44	MAC fuel-switching reductions			Mt abated	default_seeds	8
0.267332	0.418449	0.083599		0.997575	4	
45	MAC fuel-switching reductions			Mt abated	default_seeds	9
0.273330	0.343962	0.077212		0.820644	4	
46	MAC fuel-switching reductions			Mt abated	default_seeds	10
0.264502	0.314181	0.066472		0.749063	4	
47	MAC fuel-switching reductions			Mt abated	default_seeds	11
0.261214	0.258868	0.104360		0.669159	4,	
			panel	unit	sweep	n_seeds
years	mean	median	std	min	max	
0	Mean green generation share	green frac	[0, 1]	default_seeds		4
12	0.702910	0.711464	0.043682	0.616913	0.755758	
1	System emissions			Mt CO	default_seeds	4
12	14.176528	13.544021	2.274590	11.668345	18.883976	
2	System investment spend			M€	default_seeds	4
12	485.793060	482.085266	74.896217	366.527405	621.070679	
3	MAC fuel-switching reductions			Mt abated	default_seeds	4
12	0.128424	0.107751	0.123244	0.000000	0.273330)	

1.9.4 Secondary-market activity over the simulation year

Total secondary volume per year (sum of net buys across agents — by construction equal to the sum of net sells), executed buyer / seller counts vs the corresponding intent counts (a wide gap means agents post uncrossable orders), and the secondary-vs-clearing price spread.

```
[140]: fig, axes = plt.subplots(2, 2, figsize=(14, 8))

def _secondary_volume(yr):
    cols = _agent_cols_in(yr, 'secondary_net')
    if not cols:
        # Fallback: signed trade_qty across agents.
```

```

        cols = _agent_cols_in(yr, 'trade_qty')
        if not cols:
            return None
        # Mean per (year, agent) across episodes, then sum positives across agents.
        by_year = yr.groupby('year')[cols].mean()
        return by_year.clip(lower=0).sum(axis=1)

_overlay_yoy(
    axes[0, 0],
    _per_sweep_year_stack(_secondary_volume),
    ylabel='Mt traded', title='Total secondary volume')

def _intent_exec_gap_buyers(yr):
    if not {'secondary_n_buyers_intent', 'secondary_n_buyers_executed'}.
        issubset(yr.columns):
        return None
    g = yr.groupby('year')[['secondary_n_buyers_intent',
        ↪ 'secondary_n_buyers_executed']].mean()
    return g['secondary_n_buyers_intent'] - g['secondary_n_buyers_executed']

def _intent_exec_gap_sellers(yr):
    if not {'secondary_n_sellers_intent', 'secondary_n_sellers_executed'}.
        issubset(yr.columns):
        return None
    g = yr.groupby('year')[['secondary_n_sellers_intent',
        ↪ 'secondary_n_sellers_executed']].mean()
    return g['secondary_n_sellers_intent'] - g['secondary_n_sellers_executed']

_overlay_yoy(
    axes[0, 1],
    _per_sweep_year_stack(_intent_exec_gap_buyers),
    ylabel='agents (intent - executed)',
    title='Buy-side intent-execution gap')

_overlay_yoy(
    axes[1, 0],
    _per_sweep_year_stack(_intent_exec_gap_sellers),
    ylabel='agents (intent - executed)',
    title='Sell-side intent-execution gap')

def _sec_minus_clearing(yr):
    if not {'secondary_price', 'clearing_price'}.issubset(yr.columns):
        return None
    g = yr.groupby('year')[['secondary_price', 'clearing_price']].mean()
    return g['secondary_price'] - g['clearing_price']

_overlay_yoy(

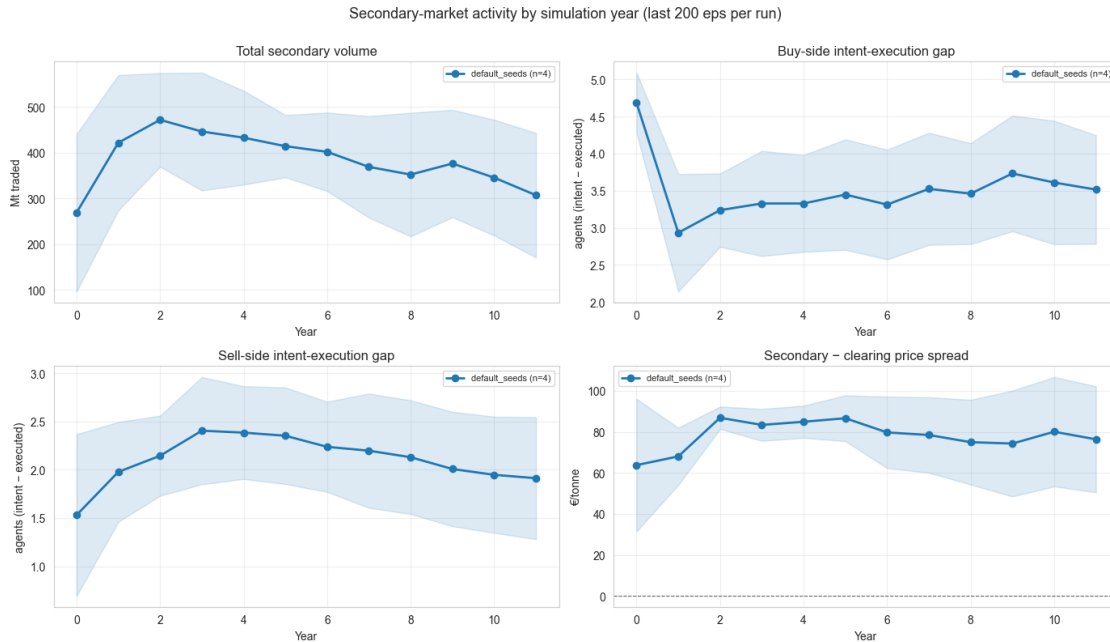
```

```

axes[1, 1],
_per_sweep_year_stack(_sec_minus_clearing),
ylabel='€/tonne', title='Secondary - clearing price spread')
axes[1, 1].axhline(0, color='black', lw=0.8, linestyle='--', alpha=0.6)

plt.suptitle(f'Secondary-market activity by simulation year (last_
↳{YOY_TAIL_EPISODES} eps per run)',
            fontsize=13, y=1.00)
plt.tight_layout(); plt.show()

```



```

[141]: # Companion data tables for the secondary-market activity figure.
def _secondary_volume_tbl(yr):
    cols = _agent_cols_in(yr, 'secondary_net')
    if not cols:
        cols = _agent_cols_in(yr, 'trade_qty')
        if not cols:
            return None
    by_year = yr.groupby('year')[cols].mean()
    return by_year.clip(lower=0).sum(axis=1)

def _intent_exec_gap_buyers_tbl(yr):
    if not {'secondary_n_buyers_intent', 'secondary_n_buyers_executed'}.
↳issubset(yr.columns):
        return None
    g = yr.groupby('year')[['secondary_n_buyers_intent',
↳'secondary_n_buyers_executed']].mean()

```

```

    return g['secondary_n_buyers_intent'] - g['secondary_n_buyers_executed']

def _intent_exec_gap_sellers_tbl(yr):
    if not {'secondary_n_sellers_intent', 'secondary_n_sellers_executed'}.issubset(yr.columns):
        return None
    g = yr.groupby('year')[['secondary_n_sellers_intent', 'secondary_n_sellers_executed']].mean()
    return g['secondary_n_sellers_intent'] - g['secondary_n_sellers_executed']

def _sec_minus_clearing_tbl(yr):
    if not {'secondary_price', 'clearing_price'}.issubset(yr.columns):
        return None
    g = yr.groupby('year')[['secondary_price', 'clearing_price']].mean()
    return g['secondary_price'] - g['clearing_price']

_panels_secondary = [
    ('Total secondary volume',
     _per_sweep_year_stack(_secondary_volume_tbl)),
    ('Buy-side intent-execution gap',
     _per_sweep_year_stack(_intent_exec_gap_buyers_tbl)),
    ('Sell-side intent-execution gap',
     _per_sweep_year_stack(_intent_exec_gap_sellers_tbl)),
    ('Secondary - clearing price spread',
     _per_sweep_year_stack(_sec_minus_clearing_tbl)),
]

_units_secondary = {
    'Total secondary volume': 'Mt traded',
    'Buy-side intent-execution gap': 'agents (intent - executed)',
    'Sell-side intent-execution gap': 'agents (intent - executed)',
    'Secondary - clearing price spread': '€/tonne',
}

_show_yoy_tables(_panels_secondary, units=_units_secondary)

```

Across-year summary (median series across 200-episode tail, per sweep):

				panel		unit	sweep
n_seeds	years	mean	median	std	min	max	
0				Total secondary volume		Mt traded	default_seeds
↪	4	12 383.7	388.9	59.22	268.6	471.5	
1				Buy-side intent-execution gap	agents (intent - executed)		default_seeds
↪	4	12 3.509	3.454	0.4221	2.933	4.685	
2				Sell-side intent-execution gap	agents (intent - executed)		default_seeds
↪	4	12 2.102	2.138	0.2479	1.53	2.405	
3				Secondary - clearing price spread		€/tonne	default_seeds
↪	4	12 78.11	79.08	7.165	63.76	86.85	

Per-year detail (median is the plotted line; std is the band; min/max are cross-

seed extremes):

		panel		unit	sweep
↪ year	median_across_seeds	std_across_seeds	min_across_seeds	↪	
↪ max_across_seeds	n_seeds				
0		Total secondary volume		Mt traded	default_seeds
↪ 0		268.6	172	37.19	442.
↪ 7	4				
1		Total secondary volume		Mt traded	default_seeds
↪ 1		421.5	147.8	211.7	536.
↪ 4	4				
2		Total secondary volume		Mt traded	default_seeds
↪ 2		471.5	102.2	281.4	498.
↪ 8	4				
3		Total secondary volume		Mt traded	default_seeds
↪ 3		446	128.7	203.1	481.
↪ 8	4				
4		Total secondary volume		Mt traded	default_seeds
↪ 4		432.6	102.6	267.1	↪
↪ 508	4				
5		Total secondary volume		Mt traded	default_seeds
↪ 5		414	68.03	374.2	531.
↪ 5	4				
6		Total secondary volume		Mt traded	default_seeds
↪ 6		401.6	85.72	353.7	549.
↪ 1	4				
7		Total secondary volume		Mt traded	default_seeds
↪ 7		368.8	110.8	343.2	580.
↪ 2	4				
8		Total secondary volume		Mt traded	default_seeds
↪ 8		351.9	135	306	602.
↪ 9	4				
9		Total secondary volume		Mt traded	default_seeds
↪ 9		376.1	117.3	252.7	502.
↪ 6	4				
10		Total secondary volume		Mt traded	default_seeds
↪ 10		345.3	126.5	252.2	540.
↪ 1	4				
11		Total secondary volume		Mt traded	default_seeds
↪ 11		306.9	136	215.8	518.
↪ 4	4				
12		Buy-side intent-execution gap	agents (intent - executed)		default_seeds
↪ 0		4.685	0.4073	4.29	5.
↪ 2	4				
13		Buy-side intent-execution gap	agents (intent - executed)		default_seeds
↪ 1		2.933	0.7916	2.245	4.
↪ 035	4				

14	Buy-side intent-execution gap	agents (intent - executed)	default_seeds
↪ 2	3.237	0.4938	2.57
↪66	4		3.
15	Buy-side intent-execution gap	agents (intent - executed)	default_seeds
↪ 3	3.327	0.7089	2.31
↪875	4		3.
16	Buy-side intent-execution gap	agents (intent - executed)	default_seeds
↪ 4	3.327	0.6525	2.305
↪795	4		3.
17	Buy-side intent-execution gap	agents (intent - executed)	default_seeds
↪ 5	3.447	0.7439	2.21
↪82	4		3.
18	Buy-side intent-execution gap	agents (intent - executed)	default_seeds
↪ 6	3.312	0.7403	2.26
↪83	4		3.
19	Buy-side intent-execution gap	agents (intent - executed)	default_seeds
↪ 7	3.525	0.7561	2.475
↪135	4		4.
20	Buy-side intent-execution gap	agents (intent - executed)	default_seeds
↪ 8	3.46	0.6801	2.545
↪01	4		4.
21	Buy-side intent-execution gap	agents (intent - executed)	default_seeds
↪ 9	3.732	0.7799	2.53
↪19	4		4.
22	Buy-side intent-execution gap	agents (intent - executed)	default_seeds
↪ 10	3.61	0.8324	2.69
↪37	4		4.
23	Buy-side intent-execution gap	agents (intent - executed)	default_seeds
↪ 11	3.515	0.7329	2.87
↪205	4		4.
24	Sell-side intent-execution gap	agents (intent - executed)	default_seeds
↪ 0	1.53	0.8381	0.645
↪685	4		2.
25	Sell-side intent-execution gap	agents (intent - executed)	default_seeds
↪ 1	1.978	0.5178	1.415
↪535	4		2.
26	Sell-side intent-execution gap	agents (intent - executed)	default_seeds
↪ 2	2.145	0.4152	1.47
↪365	4		2.
27	Sell-side intent-execution gap	agents (intent - executed)	default_seeds
↪ 3	2.405	0.556	1.315
↪465	4		2.
28	Sell-side intent-execution gap	agents (intent - executed)	default_seeds
↪ 4	2.385	0.481	1.43
↪405	4		2.

29	Sell-side intent-execution gap	agents (intent - executed)	default_seeds	
↪ 5	2.353	0.5011	1.4	2.
↪47	4			
30	Sell-side intent-execution gap	agents (intent - executed)	default_seeds	
↪ 6	2.237	0.4673	1.405	2.
↪46	4			
31	Sell-side intent-execution gap	agents (intent - executed)	default_seeds	
↪ 7	2.198	0.5922	1.08	2.
↪365	4			
32	Sell-side intent-execution gap	agents (intent - executed)	default_seeds	
↪ 8	2.13	0.5898	1.05	2.
↪365	4			
33	Sell-side intent-execution gap	agents (intent - executed)	default_seeds	
↪ 9	2.007	0.5933	1.08	2.
↪475	4			
34	Sell-side intent-execution gap	agents (intent - executed)	default_seeds	
↪ 10	1.948	0.6019	0.91	2.
↪285	4			
35	Sell-side intent-execution gap	agents (intent - executed)	default_seeds	
↪ 11	1.913	0.6328	0.935	2.
↪42	4			
36	Secondary - clearing price spread	€/tonne	default_seeds	
↪ 0	63.76	32.3	11.46	84.
↪77	4			
37	Secondary - clearing price spread	€/tonne	default_seeds	
↪ 1	68.03	13.96	48.22	80.
↪86	4			
38	Secondary - clearing price spread	€/tonne	default_seeds	
↪ 2	86.85	5.468	82.64	93.
↪18	4			
39	Secondary - clearing price spread	€/tonne	default_seeds	
↪ 3	83.38	7.784	75.3	93.
↪79	4			
40	Secondary - clearing price spread	€/tonne	default_seeds	
↪ 4	84.9	7.851	69.39	85.
↪43	4			
41	Secondary - clearing price spread	€/tonne	default_seeds	
↪ 5	86.63	11.15	69.67	95.
↪11	4			
42	Secondary - clearing price spread	€/tonne	default_seeds	
↪ 6	79.71	17.45	60.33	99.
↪78	4			
43	Secondary - clearing price spread	€/tonne	default_seeds	
↪ 7	78.45	18.39	62.36	103.
↪1	4			

44	Secondary - clearing price spread		€/tonne	default_seeds	
↪ 8	74.96	20.66	53.09	101.	
↪1	4				
45	Secondary - clearing price spread		€/tonne	default_seeds	
↪ 9	74.28	25.81	46.56		
↪101	4				
46	Secondary - clearing price spread		€/tonne	default_seeds	
↪ 10	80.05	26.65	38.61	102.	
↪5	4				
47	Secondary - clearing price spread		€/tonne	default_seeds	
↪ 11	76.34	25.88	41.1	102.	
↪6	4				

```
[141]: (
          panel
sweep year median_across_seeds std_across_seeds min_across_seeds
max_across_seeds n_seeds
0 Total secondary volume Mt traded
default_seeds 0 268.602203 172.024414 37.185738
442.727051 4
1 Total secondary volume Mt traded
default_seeds 1 421.497314 147.835022 211.671844
536.402344 4
2 Total secondary volume Mt traded
default_seeds 2 471.514893 102.230995 281.409058
498.821136 4
3 Total secondary volume Mt traded
default_seeds 3 446.031830 128.687424 203.059708
481.792969 4
4 Total secondary volume Mt traded
default_seeds 4 432.608368 102.636169 267.107880
507.962097 4
5 Total secondary volume Mt traded
default_seeds 5 414.044373 68.034584 374.236694
531.484985 4
6 Total secondary volume Mt traded
default_seeds 6 401.641113 85.722954 353.686371
549.122131 4
7 Total secondary volume Mt traded
default_seeds 7 368.759796 110.788620 343.210510
580.227417 4
8 Total secondary volume Mt traded
default_seeds 8 351.926178 135.022491 306.032898
602.941406 4
9 Total secondary volume Mt traded
default_seeds 9 376.149414 117.254730 252.711166
502.553925 4
10 Total secondary volume Mt traded
```

default_seeds	10	345.263092	126.500710	252.157379
540.075623	4			
11	Total secondary volume		Mt traded	
default_seeds	11	306.882111	135.993881	215.815613
518.418945	4			
12	Buy-side intent-execution gap agents (intent - executed)			
default_seeds	0	4.685000	0.407288	4.290000
5.200000	4			
13	Buy-side intent-execution gap agents (intent - executed)			
default_seeds	1	2.932500	0.791564	2.245000
4.035000	4			
14	Buy-side intent-execution gap agents (intent - executed)			
default_seeds	2	3.237500	0.493818	2.570000
3.660000	4			
15	Buy-side intent-execution gap agents (intent - executed)			
default_seeds	3	3.327500	0.708908	2.310000
3.875000	4			
16	Buy-side intent-execution gap agents (intent - executed)			
default_seeds	4	3.327500	0.652526	2.305000
3.795000	4			
17	Buy-side intent-execution gap agents (intent - executed)			
default_seeds	5	3.447500	0.743890	2.210000
3.820000	4			
18	Buy-side intent-execution gap agents (intent - executed)			
default_seeds	6	3.312500	0.740342	2.260000
3.830000	4			
19	Buy-side intent-execution gap agents (intent - executed)			
default_seeds	7	3.525000	0.756086	2.475000
4.135000	4			
20	Buy-side intent-execution gap agents (intent - executed)			
default_seeds	8	3.460000	0.680115	2.545000
4.010000	4			
21	Buy-side intent-execution gap agents (intent - executed)			
default_seeds	9	3.732500	0.779886	2.530000
4.190000	4			
22	Buy-side intent-execution gap agents (intent - executed)			
default_seeds	10	3.610000	0.832436	2.690000
4.370000	4			
23	Buy-side intent-execution gap agents (intent - executed)			
default_seeds	11	3.515000	0.732944	2.870000
4.205000	4			
24	Sell-side intent-execution gap agents (intent - executed)			
default_seeds	0	1.530000	0.838148	0.645000
2.685000	4			
25	Sell-side intent-execution gap agents (intent - executed)			
default_seeds	1	1.977500	0.517758	1.415000
2.535000	4			

26	Sell-side intent-execution gap agents (intent - executed)			
default_seeds	2	2.145000	0.415158	1.470000
2.365000	4			
27	Sell-side intent-execution gap agents (intent - executed)			
default_seeds	3	2.405000	0.555960	1.315000
2.465000	4			
28	Sell-side intent-execution gap agents (intent - executed)			
default_seeds	4	2.385000	0.480995	1.430000
2.405000	4			
29	Sell-side intent-execution gap agents (intent - executed)			
default_seeds	5	2.352500	0.501122	1.400000
2.470000	4			
30	Sell-side intent-execution gap agents (intent - executed)			
default_seeds	6	2.237500	0.467279	1.405000
2.460000	4			
31	Sell-side intent-execution gap agents (intent - executed)			
default_seeds	7	2.197500	0.592242	1.080000
2.365000	4			
32	Sell-side intent-execution gap agents (intent - executed)			
default_seeds	8	2.130000	0.589793	1.050000
2.365000	4			
33	Sell-side intent-execution gap agents (intent - executed)			
default_seeds	9	2.007500	0.593275	1.080000
2.475000	4			
34	Sell-side intent-execution gap agents (intent - executed)			
default_seeds	10	1.947500	0.601893	0.910000
2.285000	4			
35	Sell-side intent-execution gap agents (intent - executed)			
default_seeds	11	1.912500	0.632811	0.935000
2.420000	4			
36	Secondary - clearing price spread		€/tonne	
default_seeds	0	63.759846	32.303665	11.463219
84.765739	4			
37	Secondary - clearing price spread		€/tonne	
default_seeds	1	68.025330	13.962722	48.221443
80.861595	4			
38	Secondary - clearing price spread		€/tonne	
default_seeds	2	86.853210	5.467764	82.639328
93.175354	4			
39	Secondary - clearing price spread		€/tonne	
default_seeds	3	83.376411	7.784280	75.303947
93.785568	4			
40	Secondary - clearing price spread		€/tonne	
default_seeds	4	84.899597	7.850569	69.385895
85.434258	4			
41	Secondary - clearing price spread		€/tonne	
default_seeds	5	86.633881	11.152047	69.672234

95.106766	4						
42	Secondary - clearing price spread					€/tonne	
default_seeds	6	79.714043		17.451290			60.334133
99.781235	4						
43	Secondary - clearing price spread					€/tonne	
default_seeds	7	78.450356		18.390589			62.358330
103.122040	4						
44	Secondary - clearing price spread					€/tonne	
default_seeds	8	74.961197		20.660969			53.093800
101.119141	4						
45	Secondary - clearing price spread					€/tonne	
default_seeds	9	74.277039		25.806784			46.556797
101.007431	4						
46	Secondary - clearing price spread					€/tonne	
default_seeds	10	80.047676		26.648787			38.605736
102.480507	4						
47	Secondary - clearing price spread					€/tonne	
default_seeds	11	76.341850		25.877777			41.102036
102.634521	4,						

				panel		unit	sweep
n_seeds	years	mean	median	std	min	max	
0		Total secondary volume				Mt traded	default_seeds
4	12	383.743408	388.895264	59.218933	268.602203	471.514893	
1		Buy-side intent-execution gap agents (intent - executed)					default_seeds
4	12	3.509375	3.453750	0.422134	2.932500	4.685000	
2		Sell-side intent-execution gap agents (intent - executed)					default_seeds
4	12	2.102292	2.137500	0.247929	1.530000	2.405000	
3		Secondary - clearing price spread				€/tonne	default_seeds
4	12	78.111702	79.082199	7.165179	63.759846	86.853210)	

1.9.5 Reward and reward-component dynamics over the simulation year

System-aggregate per-year reward and its components (`reward_base` = financial – penalty + ESG, `reward_shaping` = curriculum bonuses), plus the cross-agent reward dispersion per year (std across the agent panel), which is a proxy for outcome equity. The per-episode `quality_score` does not have a within-episode year axis and is therefore covered in the convergence panels above; the reward-decomposition panels here are its per-year analogue.

```
[142]: fig, axes = plt.subplots(2, 2, figsize=(14, 8))

_overlay_yoy(
    axes[0, 0],
    _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'reward')),
    ylabel='reward (units of training-time reward)',
    title='System reward')

_overlay_yoy(
```

```

axes[0, 1],
_per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'reward_base')),
ylabel='reward_base', title='System reward_base (financial - penalty + ESG)')

_overlay_yoy(
    axes[1, 0],
    _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'reward_shaping')),
    ylabel='reward_shaping', title='System reward shaping')

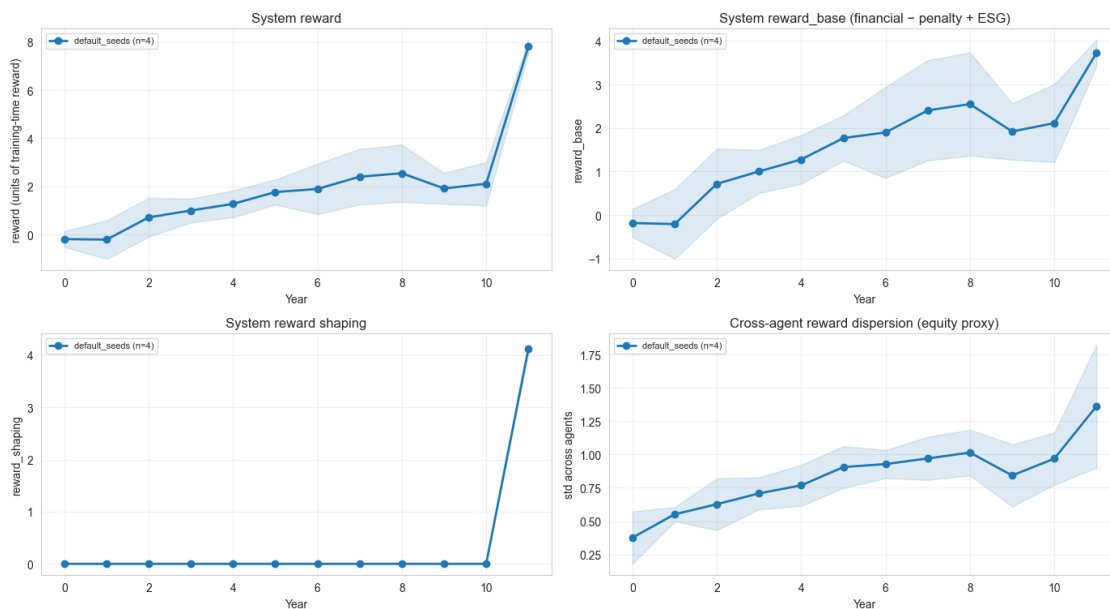
def _reward_dispersion(yr):
    cols = _agent_cols_in(yr, 'reward')
    if not cols:
        return None
    # Per-year mean across episodes per agent, then std across agents.
    by_year = yr.groupby('year')[cols].mean()
    return by_year.std(axis=1)

_overlay_yoy(
    axes[1, 1],
    _per_sweep_year_stack(_reward_dispersion),
    ylabel='std across agents',
    title='Cross-agent reward dispersion (equity proxy)')

plt.suptitle(f'Reward and reward components by simulation year (last {YOY_TAIL_EPISODES} eps per run)',
            fontsize=13, y=1.00)
plt.tight_layout(); plt.show()

```

Reward and reward components by simulation year (last 200 eps per run)



```
[143]: # Companion data tables for the reward-dynamics figure.
def _reward_dispersion_tbl(yr):
    cols = _agent_cols_in(yr, 'reward')
    if not cols:
        return None
    by_year = yr.groupby('year')[cols].mean()
    return by_year.std(axis=1)

_panels_reward = [
    ('System reward',
     _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'reward'))),
    ('System reward_base (financial - penalty + ESG)',
     _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'reward_base'))),
    ('System reward shaping',
     _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr,
↵'reward_shaping'))),
    ('Cross-agent reward dispersion (equity proxy)',
     _per_sweep_year_stack(_reward_dispersion_tbl)),
]
_units_reward = {
    'System reward': 'reward (training-time units)',
    'System reward_base (financial - penalty + ESG)': 'reward_base',
    'System reward shaping': 'reward_shaping',
    'Cross-agent reward dispersion (equity proxy)': 'std across agents',
}
_show_yoy_tables(_panels_reward, units=_units_reward)
```

Across-year summary (median series across 200-episode tail, per sweep):

					panel				unit
↵	sweep	n_seeds	years	mean	median	std	min	max	
0					System reward	reward (training-time units)			↵
↵	default_seeds	4	12	1.921	1.832	2.066	-0.2081	7.804	
1	System reward_base (financial - penalty + ESG)								reward_base ↵
↵	default_seeds	4	12	1.581	1.832	1.137	-0.2081	3.724	
2					System reward shaping	reward_shaping			↵
↵	default_seeds	4	12	0.343	0	1.188	0	4.117	
3	Cross-agent reward dispersion (equity proxy)								std across agents ↵
↵	default_seeds	4	12	0.8353	0.8742	0.2549	0.376	1.362	

Per-year detail (median is the plotted line; std is the band; min/max are cross-seed extremes):

					panel		unit
↵	sweep	year	median_across_seeds	std_across_seeds	min_across_seeds		↵
↵	max_across_seeds	n_seeds					

0				System reward	reward (training-time units)	
↪	default_seeds	0		-0.1847	0.3278	-0.7167
↪		-0.01428	4			
1				System reward	reward (training-time units)	
↪	default_seeds	1		-0.2081	0.8009	-1.085
↪		0.5331	4			
2				System reward	reward (training-time units)	
↪	default_seeds	2		0.717	0.8078	-0.6435
↪		1.142	4			
3				System reward	reward (training-time units)	
↪	default_seeds	3		1.001	0.4941	0.2257
↪		1.377	4			
4				System reward	reward (training-time units)	
↪	default_seeds	4		1.277	0.5591	0.4711
↪		1.736	4			
5				System reward	reward (training-time units)	
↪	default_seeds	5		1.767	0.5203	1.029
↪		2.256	4			
6				System reward	reward (training-time units)	
↪	default_seeds	6		1.897	1.047	0.7994
↪		3.165	4			
7				System reward	reward (training-time units)	
↪	default_seeds	7		2.404	1.15	0.6668
↪		3.282	4			
8				System reward	reward (training-time units)	
↪	default_seeds	8		2.548	1.186	0.8529
↪		3.354	4			
9				System reward	reward (training-time units)	
↪	default_seeds	9		1.918	0.646	1.257
↪		2.637	4			
10				System reward	reward (training-time units)	
↪	default_seeds	10		2.111	0.8976	0.8775
↪		2.761	4			
11				System reward	reward (training-time units)	
↪	default_seeds	11		7.804	0.2873	7.408
↪		8.1	4			
12	System reward_base (financial - penalty + ESG)					reward_base
↪	default_seeds	0		-0.1847	0.3278	-0.7167
↪		-0.01428	4			
13	System reward_base (financial - penalty + ESG)					reward_base
↪	default_seeds	1		-0.2081	0.8009	-1.085
↪		0.5331	4			
14	System reward_base (financial - penalty + ESG)					reward_base
↪	default_seeds	2		0.717	0.8078	-0.6435
↪		1.142	4			

15	System reward_base (financial - penalty + ESG)					reward_base
↪	default_seeds	3	1.001	0.4941	0.2257	↪
↪		1.377	4			
16	System reward_base (financial - penalty + ESG)					reward_base
↪	default_seeds	4	1.277	0.5591	0.4711	↪
↪		1.736	4			
17	System reward_base (financial - penalty + ESG)					reward_base
↪	default_seeds	5	1.767	0.5203	1.029	↪
↪		2.256	4			
18	System reward_base (financial - penalty + ESG)					reward_base
↪	default_seeds	6	1.897	1.047	0.7994	↪
↪		3.165	4			
19	System reward_base (financial - penalty + ESG)					reward_base
↪	default_seeds	7	2.404	1.15	0.6668	↪
↪		3.282	4			
20	System reward_base (financial - penalty + ESG)					reward_base
↪	default_seeds	8	2.548	1.186	0.8529	↪
↪		3.354	4			
21	System reward_base (financial - penalty + ESG)					reward_base
↪	default_seeds	9	1.918	0.646	1.257	↪
↪		2.637	4			
22	System reward_base (financial - penalty + ESG)					reward_base
↪	default_seeds	10	2.111	0.8976	0.8775	↪
↪		2.761	4			
23	System reward_base (financial - penalty + ESG)					reward_base
↪	default_seeds	11	3.724	0.3017	3.276	↪
↪		3.999	4			
24	System reward shaping					reward_shaping
↪	default_seeds	0	0	0	0	↪
↪		0	4			
25	System reward shaping					reward_shaping
↪	default_seeds	1	0	0	0	↪
↪		0	4			
26	System reward shaping					reward_shaping
↪	default_seeds	2	0	0	0	↪
↪		0	4			
27	System reward shaping					reward_shaping
↪	default_seeds	3	0	0	0	↪
↪		0	4			
28	System reward shaping					reward_shaping
↪	default_seeds	4	0	0	0	↪
↪		0	4			
29	System reward shaping					reward_shaping
↪	default_seeds	5	0	0	0	↪
↪		0	4			

30			System reward shaping		reward_shaping	␣
↪	default_seeds	6	0	0	0	␣
↪		0	4			
31			System reward shaping		reward_shaping	␣
↪	default_seeds	7	0	0	0	␣
↪		0	4			
32			System reward shaping		reward_shaping	␣
↪	default_seeds	8	0	0	0	␣
↪		0	4			
33			System reward shaping		reward_shaping	␣
↪	default_seeds	9	0	0	0	␣
↪		0	4			
34			System reward shaping		reward_shaping	␣
↪	default_seeds	10	0	0	0	␣
↪		0	4			
35			System reward shaping		reward_shaping	␣
↪	default_seeds	11	4.117	0.08341	3.982	␣
↪		4.177	4			
36	Cross-agent reward dispersion (equity proxy)				std across agents	␣
↪	default_seeds	0	0.376	0.1966	0.09554	␣
↪		0.5218	4			
37	Cross-agent reward dispersion (equity proxy)				std across agents	␣
↪	default_seeds	1	0.5516	0.05447	0.5114	␣
↪		0.6385	4			
38	Cross-agent reward dispersion (equity proxy)				std across agents	␣
↪	default_seeds	2	0.6269	0.1946	0.5406	␣
↪		0.9529	4			
39	Cross-agent reward dispersion (equity proxy)				std across agents	␣
↪	default_seeds	3	0.7083	0.1206	0.6695	␣
↪		0.932	4			
40	Cross-agent reward dispersion (equity proxy)				std across agents	␣
↪	default_seeds	4	0.7685	0.154	0.6455	␣
↪		1.012	4			
41	Cross-agent reward dispersion (equity proxy)				std across agents	␣
↪	default_seeds	5	0.9057	0.1573	0.6937	␣
↪		1.078	4			
42	Cross-agent reward dispersion (equity proxy)				std across agents	␣
↪	default_seeds	6	0.9279	0.1057	0.7781	␣
↪		1.006	4			
43	Cross-agent reward dispersion (equity proxy)				std across agents	␣
↪	default_seeds	7	0.9705	0.1617	0.7619	␣
↪		1.156	4			
44	Cross-agent reward dispersion (equity proxy)				std across agents	␣
↪	default_seeds	8	1.014	0.1713	0.7139	␣
↪		1.105	4			

45	Cross-agent reward dispersion (equity proxy)			std across agents	
↪	default_seeds	9	0.8427	0.2332	0.7132
↪		1.236	4		
46	Cross-agent reward dispersion (equity proxy)			std across agents	
↪	default_seeds	10	0.9688	0.1956	0.6517
↪		1.112	4		
47	Cross-agent reward dispersion (equity proxy)			std across agents	
↪	default_seeds	11	1.362	0.4618	0.6443
↪		1.748	4		

```
[143]: (
    unit sweep year median_across_seeds std_across_seeds
min_across_seeds max_across_seeds n_seeds
0 System reward reward (training-time
units) default_seeds 0 -0.184690 0.327837
-0.716697 -0.014279 4
1 System reward reward (training-time
units) default_seeds 1 -0.208070 0.800853
-1.084718 0.533110 4
2 System reward reward (training-time
units) default_seeds 2 0.717044 0.807827
-0.643519 1.141691 4
3 System reward reward (training-time
units) default_seeds 3 1.001291 0.494138
0.225725 1.376533 4
4 System reward reward (training-time
units) default_seeds 4 1.276917 0.559057
0.471130 1.736027 4
5 System reward reward (training-time
units) default_seeds 5 1.766627 0.520271
1.029453 2.256327 4
6 System reward reward (training-time
units) default_seeds 6 1.896862 1.047184
0.799436 3.165222 4
7 System reward reward (training-time
units) default_seeds 7 2.403752 1.150350
0.666794 3.282076 4
8 System reward reward (training-time
units) default_seeds 8 2.547818 1.186090
0.852883 3.354256 4
9 System reward reward (training-time
units) default_seeds 9 1.918330 0.645956
1.256838 2.636700 4
10 System reward reward (training-time
units) default_seeds 10 2.110819 0.897649
0.877542 2.760867 4
11 System reward reward (training-time
```

units)	default_seeds	11	7.803879	0.287345
7.407753	8.100058	4		
12	System reward_base (financial - penalty + ESG)			
reward_base	default_seeds	0	-0.184690	0.327837
-0.716697	-0.014279	4		
13	System reward_base (financial - penalty + ESG)			
reward_base	default_seeds	1	-0.208070	0.800853
-1.084718	0.533110	4		
14	System reward_base (financial - penalty + ESG)			
reward_base	default_seeds	2	0.717044	0.807827
-0.643519	1.141691	4		
15	System reward_base (financial - penalty + ESG)			
reward_base	default_seeds	3	1.001291	0.494138
0.225725	1.376533	4		
16	System reward_base (financial - penalty + ESG)			
reward_base	default_seeds	4	1.276917	0.559057
0.471130	1.736027	4		
17	System reward_base (financial - penalty + ESG)			
reward_base	default_seeds	5	1.766627	0.520271
1.029453	2.256327	4		
18	System reward_base (financial - penalty + ESG)			
reward_base	default_seeds	6	1.896862	1.047184
0.799436	3.165222	4		
19	System reward_base (financial - penalty + ESG)			
reward_base	default_seeds	7	2.403752	1.150350
0.666794	3.282076	4		
20	System reward_base (financial - penalty + ESG)			
reward_base	default_seeds	8	2.547818	1.186090
0.852883	3.354256	4		
21	System reward_base (financial - penalty + ESG)			
reward_base	default_seeds	9	1.918330	0.645956
1.256838	2.636700	4		
22	System reward_base (financial - penalty + ESG)			
reward_base	default_seeds	10	2.110819	0.897649
0.877542	2.760867	4		
23	System reward_base (financial - penalty + ESG)			
reward_base	default_seeds	11	3.724315	0.301708
3.275746	3.998940	4		
24	System reward shaping			
reward_shaping	default_seeds	0	0.000000	0.000000
0.000000	0.000000	4		
25	System reward shaping			
reward_shaping	default_seeds	1	0.000000	0.000000
0.000000	0.000000	4		
26	System reward shaping			
reward_shaping	default_seeds	2	0.000000	0.000000
0.000000	0.000000	4		

27			System reward shaping	
reward_shaping	default_seeds	3	0.000000	0.000000
0.000000	0.000000	4		
28			System reward shaping	
reward_shaping	default_seeds	4	0.000000	0.000000
0.000000	0.000000	4		
29			System reward shaping	
reward_shaping	default_seeds	5	0.000000	0.000000
0.000000	0.000000	4		
30			System reward shaping	
reward_shaping	default_seeds	6	0.000000	0.000000
0.000000	0.000000	4		
31			System reward shaping	
reward_shaping	default_seeds	7	0.000000	0.000000
0.000000	0.000000	4		
32			System reward shaping	
reward_shaping	default_seeds	8	0.000000	0.000000
0.000000	0.000000	4		
33			System reward shaping	
reward_shaping	default_seeds	9	0.000000	0.000000
0.000000	0.000000	4		
34			System reward shaping	
reward_shaping	default_seeds	10	0.000000	0.000000
0.000000	0.000000	4		
35			System reward shaping	
reward_shaping	default_seeds	11	4.116562	0.083411
3.982029	4.177100	4		
36	Cross-agent reward dispersion (equity proxy)			std across
agents	default_seeds	0	0.376046	0.196597
0.095541	0.521817	4		
37	Cross-agent reward dispersion (equity proxy)			std across
agents	default_seeds	1	0.551597	0.054467
0.511405	0.638525	4		
38	Cross-agent reward dispersion (equity proxy)			std across
agents	default_seeds	2	0.626919	0.194554
0.540569	0.952940	4		
39	Cross-agent reward dispersion (equity proxy)			std across
agents	default_seeds	3	0.708252	0.120564
0.669465	0.932038	4		
40	Cross-agent reward dispersion (equity proxy)			std across
agents	default_seeds	4	0.768479	0.154002
0.645491	1.012454	4		
41	Cross-agent reward dispersion (equity proxy)			std across
agents	default_seeds	5	0.905707	0.157303
0.693749	1.077559	4		
42	Cross-agent reward dispersion (equity proxy)			std across
agents	default_seeds	6	0.927905	0.105733

0.778062	1.006343	4						
43	Cross-agent reward dispersion (equity proxy)					std across		
agents	default_seeds	7		0.970466		0.161722		
0.761923	1.155962	4						
44	Cross-agent reward dispersion (equity proxy)					std across		
agents	default_seeds	8		1.014208		0.171293		
0.713875	1.104995	4						
45	Cross-agent reward dispersion (equity proxy)					std across		
agents	default_seeds	9		0.842725		0.233213		
0.713193	1.235877	4						
46	Cross-agent reward dispersion (equity proxy)					std across		
agents	default_seeds	10		0.968814		0.195552		
0.651708	1.111633	4						
47	Cross-agent reward dispersion (equity proxy)					std across		
agents	default_seeds	11		1.362413		0.461762		
0.644280	1.747810	4,						
					panel			unit
sweep	n_seeds	years	mean	median	std	min	max	
0					System reward	reward (training-time units)		
default_seeds	4	12	1.920882	1.831744	2.066437	-0.208070	7.803879	
1					System reward_base (financial - penalty + ESG)	reward_base		
default_seeds	4	12	1.580918	1.831744	1.137284	-0.208070	3.724315	
2					System reward shaping	reward_shaping		
default_seeds	4	12	0.343047	0.000000	1.188349	0.000000	4.116562	
3					Cross-agent reward dispersion (equity proxy)	std across agents		
default_seeds	4	12	0.835294	0.874216	0.254946	0.376046	1.362413	

Reading these panels. Each coloured line is one sweep’s cross-seed median, with a ± 1 std fan when the sweep contains more than one seed. Vertical separation between coloured medians is the cross-sweep effect; band overlap indicates that the sweeps are indistinguishable at the loaded sample size. Because every series is drawn from the converged tail, year-to-year structure here reflects the *shape* of the steady-state policy, not transient learning dynamics.

1.10 What this notebook does not answer

- Cross-config sensitivity — needs sweep variants. *Out of scope here.*
- Causal attribution of regulatory mechanisms (e.g. counterfactual “what if MSR were off”) — needs sweep variants.
- Real-world EU-ETS empirical validation beyond the qualitative price-band check in RQ1.3.
- Strategy clustering, theory tests (LCOE crossover, archetype heterogeneity), and predictive models of clearing price — these are in the analysis plan (planning/data_science_analysis_plan.md §B1, B2, B4) and will get their own notebooks where they can be evaluated cleanly with cross-validation.

The default-config evidence base assembled above is the *baseline* that those follow-on analyses will perturb and test against.